Yahtzee in C#

F20SC Industrial Programming · Coursework 1
Code Guide, Architecture and Development Log

This guide explains **every file and every line** of the code written so far, the design behind it, and every step taken to build and verify it. It is meant to help you understand, explain and continue the project (including in a possible in-person demonstration).

Status on 8 October 2026: all 7 implementation steps are complete, but the submission is not finished yet.
Done: every feature in the specification's checklist, 130 passing unit tests, an end-to-end UI test, and the standalone `publish\Yahtzee.exe`.
Still to do (your work): the report (PDF, max 10 pages), the narrated video, testing the exe on a lab machine, and your evaluation notes in the GenAI log. See section 1 and section 13.

> **This is not your coursework report.** The specification requires a report (max 10 pages) written and evaluated by you, with a GenAI usage statement and your prompts and the model's responses in an appendix. You can use this guide as source material while writing it, but submitting it as the report would not meet the specification.

Generated with Claude Code (Claude Opus 5.5) by `docs/guide-src/build_guide.ps1`.
Source listings are read directly from the repository's files each time the PDF is built, so they always match the code.

Contents

1. Is the project complete? Status against the specification
2. The assignment in brief
3. Development log: every step performed
4. Building, running and testing
5. Architecture and design (including save files, database, AI)
6. The scoring rules, illustrated
7. The user interface
8. C# language features used

How to read the line-by-line walkthrough

9. Walkthrough: Build and project files
    - 9.1 Solution file (Yahtzee.slnx)
    - 9.2 Shared MSBuild settings (Directory.Build.props)
    - 9.3 Files excluded from version control (.gitignore)
    - 9.4 Core library project (src/Yahtzee.Core/Yahtzee.Core.csproj)
    - 9.5 Windows Forms application project (src/Yahtzee.WinForms/Yahtzee.WinForms.csproj)
    - 9.6 Unit-test project (tests/Yahtzee.Core.Tests/Yahtzee.Core.Tests.csproj)
    - 9.7 Publish profile for the submission executable (src/Yahtzee.WinForms/Properties/PublishProfiles/Standalone.pubxml)
10. Walkthrough: Yahtzee.Core: the game logic
    - 10.1 The 13 scorecard categories (src/Yahtzee.Core/Model/Category.cs)
    - 10.2 Scoring rules (src/Yahtzee.Core/Rules/Scorer.cs)
    - 10.3 Random-number abstraction (src/Yahtzee.Core/Model/IDiceRoller.cs)
    - 10.4 The five dice and their held flags (src/Yahtzee.Core/Model/DiceCup.cs)
    - 10.5 One player's scorecard (src/Yahtzee.Core/Model/ScoreCard.cs)
    - 10.6 A player (src/Yahtzee.Core/Model/Player.cs)
    - 10.7 AI strategy contract (src/Yahtzee.Core/AI/IPlayerStrategy.cs)
    - 10.8 Event data types (src/Yahtzee.Core/Game/GameEvents.cs)
    - 10.9 The game engine (state machine) (src/Yahtzee.Core/Game/GameEngine.cs)
    - 10.10 Saving and restoring games (engine side) (src/Yahtzee.Core/Game/GameEngine.Persistence.cs)
    - 10.11 Save-file data types (src/Yahtzee.Core/Persistence/GameSnapshot.cs)
    - 10.12 Save-file error type (src/Yahtzee.Core/Persistence/InvalidSaveFileException.cs)
    - 10.13 Save-file reading and writing (src/Yahtzee.Core/Persistence/SaveGameSerializer.cs)
    - 10.14 Games-history interface (src/Yahtzee.Core/Data/IGameHistoryRepository.cs)
    - 10.15 SQLite games history (src/Yahtzee.Core/Data/SqliteGameHistoryRepository.cs)
    - 10.16 Shared AI judgement (src/Yahtzee.Core/AI/ScoringHeuristics.cs)
    - 10.17 Easy computer player (src/Yahtzee.Core/AI/RandomStrategy.cs)
    - 10.18 Medium computer player (src/Yahtzee.Core/AI/GreedyStrategy.cs)
    - 10.19 Hard computer player (src/Yahtzee.Core/AI/ExpectedValueStrategy.cs)
    - 10.20 Strategy registry (src/Yahtzee.Core/AI/StrategyCatalog.cs)
11. Walkthrough: Yahtzee.WinForms: the user interface
    - 11.1 Application entry point (src/Yahtzee.WinForms/Program.cs)
    - 11.2 Shared colours and fonts (src/Yahtzee.WinForms/UiTheme.cs)
    - 11.3 Custom-drawn die (src/Yahtzee.WinForms/Controls/DieControl.cs)
    - 11.4 Player type choices (src/Yahtzee.WinForms/Forms/PlayerTypeOption.cs)
    - 11.5 New-game dialog (src/Yahtzee.WinForms/Forms/NewGameDialog.cs)
    - 11.6 End-of-game results (src/Yahtzee.WinForms/Forms/ResultsForm.cs)
    - 11.7 High-score tables (src/Yahtzee.WinForms/Forms/HighScoresForm.cs)
    - 11.8 The main game window (src/Yahtzee.WinForms/Forms/MainForm.cs)
12. Walkthrough: Yahtzee.Core.Tests: the unit tests
    - 12.1 Scripted dice for tests (tests/Yahtzee.Core.Tests/FixedDiceRoller.cs)
    - 12.2 Other test doubles (tests/Yahtzee.Core.Tests/TestDoubles.cs)
    - 12.3 Scoring rule tests (tests/Yahtzee.Core.Tests/ScorerTests.cs)
    - 12.4 Scorecard tests (tests/Yahtzee.Core.Tests/ScoreCardTests.cs)
    - 12.5 Dice cup tests (tests/Yahtzee.Core.Tests/DiceCupTests.cs)

12.6 Player tests (tests/Yahtzee.Core.Tests/PlayerTests.cs)
12.7 Game engine tests (tests/Yahtzee.Core.Tests/GameEngineTests.cs)
12.8 Save-file tests (tests/Yahtzee.Core.Tests/SaveGameTests.cs)
12.9 Database tests (tests/Yahtzee.Core.Tests/GameHistoryRepositoryTests.cs)
12.10 AI tests (tests/Yahtzee.Core.Tests/AiStrategyTests.cs)
13. What remains: steps 4–7, report, video and submission
14. Glossary

1. Is the project complete? Status against the specification

The program is complete: every feature in the specification's checklist is implemented, unit tested, and was exercised end to end through the real user interface of the published executable. **The submission is not complete yet:** the report, the narrated video and a test on a lab machine are your work.

Feature checklist (the exact rows required by Appendix A of the specification)

Feature	Status	Where / how it was verified
Single-player gameplay	fully implemented	Full 13-round game played through the UI.
Multi-player gameplay	fully implemented	Hotseat, up to 6 players (any mix of humans and computers).
Specifying player names	fully implemented	New-game dialog; blank/duplicate names rejected.
Displaying whose turn it is	fully implemented	Heading plus highlighted scorecard column.
Displaying the number of rolls available	fully implemented	"Rolls left: n of 3".
Indicating which dice will be rolled/set aside	fully implemented	Gold HELD dice vs WILL ROLL captions; roll button counts dice.
Score preview	fully implemented	"This roll" column after every roll.
Saving/loading games	fully implemented	Save after any roll (Ctrl+S), JSON `.yahtzee` file, validated on load; AI turns resume.
Offer to save the game before exiting	fully implemented	Yes/No/Cancel *only when there are unsaved changes* (also before New Game / Open).
Result summary at the end of the game	fully implemented	Results dialog, ordered by score, ties share a rank.
Games database	fully implemented	SQLite; every finished game recorded; top 5 overall and per player, with dates.
AI opponents	fully implemented	Three strategies (easy/medium/hard); averages 39 / 176 / 228 points.

Deliverables required for submission (section 6 of the specification)

Deliverable	Status	Notes
Complete source code	done	Zip without `bin/`, `obj/`, `.vs/`, `publish/` build folders.
Standalone executable (Windows 11)	built	`publish\Yahtzee.exe` (53 MB, needs no .NET). Passed the end-to-end UI test here; **still test it on a lab machine**.
Report (PDF, max 10 pages, 7 sections)	your work	Must be written and evaluated by you; see section 13.
Video with voice-over showing every feature	your work	**Mandatory**: without it the submission is incomplete and may score 0. Features not shown count as not implemented.

Marks (from the marking scheme)

Criterion	Marks	Covered by the program?
Single-player gameplay	5	yes
Multi-player gameplay	10	yes
Score preview	5	yes
Saving/loading game	10	yes
GUI quality	5	yes (judged by the marker)
Games database	15	yes

AI opponents	10	**yes, with three strategies**
Code organisation and quality	20	**layered design, 130 tests, XML docs, modern C# (judged by the marker)**
Report quality	15	**your report**
Video demonstration	5	**your video**

> Every feature only counts if it is **shown in your video**. Deadline: **29 October 2026**.

2. The assignment in brief

The specification (`F20SC_Coursework1.pdf`, 6 pages) asks for an **individual** implementation of a simplified Yahtzee in C# with a graphical user interface, targeting Visual Studio 2026 on Windows 11. The key points that shaped the design:

Area	Requirement (MoSCoW wording from the spec)
GUI	**Must** provide a GUI for starting, playing, saving and loading games.
Players	**Must** allow single-player; **should** allow hotseat multi-player (a player limit is allowed); **must** let users name each player.
During a turn	**Must** show whose turn it is and how many rolls remain; **must** show which dice will be rolled and which are set aside; **should** show a score preview for every available category after each roll.
Saving	**Should** allow saving after any roll with a self-designed save-file format; **should** offer to save when exiting an unsaved game.
End of game	**Must** display names and scores ordered by score.
Database	**Should** record completed games (date, names, scores) using a database engine; **should** show the top 5 overall and the top 5 of a player; **could** show the date of each high score.
AI	**Should** allow computer opponents (even trivial); **could** offer different strategies.
Rules (Appendix B)	13 rounds; up to 3 rolls; hold any dice between rolls; 13 categories each used once; non-matching dice score 0; total = sum of the 13 boxes; **a Yahtzee is not a valid full house**; no bonuses.
Submission	Zip with source, standalone executable and report PDF; narrated video (mandatory); possible in-person demo.
GenAI policy	Level 6: GenAI may be used, but you must evaluate its output yourself, be able to explain all code, and include your prompts and the model's responses in the report's appendix.

3. Development log: every step performed

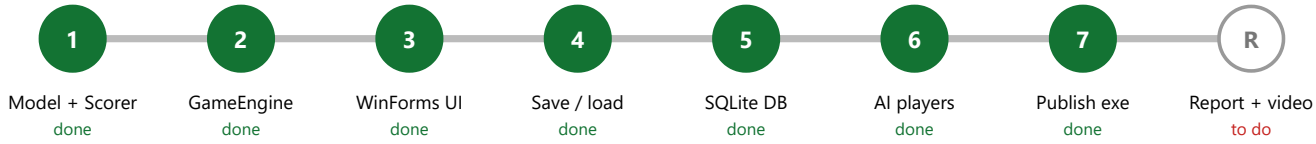


Figure 1. The seven implementation steps from the plan in CLAUDE.md, plus the report and video.

The work was done in conversation sessions with Claude Code on 6–7 October 2026. Each prompt is also recorded in `docs/genai-log.md` for the GenAI appendix. Below, everything that was done, including problems and how they were fixed.

Session 1: reading the specification and planning (prompt: */init … read the pdf and implement a plan …*)

1. Listed the project folder: it contained only `F20SC_Coursework1.pdf`; it was not a git repository.
2. Read all six pages of the PDF: overview, lab environment, functional requirements (3.1 basic, 3.2 advanced), report format, video format, submission and marking scheme, GenAI policy, academic integrity, Appendix A (feature checklist) and Appendix B (simplified rules).
3. Checked for a .NET SDK with `dotnet --list-sdks`: **none installed**.
4. Chose the technology: .NET 10 (LTS, ships with VS 2026), Windows Forms (recommended by the spec and the simplest), JSON save files (`System.Text.Json`, built in), SQLite via `Microsoft.Data.Sqlite` (a real database engine with no server), xUnit for tests.
5. Wrote `CLAUDE.md`: project context, planned commands, layered architecture, design rules, the simplified scoring rules, the planned save-file format and database schema, a requirement-to-class map, the 7-step order of work, and the submission format.
6. Pointed out that the specification says the coursework is an **individual** project (the request had called it a group project).

Session 2: step 1, the model and the scoring rules (prompt: *Proceed with step 1.*)

1. Created `Yahtzee.slnx`, `Directory.Build.props` (latest C#, nullable, implicit usings, warnings as errors) and `.gitignore`.
2. Created the `Yahtzee.Core` class library with XML documentation generation, then wrote `Category`, `Scorer`, `IDiceRoller`/`RandomDiceRoller`, `DiceCup` and `ScoreCard`.
3. Created the xUnit test project with `FixedDiceRoller` and the tests `ScorerTests`, `ScoreCardTests`, `DiceCupTests`.
4. Replaced collection expressions inside `Assert.Equal(...)` calls with `new[] { ... }` arrays, because collection expressions there can make the overload choice ambiguous.
5. Installed the .NET 10 SDK (10.0.401) **temporarily**, into the session's scratch folder (not system-wide), using Microsoft's `dotnet-install.ps1` script.
6. Ran `dotnet test`: build clean, **57 tests passed**.
7. Updated `CLAUDE.md` and created `docs/genai-log.md`.

Session 3: step 2, the game engine (prompt: *proceed with step 2.*)

1. Wrote `IPlayerStrategy` (the AI contract), `Player`, `GameEvents` (event records) and `GameEngine`.
2. Design decisions: the round number is *derived* rather than stored; `CanRoll`/`CanHold`/`CanScore` guard every action; a `StateChanged` event after every change; `PlayAiStep()` performs one visible AI action at a time; time comes from an injected `TimeProvider`.
3. Added an explicit index check to `SetHeld` so a bad index gives `ArgumentOutOfRangeException` rather than an indexing error.
4. Wrote `TestDoubles.cs`, `PlayerTests` and `GameEngineTests`.
5. Fixed problems found by the build: one test scripted 10 die values but needed 11 (the third AI roll re-rolls three dice); the xUnit analyzer rule **xUnit2013** rejected `Assert.Equal(1, collection.Count)`, replaced with `Assert.Single(...)`.
6. Result: **88 tests passed**. Updated `CLAUDE.md` and the GenAI log (and fixed the order of two log rows).

Session 4: step 3, the Windows Forms interface (prompts: *start with step 3. / continue*)

1. Created `Yahtzee.WinForms` (WinExe, `net10.0-windows`, PerMonitorV2 DPI, output `Yahtzee.exe`) and added it to the solution.
2. Wrote `Program`, `UiTheme`, `DieControl`, `PlayerTypeOption`, `NewGameDialog`, `ResultsForm` and `MainForm`. Forms are built in code (no designer files) with layout panels.
3. Fixed compile issues before building: a `(int)` cast for `Keys` arithmetic and a missing `using` in `Program.cs`.

4. The first build attempt was interrupted (the tool permission prompt closed); the next day the build succeeded with **0 warnings, 0 errors**.
5. **Verified the real app** by driving it with Windows UI Automation from PowerShell scripts and taking screenshots: welcome screen, new-game dialog, a roll with two held dice, the zero-score confirmation, a complete 13-round single-player game, the results dialog and the final board.
6. Testing found two **display-scaling (DPI) bugs** on this 150% display: scorecard rows and headers were not scaled (fixed with `LogicalToDeviceUnits` when the grid is built), and the results list's column widths left an empty strip (fixed by sizing columns in the `Load` event). Both were re-tested.
7. Re-ran all tests (88 passing). Updated `CLAUDE.md` (WinForms conventions, DPI notes) and the GenAI log.

Session 5: this guide (prompt: *Is the project complete … prepare a PDF …*)

1. Answered the status question (section 1).
2. Retook the screenshots after the DPI fixes. While doing so, the first version of the screenshot script found its window by the title "Yahtzee". Because you had your own copy of the game open, two runs may have sent `Ctrl+N` and a "Start Game" click to **your** window and started a new two-player game in it. The script was changed to only control the process it starts itself.
3. Generated the first version of this document: a Python script reads every source file, attaches explanations to specific source lines (the build stops if a line it refers to cannot be found, so explanations cannot silently drift from the code), adds diagrams and screenshots, and Microsoft Edge prints it to PDF.

Session 6: steps 4–7 and the guide generator (prompt: *okay continue with next steps and also add the pdf script*)

1. **Step 4, saving and loading.** Wrote `GameSnapshot`/`PlayerSnapshot`, `SaveGameSerializer` (strict JSON, versioned, safe write via a temporary file), `InvalidSaveFileException`, and `GameEngine.Persistence.cs` (`CreateSnapshot`, a validating `Restore`). The engine became a `partial` class with a private shared constructor. 21 tests (109 in total) passed first time.
2. **Step 5, database.** Added `Microsoft.Data.Sqlite` 10.0.12 (latest stable, checked on NuGet), `IGameHistoryRepository` and `SqliteGameHistoryRepository` (schema created on first use, one transaction per game, parameterised queries). 7 tests including an SQL-injection attempt.
3. **Step 6, AI.** `RandomStrategy`, `GreedyStrategy`, `ExpectedValueStrategy`, `StrategyCatalog` and `ScoringHeuristics`. 14 tests, including 30 seeded games per strategy. Fixes: an `ITestOutputHelper` namespace import, and a test comment that described the wrong dice. Result: **130 tests passing**.
4. **User interface.** Game menu (Open, Save, Save As, High Scores), unsaved-changes tracking with a change counter, the Save/Don't save/Cancel prompt, `ShowModal` (pauses computer players during dialogs), recording finished games, `HighScoresForm`, computer types in the new-game dialog (automatically renamed "Computer n"), and a `--database` command-line option so automated tests never touch the real high scores.
5. **End-to-end UI test** (UI Automation script against the app's own process and a throwaway database). It created a game against the hard computer, saved after a roll, played on, and checked that exiting asks to save (Cancel keeps the window open, No exits). It then reopened the save, confirmed it resumed at the saved roll, played the game to the end, checked the results and high-score windows, and checked that a finished game exits without a prompt. Two failures were problems in the test script, not the app: it treated the disabled Roll button of the AI's turn as a human turn, and it gave up too soon waiting for a zero-score prompt.
6. **Step 7, executable.** Added the `Standalone.pubxml` publish profile (self-contained, single file, compressed, no extra files): `publish\Yahtzee.exe`, 53 MB. The whole end-to-end UI test passed against the published exe.
7. Moved this guide's generator into the project (`docs/guide-src/`), retook all screenshots, added explanations for every new file, and updated `CLAUDE.md` and the GenAI log. The walkthrough now covers **4149 source lines in 961 explained segments**.

> **Nothing outside the project was changed permanently.** The temporary .NET SDK and the UI-test scripts live in the session's scratch folder. The project files are the ones described in sections 9–12, plus `CLAUDE.md`, `docs/genai-log.md`, this PDF and its generator in `docs/guide-src/`.

4. Building, running and testing

Folder structure

```
yahtzee-proj/
├── F20SC_Coursework1.pdf          the specification
├── CLAUDE.md                      plan and conventions (guidance for future AI sessions)
├── Yahtzee.slnx                   solution (open this in Visual Studio 2026)
├── Directory.Build.props          settings shared by all projects
├── .gitignore
├── docs/
│   ├── genai-log.md               prompts and outcomes, for the report's GenAI appendix
│   ├── Yahtzee_Code_Guide.pdf     this document
│   └── guide-src/                 the generator of this document (Python + Edge)
├── publish/Yahtzee.exe            the standalone executable (created by dotnet publish)
├── src/
│   ├── Yahtzee.Core/              game logic library
│   │   ├── AI/                    IPlayerStrategy, Random/Greedy/ExpectedValue strategies, StrategyCatalog, ScoringHeuristics
│   │   ├── Data/                  IGameHistoryRepository, SqliteGameHistoryRepository
│   │   ├── Game/                  GameEngine.cs, GameEngine.Persistence.cs, GameEvents.cs
│   │   ├── Model/                 Category, DiceCup, IDiceRoller, Player, ScoreCard
│   │   ├── Persistence/           GameSnapshot, SaveGameSerializer, InvalidSaveFileException
│   │   └── Rules/                 Scorer
│   └── Yahtzee.WinForms/          the application (Yahtzee.exe)
│       ├── Program.cs, UiTheme.cs
│       ├── Controls/DieControl.cs
│       ├── Forms/                 MainForm, NewGameDialog, ResultsForm, HighScoresForm, PlayerTypeOption
│       └── Properties/PublishProfiles/Standalone.pubxml
└── tests/Yahtzee.Core.Tests/      xUnit tests (130)
```

Commands

```
# build everything (warnings are errors)
dotnet build Yahtzee.slnx

# run the game
dotnet run --project src/Yahtzee.WinForms

# run all tests, one class, or one test
dotnet test
dotnet test --filter "FullyQualifiedName~ScorerTests"
dotnet test --filter "FullyQualifiedName~ScorerTests.FullHouse_RejectsYahtzee"

# standalone executable for submission: publish\Yahtzee.exe (needs no .NET installation)
dotnet publish src/Yahtzee.WinForms -p:PublishProfile=Standalone

# run with a separate games-history database (e.g. a clean one for the video)
publish\Yahtzee.exe --database C:\temp\demo.db

# rebuild this PDF after changing the code (needs Python 3 with Pillow, and Microsoft Edge)
powershell -File docs\guide-src\build_guide.ps1
```

In Visual Studio 2026: open `Yahtzee.slnx`, set **Yahtzee.WinForms** as the startup project, press F5. Tests appear in **Test → Test Explorer**.

> Where things are stored: save files wherever you choose (extension `.yahtzee`); the games history in `%LOCALAPPDATA%\Yahtzee\yahtzee.db` (delete that file to reset the high scores). If `dotnet` is not found in a terminal, install the .NET 10 SDK (Visual Studio 2026 includes it).

5. Architecture and design

5.1 Layers

The solution is split so that **all rules live in a UI-free library**. This is the single most important design decision: the rules can be unit tested without a window, the UI cannot accidentally break them, and another front end (for example MAUI) could reuse the library unchanged.

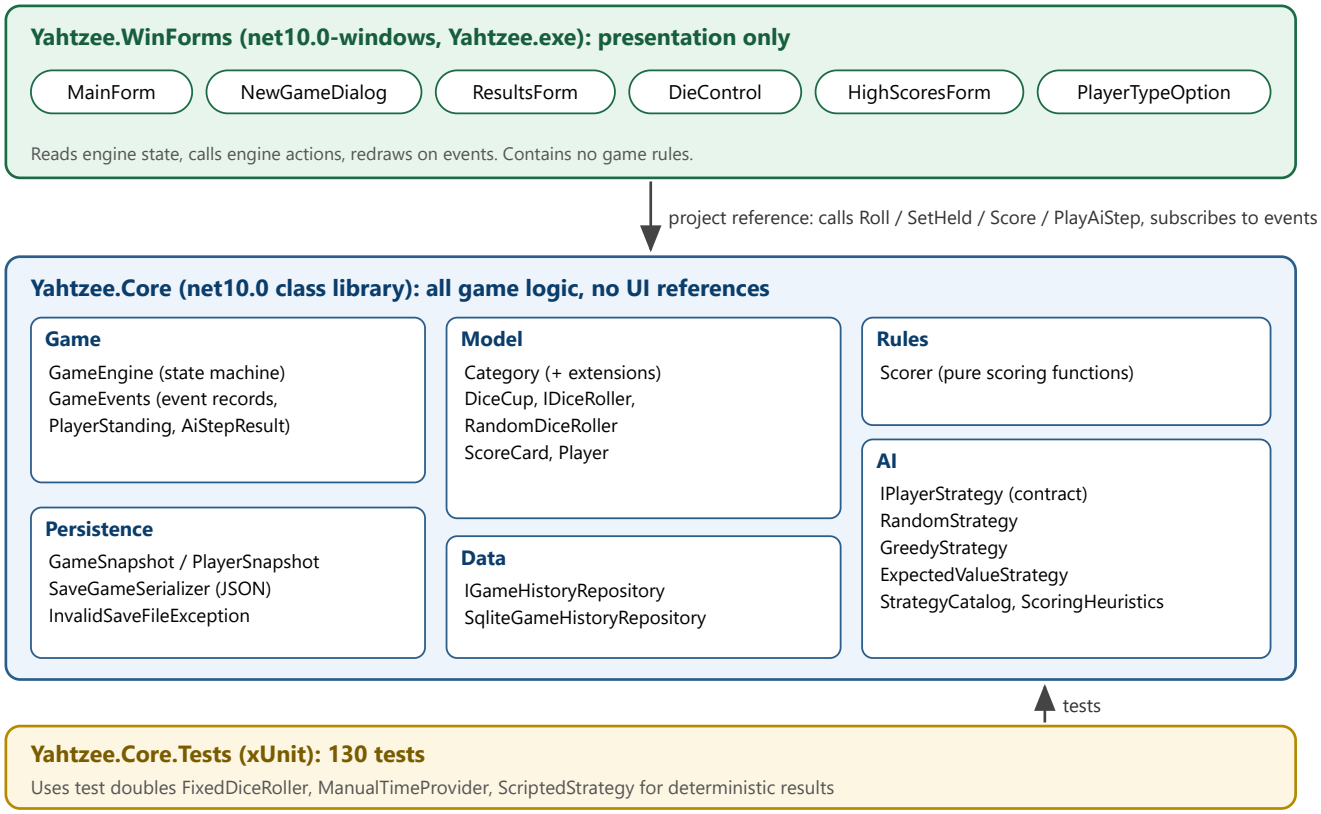
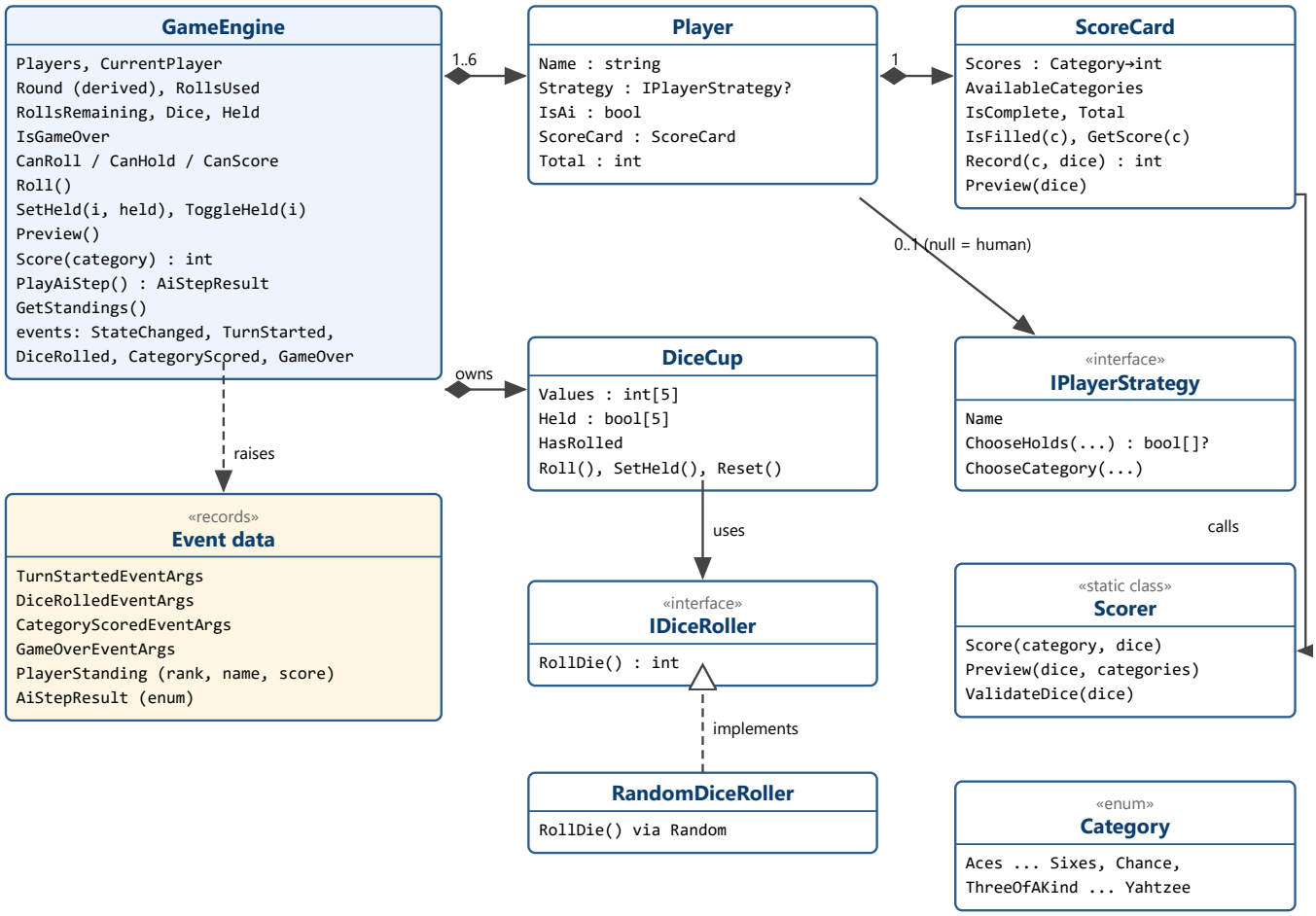


Figure 2. Projects and namespaces.

5.2 Classes and their relationships

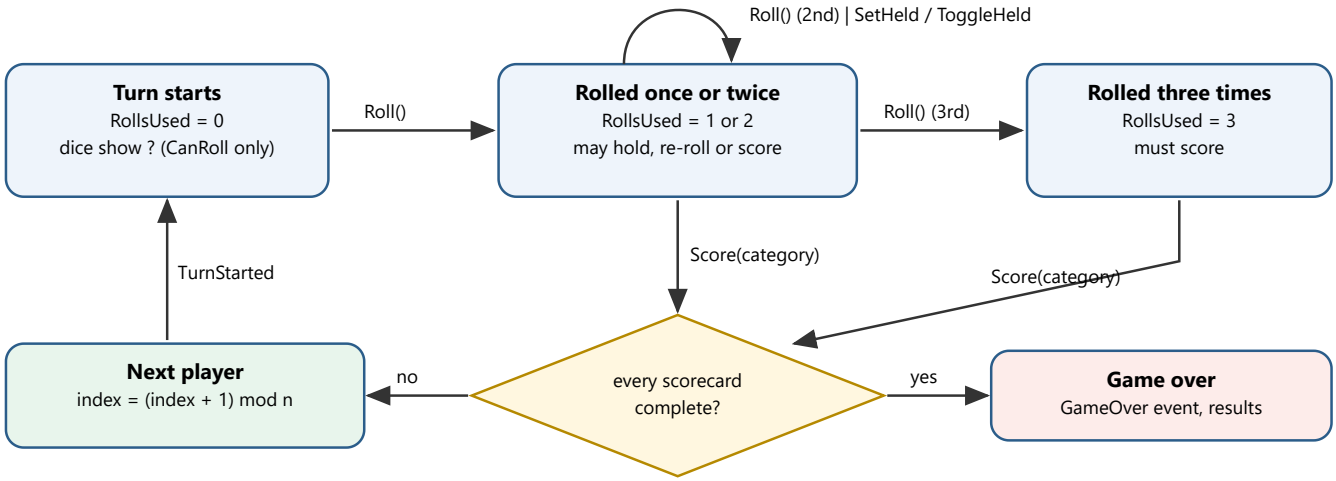


Filled diamond = owns (composition). Hollow triangle = implements. Dashed arrow = creates/raises. Category is used by almost every class (lines omitted).

Figure 3. Class diagram of Yahtzee.Core (main members only).

Class	Responsibility (one job each)
`Scorer`	Pure scoring arithmetic for one category and five dice.
`ScoreCard`	One player's 13 boxes: once-only rule, totals, preview.
`DiceCup`	Five values and held flags; re-rolls the unheld dice.
`IDiceRoller`	Where randomness comes from (swappable for tests).
`Player`	Name + scorecard + optional AI strategy.
`IPlayerStrategy`	AI decisions only (which dice to hold, which category).
`GameEngine`	Turn order, roll limit, legality of every action, game end, events.
`MainForm` etc.	Displaying state and forwarding user input; no rules.

5.3 The turn state machine



Every transition also raises StateChanged. Illegal actions (e.g. Roll() in the right-hand state, Score() in the left-hand state) throw InvalidOperationException; the UI never offers them because it enables buttons from CanRoll / CanHold / CanScore.

Figure 4. States of a turn as enforced by GameEngine.

5.4 Event-driven updates

One-way data flow: the UI never edits state directly, so the screen can never disagree with the rules.

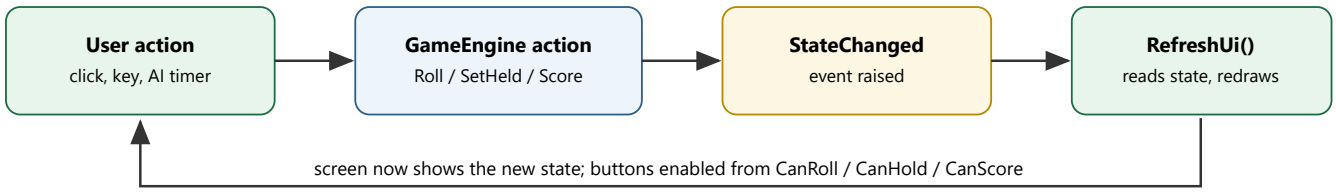


Figure 5. Every change goes through the engine and comes back to the screen via StateChanged.

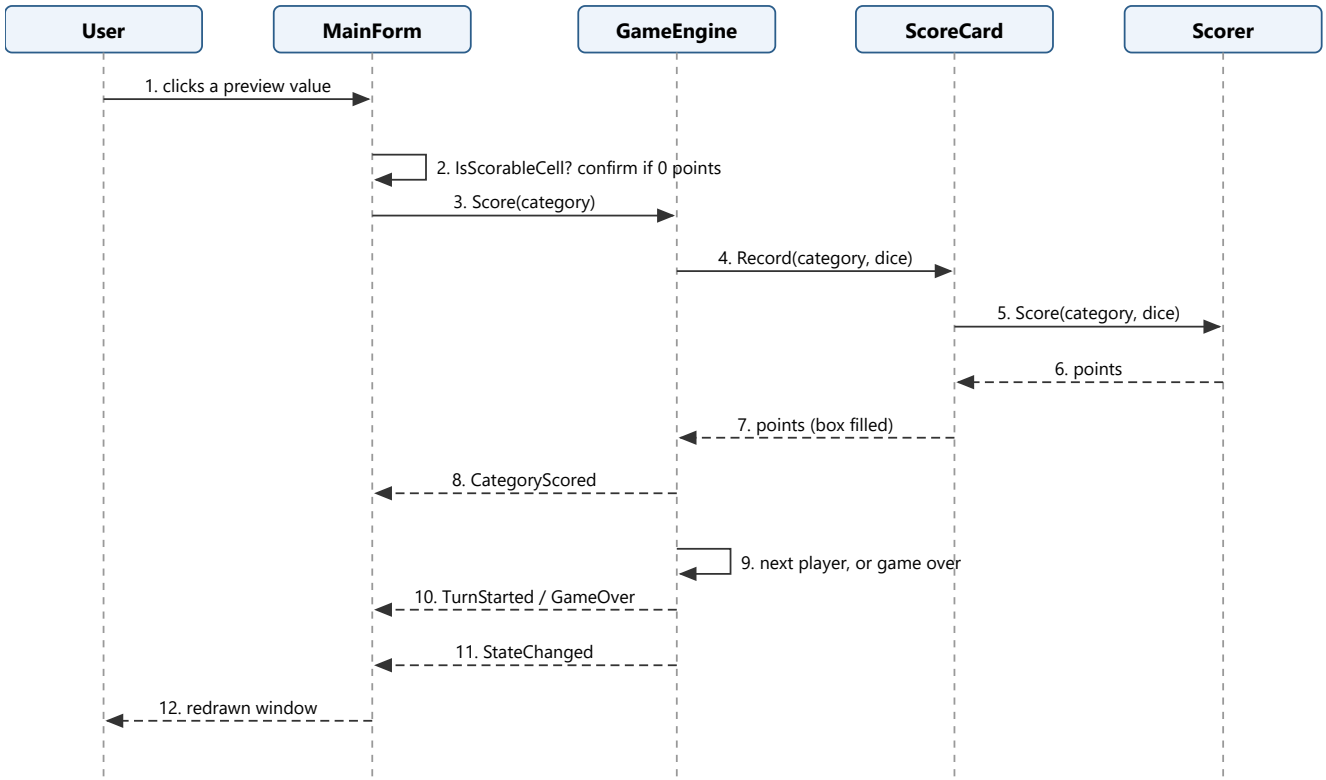
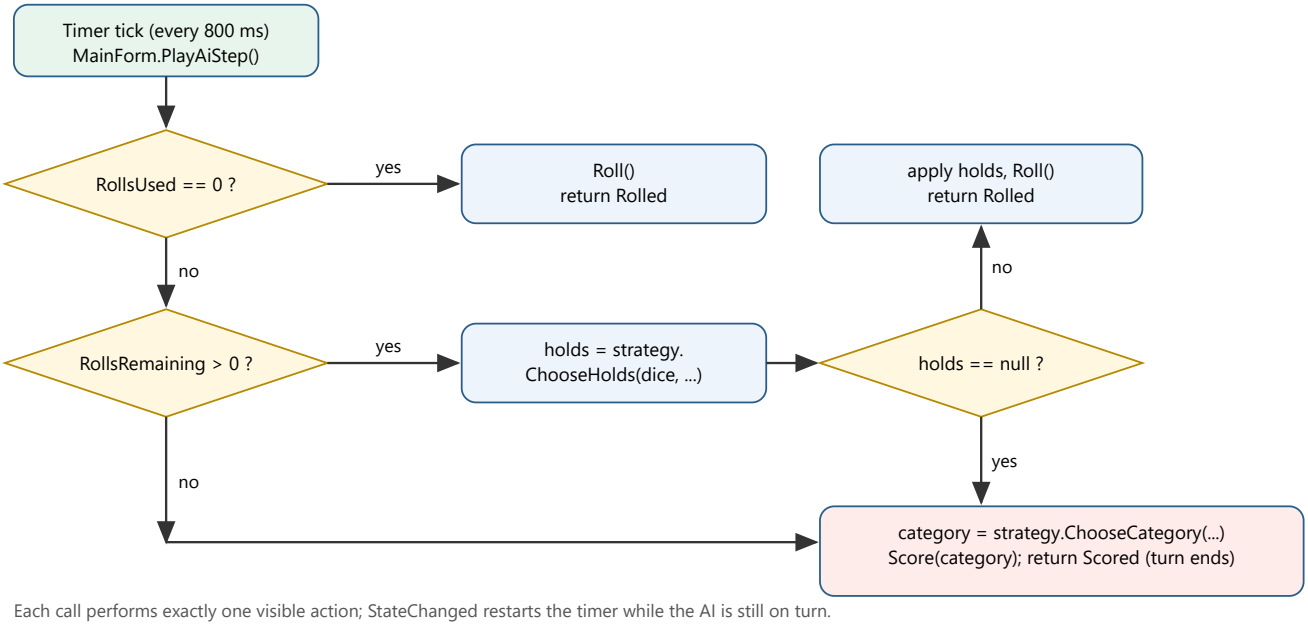


Figure 6. Sequence of calls when a human scores a category. Step 8 updates the status bar; step 10 shows the results dialog (via BeginInvoke) if the game ended; step 11 makes RefreshUi() redraw everything from the engine's state.

5.5 How computer players take turns



Each call performs exactly one visible action; StateChanged restarts the timer while the AI is still on turn.

Figure 7. GameEngine.PlayAiStep(): one visible action per timer tick.

5.6 Design decisions and why (useful for the report's "Design considerations")

Decision	Reason / trade-off
Windows Forms over MAUI/WPF	Recommended by the spec as the simplest; target platform is Windows only. Trade-off: not cross-platform.
Core library with no UI references	Testability and separation of concerns; the developer guide can describe rules independently of the GUI.
Frequency table in `Scorer`	Counting faces once (an `int[7]`) makes every rule a one-liner and runs in constant time.

Dictionary for the scorecard	An absent key = empty box; avoids parallel "filled" flags; O(1) lookups.
Round derived, not stored	No duplicated state to keep consistent or to save.
`Can…` guards + exceptions	The UI enables controls from the guards; the engine still rejects illegal calls, so rules hold even if the UI has a bug.
Events + one refresh method	One-way data flow; the screen always matches the state; also gives the save-before-exit "dirty" flag for free (step 4).
Strategy pattern for AI	Strategies only decide, the engine applies: AIs cannot cheat; new strategies plug in without changing the engine.
Injected randomness and clock	Deterministic tests (`FixedDiceRoller`, `ManualTimeProvider`).
Stable enum values	The save format and database will store category numbers.
Warnings as errors + XML docs	Keeps code clean and documented (code-quality marks).
Forms built in code, custom-drawn dice	Readable diffs and explicit layout; dice stay sharp at any display scale.
JSON save files with a version number	Human-readable, built into .NET, easy to extend; old files are recognised by their version.
Strict parsing + logical validation on load	A damaged or hand-edited file gives a clear message instead of a crash or an impossible game.
Write to a temporary file, then move	A crash during saving can never destroy the previous save.
Change counter instead of a dirty flag	Stays correct when an AI move happens during an asynchronous save.
Repository interface + SQLite	A real database engine with no server or installation; the UI depends only on the interface.
One transaction per game, parameterised SQL	No half-recorded games; player names can never inject SQL.
Strategy catalog by name	One list feeds the new-game dialog and save loading; adding a strategy is one line.
Expected-value AI with a cache	Exact probabilities instead of guesses; caching by sorted dice (252 hands) keeps each decision to milliseconds.
async/await for files and database	The window never freezes while the disk is busy.

5.7 The save-file format

A save is one JSON text file with the extension `.yahtzee`. It can be written **at any moment of a turn**, including straight after a roll with some dice held, because the dice values, the held flags and the number of rolls used are all part of it. This excerpt is from a real save made during the UI test (a human against the hard computer, first roll made, die 1 held):

```
{
  "formatVersion": 1,
  "startedAt": "2026-10-08T11:51:04.6340732+00:00",
  "players": [
    {
      "name": "Player 1",
      "strategy": null,
      "scores": { "Aces": null, "Twos": null, ... (13 categories) ..., "Yahtzee": null }
    },
    {
      "name": "Computer 2",
      "strategy": "ExpectedValue",
      "scores": { "Aces": null, "Twos": null, ... (13 categories) ..., "Yahtzee": null }
    }
  ],
  "currentPlayerIndex": 0,
  "rollsUsed": 1,
  "dice": [3, 2, 2, 5, 6],
  "held": [true, false, false, false, false]
}
```

Shortened for this guide: in the real file each of the 13 categories is listed on its own line for every player.

Field	Meaning
`formatVersion`	Always 1 for this program; any other value is refused.
`startedAt`	When the game was started (ISO-8601 with time zone offset).
`players`	In turn order. `name`; `strategy` = `null` for a human or one of `Random`, `Greedy`, `ExpectedValue`; `scores` = every category with its score, or `null` if the box is empty.

`currentPlayerIndex`	Whose turn it is (0 = first player).
`rollsUsed`	0–3 rolls already made in this turn.
`dice`, `held`	Five values (all 0 before the first roll) and five held flags.

The round number is **not** saved: it is calculated from the scorecards. On loading, the file is checked in two layers (Figure 8): the serializer checks the JSON shape, types and version; `GameEngine.Restore` then checks that the game is possible (valid dice, 1–6 unique players, a known strategy, and scorecards consistent with whose turn it is).

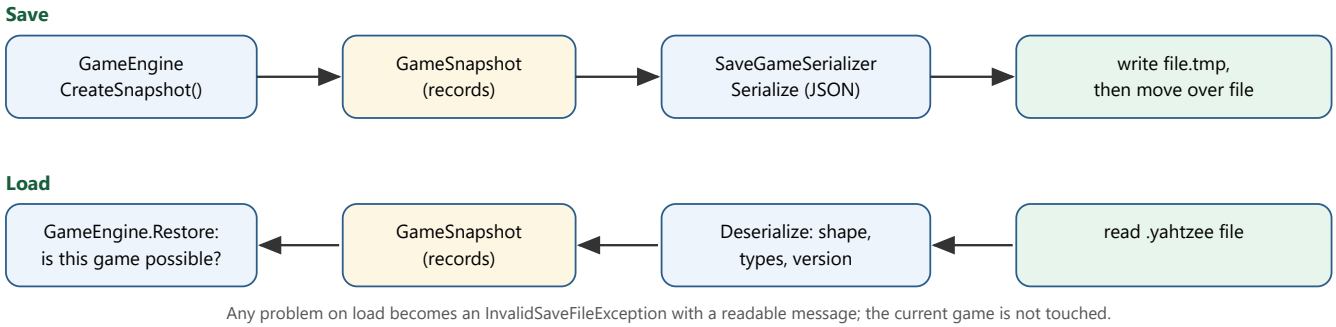


Figure 8. Saving and loading.

5.8 The games database

Every finished game is recorded automatically in a SQLite database file, `%LOCALAPPDATA%\Yahtzee\yahtzee.db`, created on first use.

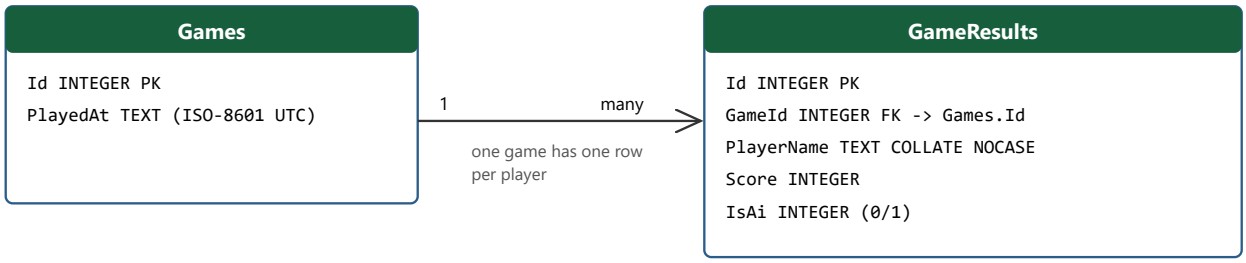


Figure 8b. Database schema: one Games row per finished game, one GameResults row per player.

Operation	SQL (simplified)
Record a game	`INSERT INTO Games`, then one `INSERT INTO GameResults` per player, all in **one transaction**.
Top 5 overall	`SELECT … FROM GameResults r JOIN Games g ON g.Id = r.GameId ORDER BY r.Score DESC, g.PlayedAt LIMIT $count`
Top 5 of a player	Same, with `WHERE r.PlayerName = $name` (case-insensitive because of `COLLATE NOCASE`).
Player names	`SELECT DISTINCT PlayerName FROM GameResults ORDER BY PlayerName`

Values are always passed as **parameters** (`$name`, `$count`), never pasted into the SQL text, so a player called `O'Brien"; DROP TABLE Games; --` is stored safely (there is a test for exactly that).

5.9 The computer players

Level	Class	How it plays	Average score*
Easy	`RandomStrategy`	Random holds, stops one time in three, scores in a random open box.	39
Medium	`GreedyStrategy`	Stops on a Yahtzee or large straight; keeps a run of four for straights; otherwise keeps its most common face. Scores the most points available, sacrificing the cheapest box on ties.	176
Hard	`ExpectedValueStrategy`	For each of the 32 ways to hold dice, computes the exact average result of the next roll over all outcomes; values results as "points above par" so strong boxes are saved for good rolls.	228

* Average over 30 seeded single-player games, from the `StrongerStrategies_ScoreMoreOnAverage` test.

> **Expected value, briefly.** Suppose the hard computer holds three dice and re-rolls two. Two dice have 6 × 6 = 36 equally likely outcomes. For each outcome it works out the best "points above par" it could then score, and averages the 36 numbers. It does this for all 32 hold choices (from re-rolling nothing to re-rolling all five, which has 6^5 = 7,776 outcomes) and picks the highest average. If no re-roll beats keeping the current dice, it stops. Because only the *sorted* dice matter, the 16,807 outcomes collapse to at most 252 different hands, which are cached.

6. The scoring rules, illustrated

All scoring goes through `Scorer.Score`. It first converts the five dice into a **frequency table** (how many dice show each face), then each rule is a simple question about that table. Worked example with the dice from the screenshot in section 7 (4 1 6 6 1):

Face	1	2	3	4	5	6
counts[face]	2	0	0	1	0	2

Derived value	Result
sum of dice	18
largest count	2
longest run of present faces	1 (faces 1, 4, 6 are not consecutive)

Category	Rule in code	Example result
Aces … Sixes	`counts[face] * face`	Aces 2×1 = **2**, Fours **4**, Sixes 2×6 = **12**, others 0
Chance	`sum`	**18**
Three / Four of a Kind	`counts.Max() >= 3 (4) ? sum : 0`	largest count 2 → **0**
Full House	`counts.Contains(3) && counts.Contains(2) ? 25 : 0`	no face appears 3 times → **0**
Small Straight	`LongestRun(counts) >= 4 ? 30 : 0`	run 1 → **0**
Large Straight	`LongestRun(counts) == 5 ? 40 : 0`	**0**
Yahtzee	`counts.Contains(5) ? 50 : 0`	**0**

These are exactly the preview values shown by the application in that situation (Aces 2, Fours 4, Sixes 12, Chance 18, all others 0).

> **Why a Yahtzee is not a Full House:** five 4s give counts `[0,0,0,0,5,0,0]`. That table contains neither a 3 nor a 2, so the Full House test fails, as the note under Figure 1 of the specification requires. A test (`FullHouse_RejectsYahtzee`) guards this.
> **Why a large straight also counts as a small straight:** 1-2-3-4-5 has a longest run of 5, which is ≥ 4.
> **Duplicates inside a straight** (2-3-4-5-5) are fine because the run only checks whether each face is present.

7. The user interface

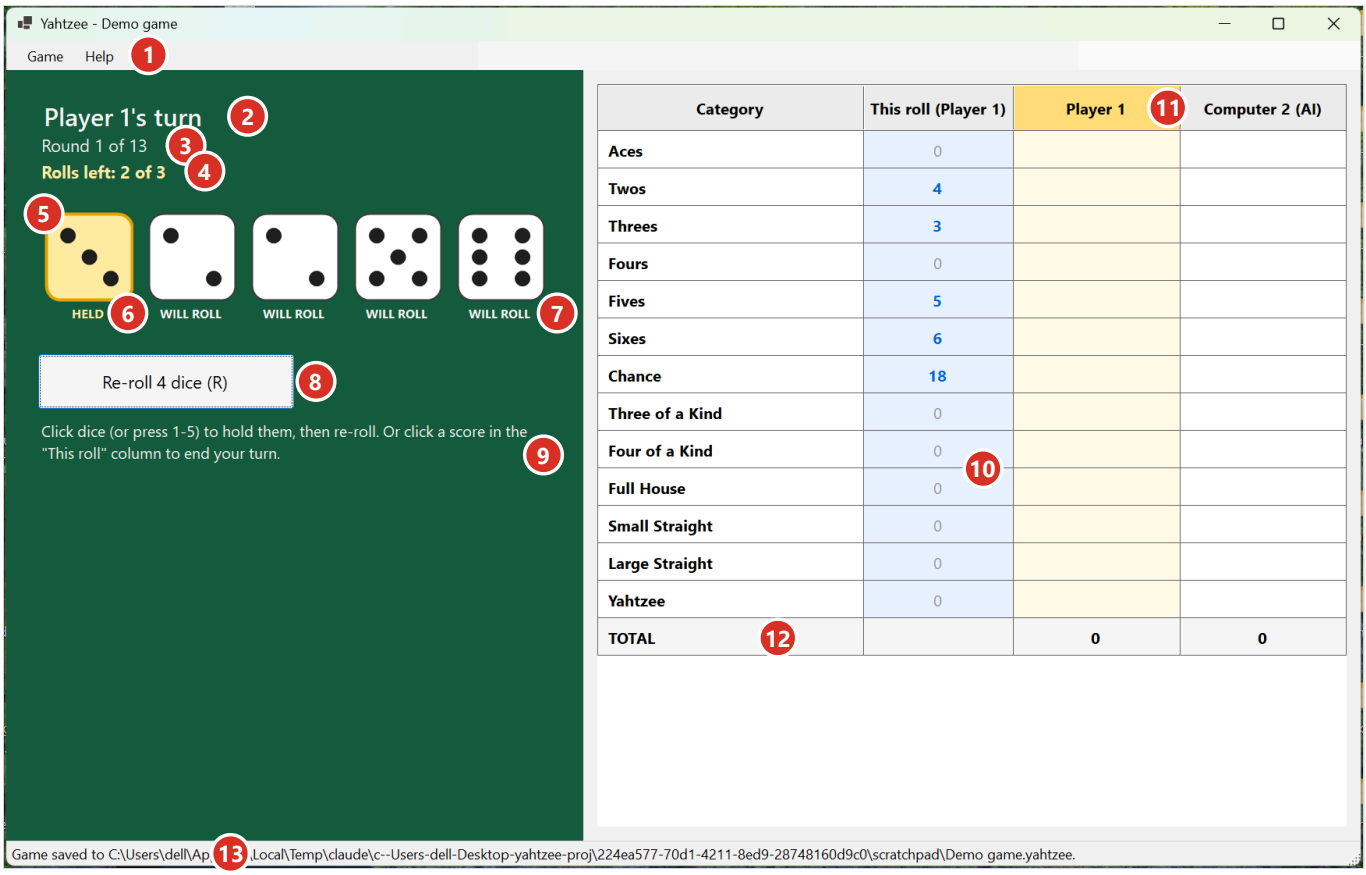


Figure 9. Anatomy of the main window during a turn (a human against the hard computer, after the first roll, die 1 held).

1. **Menu bar**: Game (New Game Ctrl+N, Exit) and Help (How to Play F1).
2. **Whose turn it is** (requirement). AI players are marked "(computer)".
3. **Round** n of 13, derived from the scorecard.
4. **Rolls left** (requirement), in bold gold.
5. **Held die**: gold face and border.
6. **HELD** caption: this die is set aside (requirement).
7. **WILL ROLL** caption: this die will be re-rolled (requirement).
8. **Roll button** states exactly how many dice it will roll; R is the shortcut.
9. **Hint** text explaining what the player can do next.
10. **"This roll" preview column** (requirement): blue = points available, grey = 0. Click a value to score it.
11. **Current player's column**, highlighted gold/cream.
12. **TOTAL row**: running total for every player.
13. **Status bar**: the last roll or score, e.g. "Player 1 rolled 3 5 3 1 6".

Screens

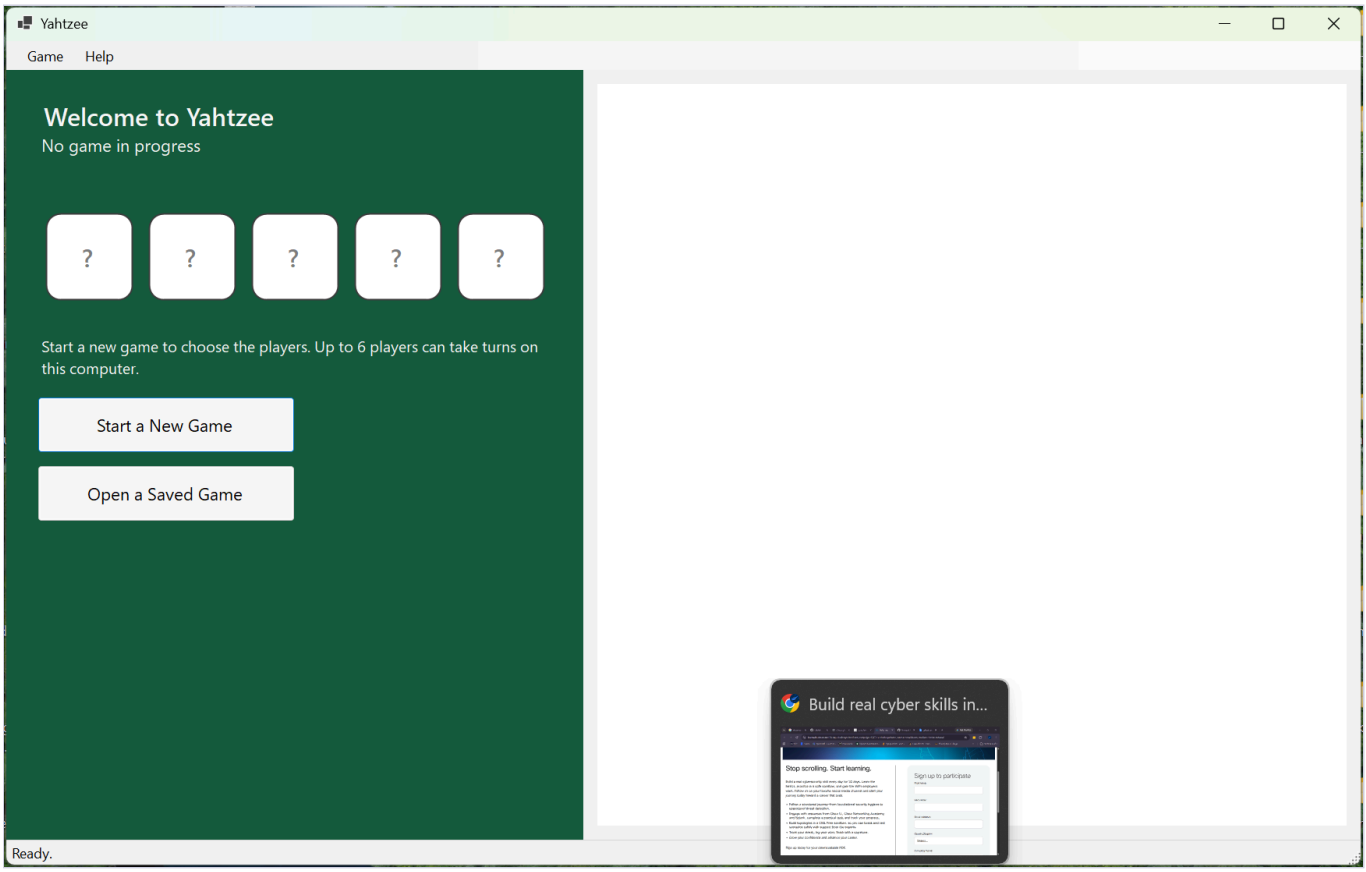


Figure 10. Welcome screen before any game: start a new game or open a saved one.

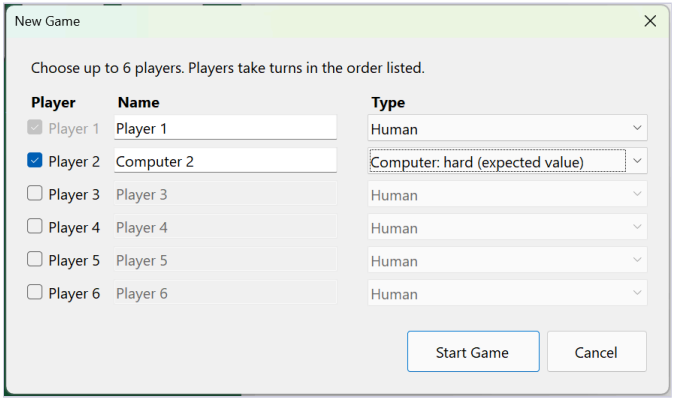


Figure 11. New-game dialog: seat 2 switched to the hard computer and renamed automatically.

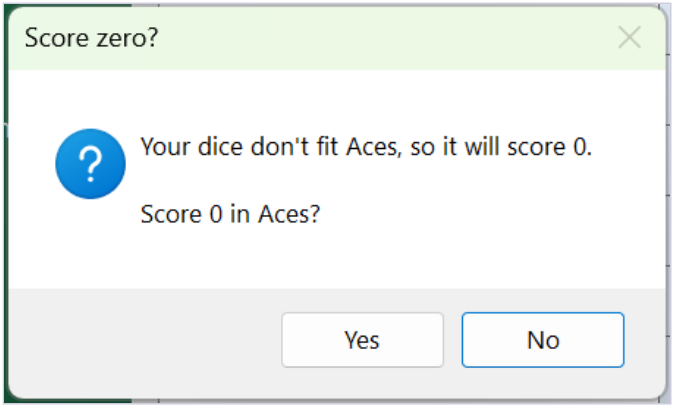


Figure 12. Confirmation before scoring 0 (a common mis-click).

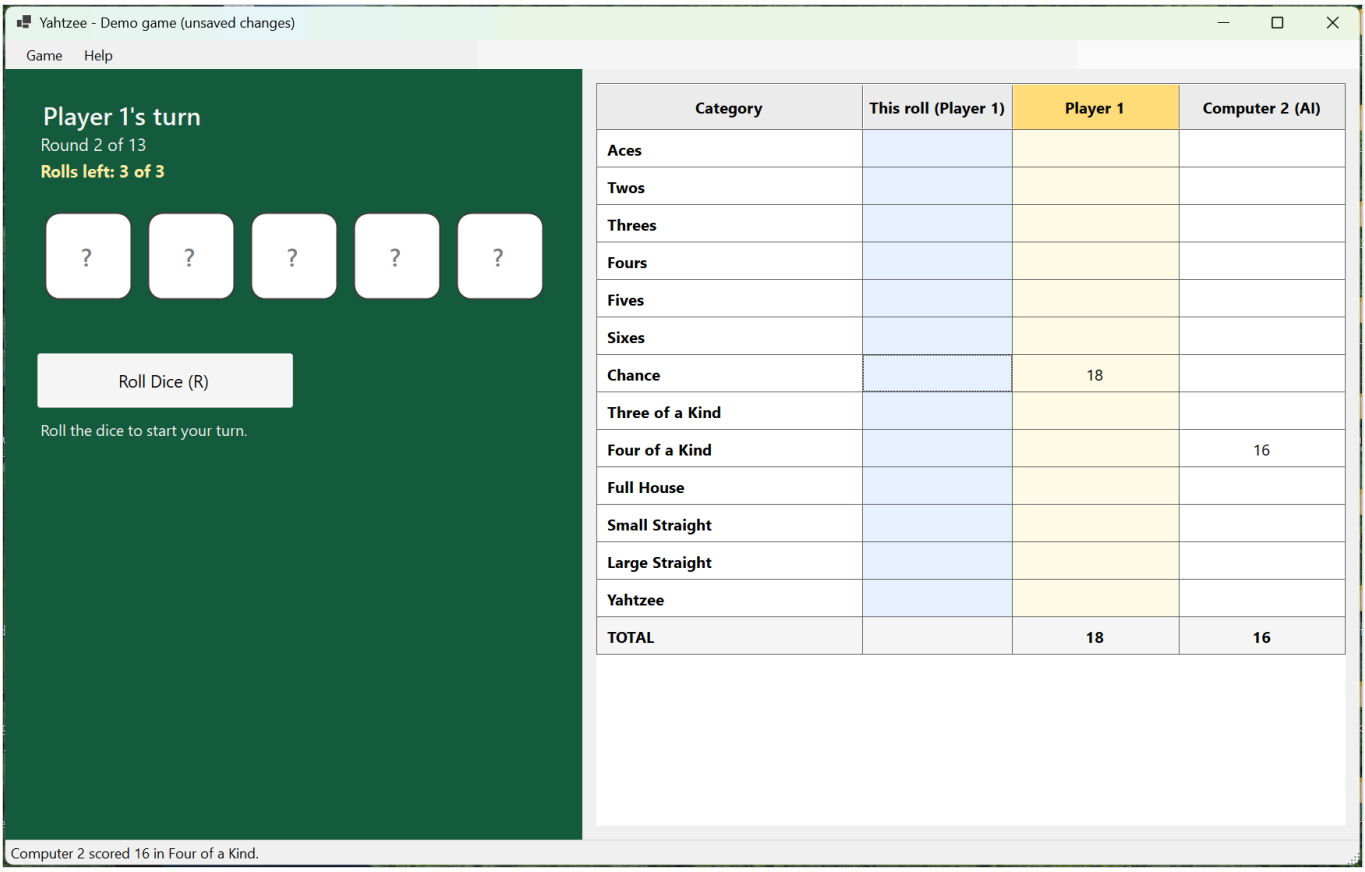


Figure 13. After the computer's turn: its score appears in its column and the status bar reports the move.

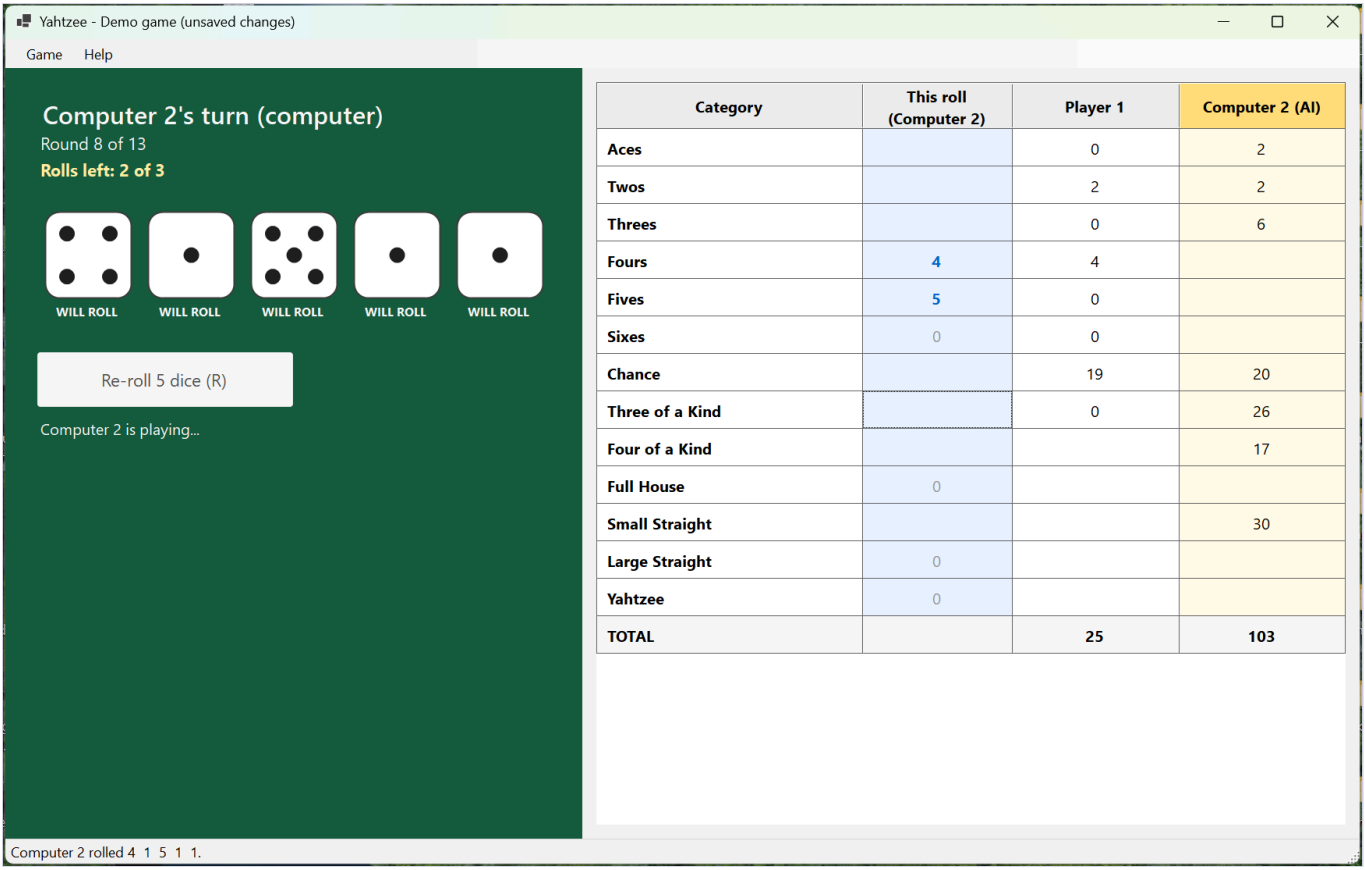


Figure 14. Mid-game against the computer: filled boxes disappear from the preview column; totals update.

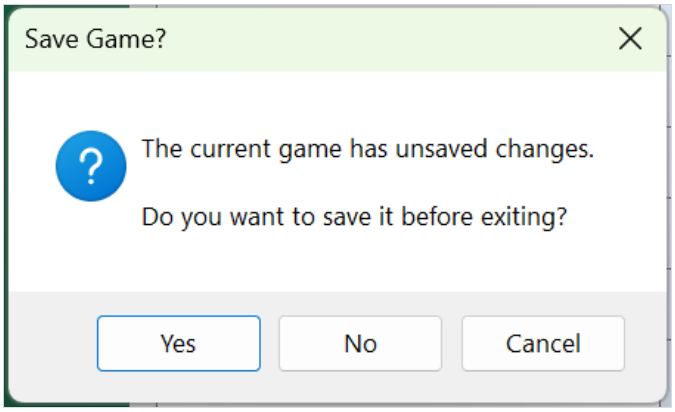


Figure 15. Exiting with unsaved changes: Yes saves, No discards, Cancel keeps playing.

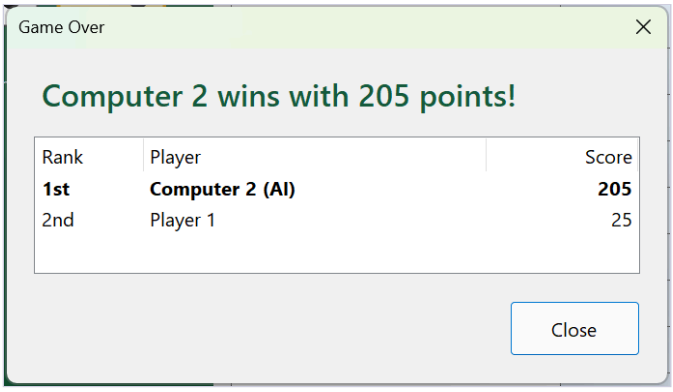


Figure 16. End-of-game summary ordered by score.

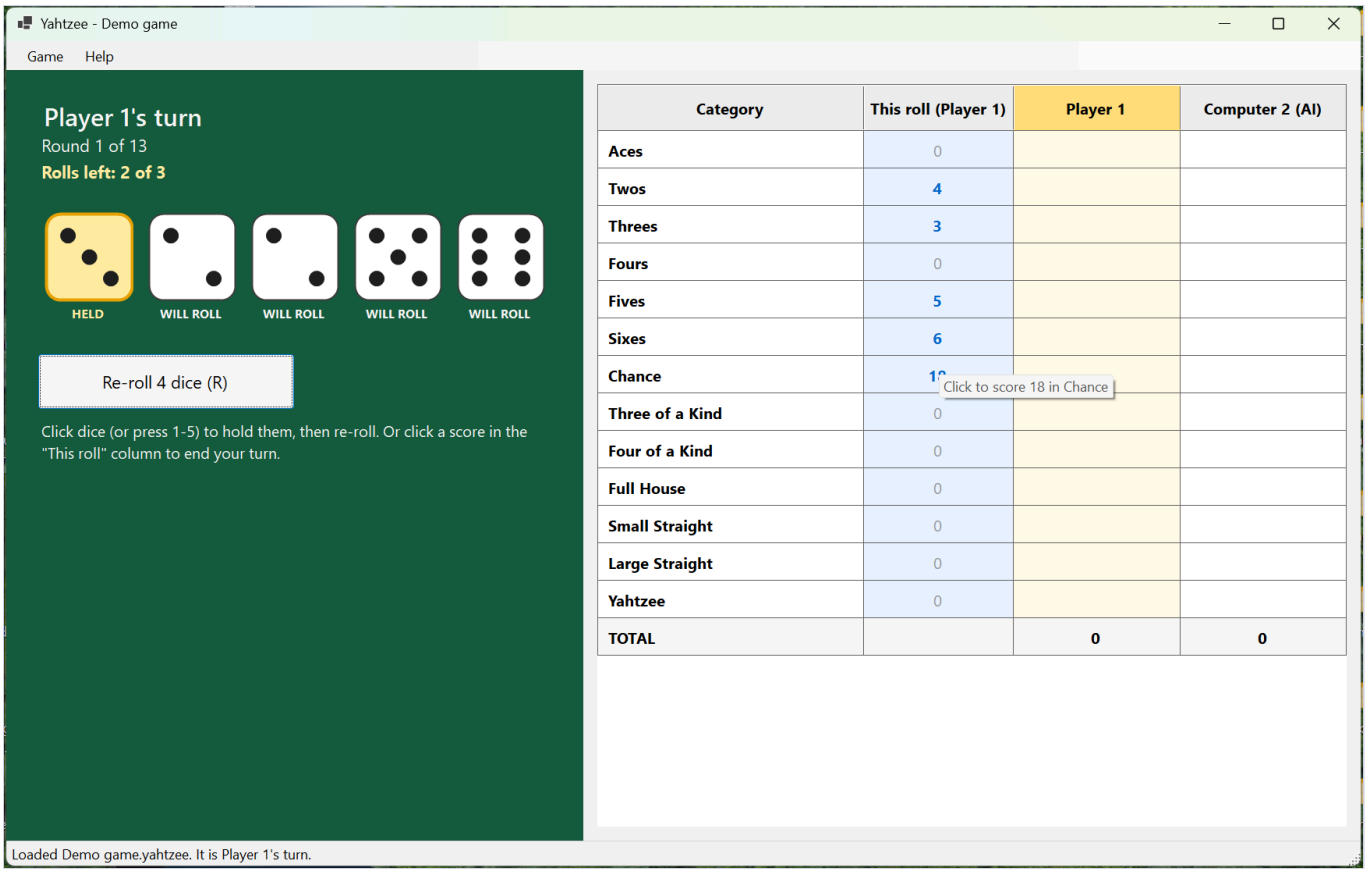


Figure 17. A saved game reopened: the title shows the file name, and the turn resumes at the saved roll ("Re-roll 4 dice").

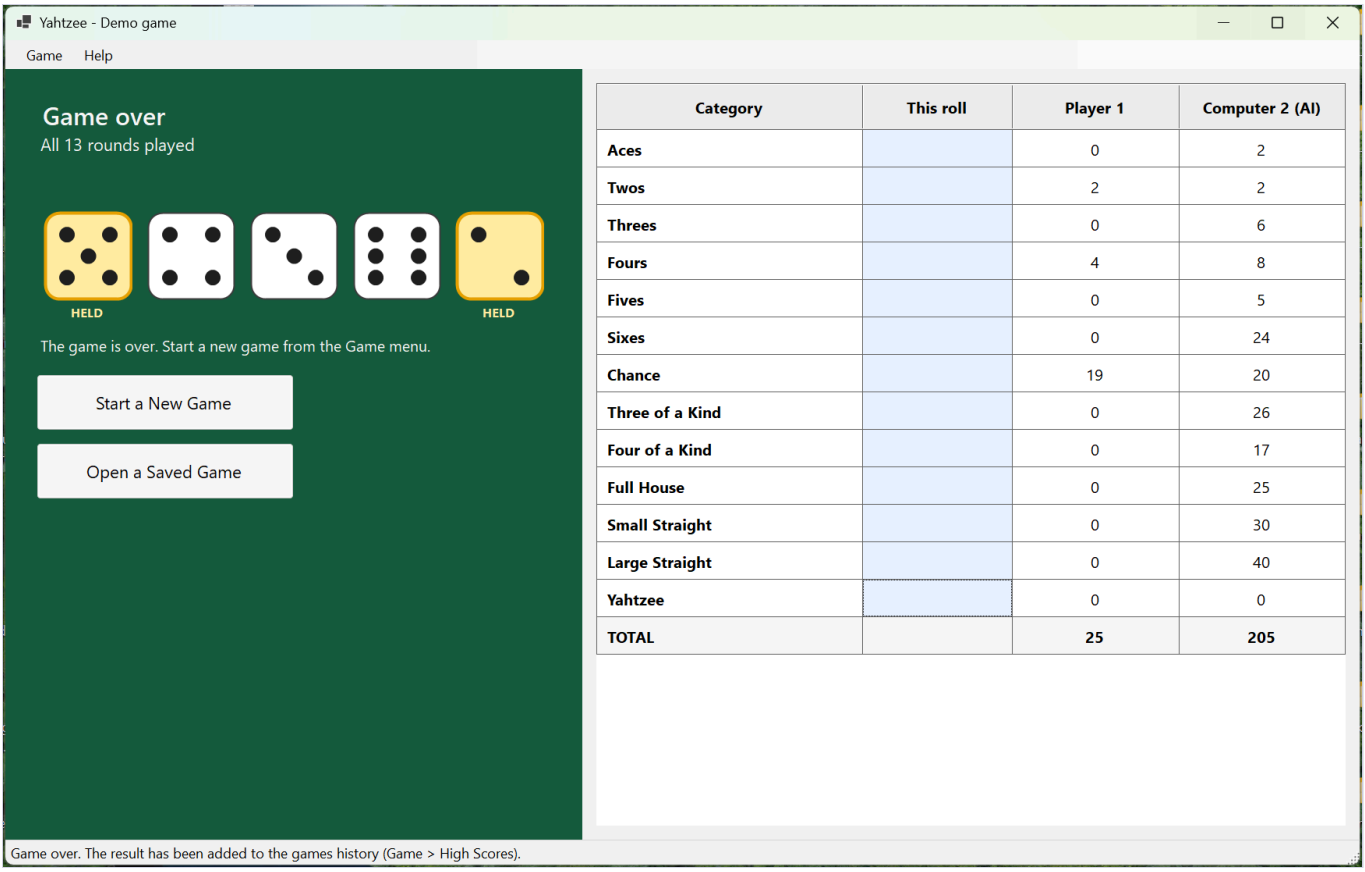


Figure 18. The board after the game, with buttons to start or open another game.

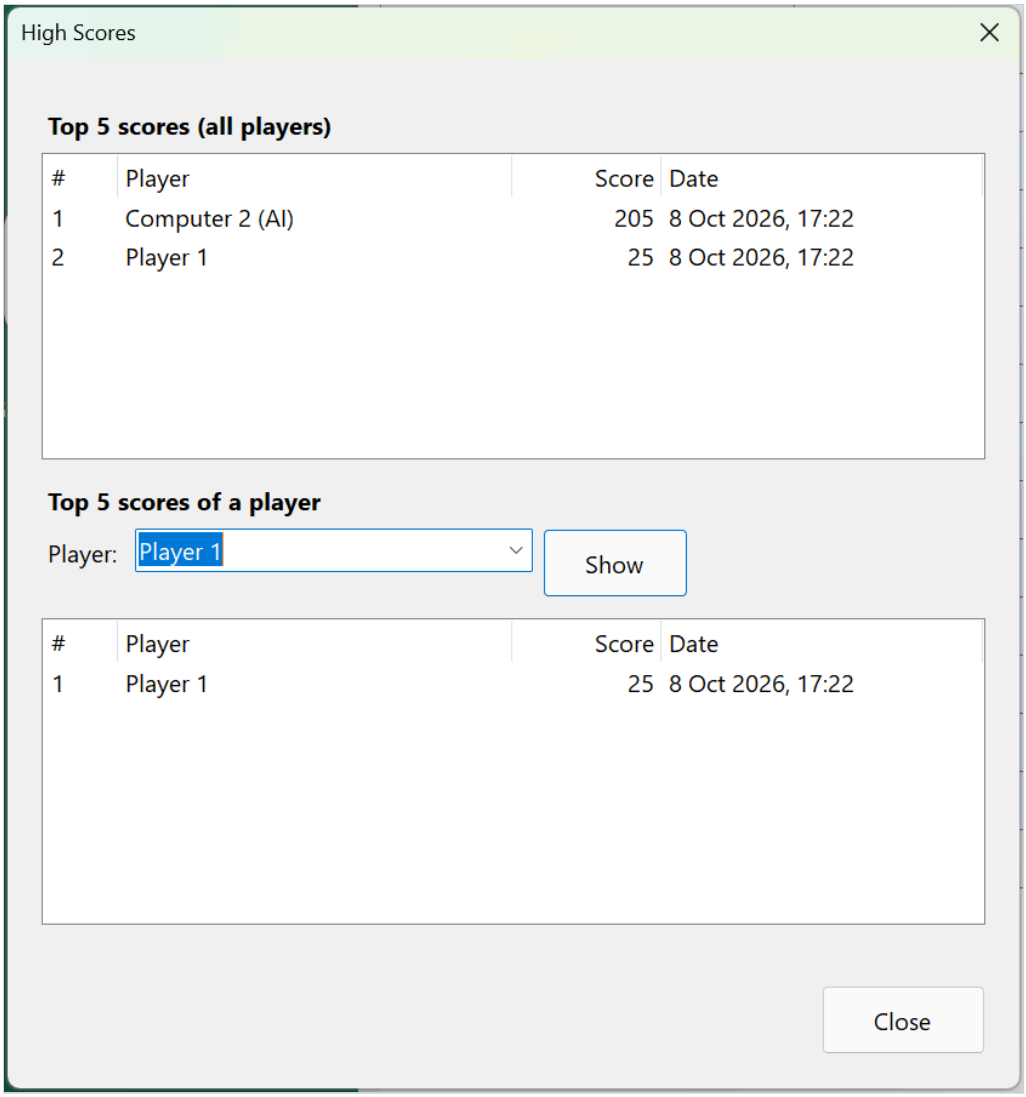
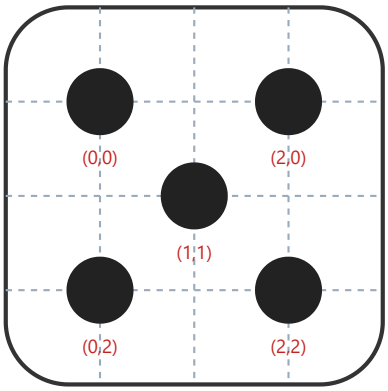


Figure 19. High scores from the database: top 5 overall and top 5 of a chosen player, with dates.

How requirements map to what you see

Requirement	Where on screen / in code
Whose turn it is	Heading (callout 2) and gold column (11) — `RefreshUi`, `RefreshScoreGrid`.
Rolls available	"Rolls left" (4) — `GameEngine.RollsRemaining`.
Which dice roll / are set aside	HELD / WILL ROLL (5–7), roll button text (8) — `DieControl`.
Score preview	"This roll" column (10) — `GameEngine.Preview()`.
Result summary	Figure 16 — `ResultsForm`, `GetStandings()`.
Saving/loading	Game menu (Ctrl+S, Ctrl+O), title bar, Figure 17 — `SaveGameAsync`, `OpenGameAsync`.
Offer to save before exiting	Figure 15 — `OnFormClosing`, `ConfirmUnsavedChangesAsync`.
Games database	Figure 19 (Ctrl+H) — `HighScoresForm`, `SqliteGameHistoryRepository`.
AI opponents	Figures 11, 13, 14 — strategies chosen per seat; AI moves every 0.8 s.

Drawing a die



Face 5: PipLayout[5]

= [(0,0), (2,0), (1,1), (0,2), (2,2)]

cell = width / 4: grid lines at 1/4, 1/2, 3/4

pip centre x = left + cell * (col + 1)

pip centre y = top + cell * (row + 1)

pip radius = width / 11

Corner radius of the face = width / 6

Figure 20. How DieControl places pips: a 3×3 grid and a lookup table of positions per face.

Display scaling (DPI): a bug found by testing

Windows can scale the interface (this computer uses 150%). WinForms automatically scales control *sizes, positions and fonts* when a form uses `AutoScaleMode.Dpi`, but some pixel values are **not** scaled: DataGridView row and header heights and ListView column widths. At 150% the scorecard rows were therefore cramped. The fix converts those values with `LogicalToDeviceUnits` once the window exists (or sizes columns in the `Load` event), so a "32-pixel" row becomes 48 physical pixels at 150%.

Keyboard support

Ctrl+N new game · **Ctrl+O** open · **Ctrl+S** save · **Ctrl+H** high scores · **R** roll · **1–5** hold/release a die · **Tab** to a die then **Space/Enter** · **F1** how to play · **Enter/Esc** in dialogs.

8. C# language features used

The marking scheme rewards "use of advanced language features". These are the ones used so far, with an example of where to find each.

Feature	Example	Where
Records	`record PlayerStanding(int Rank, ...)`	GameEvents.cs, PlayerTypeOption.cs, NewGameDialog.SeatRow
Primary constructors	`class RandomDiceRoller(Random? random = null)`	IDiceRoller.cs, test doubles
Switch expressions	`category switch { ... }`	Scorer, Category.DisplayName, MainForm
Relational/logical patterns	`is >= Category.Aces and <= Category.Sixes`	Scorer, Category.Face, MainForm keys
Property patterns	`_game is { IsGameOver: false, CurrentPlayer.IsAi: false }`	MainForm
Tuple patterns	`(_value, _held, _showRollCaption) switch`	DieControl.DrawCaption, ResultsForm.Ordinal
List patterns	`[var only] when ...`	ResultsForm.DescribeWinner
Collection expressions, spread	`[.. players]`, `[]`	GameEngine, ScoreCard, DieControl
Nullable reference types	`IPlayerStrategy?`, `int?`	everywhere
LINQ	`OrderByDescending`, `DistinctBy`, `ToDictionary`	GameEngine, Scorer, ScoreCard
Events and delegates	`event EventHandler<DiceRolledEventArgs>?`, `Func<IPlayerStrategy?>`	GameEngine, PlayerTypeOption
Lambdas	`die.Click += (_, _) => ToggleDie(index);`	MainForm, dialogs
Interfaces & generics	`IDiceRoller`, `IReadOnlyList<int>`	Core
Extension methods	`category.DisplayName()`	Category.cs
Deconstruction	`foreach (var (category, score) in scores)`	ScoreCard, DieControl
Throw expressions	`roller ?? throw new ...`	DiceCup, GameEngine
File-scoped namespaces	`namespace Yahtzee.Core.Model;`	every file
XML documentation	`/// <summary>`	every public member of Core (enforced by the build)
async/await, Task<T>	`await SaveGameSerializer.SaveAsync(...)`	SaveGameSerializer, SqliteGameHistoryRepository, MainForm, HighScoresForm
Generic methods	`T ShowModal<T>(Func<T> show)`	MainForm
Partial classes	`public sealed partial class GameEngine`	GameEngine.cs + GameEngine.Persistence.cs
Static factory method	`GameEngine.Restore(...)`	GameEngine.Persistence.cs
Exception filters	`catch (Exception ex) when (ex is IOException or ...)`	MainForm, HighScoresForm
Custom exception type	`InvalidSaveFileException`	Persistence
Raw string literals	`""" CREATE TABLE ... """`	SqliteGameHistoryRepository
Local functions	`bool IsOpenAndMade(Category c) => ...`	GreedyStrategy
Nested classes	`PositionEvaluator`	ExpectedValueStrategy
Bitwise operators	`(mask & (1 << i)) != 0`	ExpectedValueStrategy
Method groups as delegates	`GameEngine.Restore(snapshot, roller, StrategyCatalog.Create)`	MainForm, GameEngine.Persistence
`with` expressions	`snapshot with { RollsUsed = 4 }`	SaveGameTests
Repository and strategy patterns, dependency injection	`new MainForm(history)`	Program, MainForm

How to read the line-by-line walkthrough (sections 9–12)

Every source file is reproduced **in full**, split into small consecutive segments. Each segment shows the code with its real line numbers, followed by an explanation of those lines. Segments follow each other without gaps, so every line of every file is covered. Blank lines and closing braces are shown and belong to the segment above them. Syntax colours follow Visual Studio: keywords, type names, strings, comments, documentation comments.

Recommended reading order: `Category` → `Scorer` → `DiceCup` → `ScoreCard` → `Player` → `GameEngine` → `MainForm`, with the tests next to the class they test.

9. Line-by-line walkthrough: Build and project files

9.1 Solution file

`Yahtzee.slnx · 9 lines`

The solution groups the three projects so Visual Studio 2026 (or `dotnet build`) can open and build them together. It uses the newer XML `.slnx` format, which is much shorter and easier to read than the classic `.sln`.

```
1 <Solution>
```

Line 1 Root element of the solution file. Everything inside describes which projects belong to the solution.

```
2   <Folder Name="/src/">
3     <Project Path="src/Yahtzee.Core/Yahtzee.Core.csproj" />
4     <Project Path="src/Yahtzee.WinForms/Yahtzee.WinForms.csproj" />
5   </Folder>
```

Lines 2–5 A *solution folder* called `src`. It is only a grouping in Visual Studio's Solution Explorer; it holds the two application projects: the game-logic library (`Yahtzee.Core`) and the Windows Forms app (`Yahtzee.WinForms`). Paths are relative to the solution file. `</Folder>` closes the group.

```
6   <Folder Name="/tests/">
7     <Project Path="tests/Yahtzee.Core.Tests/Yahtzee.Core.Tests.csproj" />
8   </Folder>
```

Lines 6–8 A second solution folder holding the unit-test project.

```
9 </Solution>
```

Line 9 Closes the root element.

9.2 Shared MSBuild settings

`Directory.Build.props · 9 lines`

MSBuild automatically imports a file with this exact name into *every* project in this folder and below it, so the settings are written once instead of being repeated in each `.csproj`.

```
1  <Project>
2    <!-- Settings shared by every project in the solution. -->
3    <PropertyGroup>
```

Lines 1–3 Root of an MSBuild file. The comment explains the file's purpose.

```
4      <LangVersion>latest</LangVersion>
```

Line 4 Use the newest C# language version the installed compiler supports (C# 14 with the .NET 10 SDK). This enables features such as primary constructors and collection expressions used throughout the code.

```
5      <Nullable>enable</Nullable>
```

Line 5 Turns on **nullable reference types**: the compiler warns whenever a value that might be `null` is used without a check. A type written `Player?` may be null; `Player` may not.

```
6      <ImplicitUsings>enable</ImplicitUsings>
```

Line 6 Adds common `using` directives automatically (e.g. `System`, `System.Linq`, `System.Collections.Generic`; for the WinForms project also `System.Windows.Forms` and `System.Drawing`).

```
7      <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
```

Line 7 Any compiler or analyzer warning fails the build. This keeps the code clean: nullability mistakes, missing XML comments in Core, and xUnit/WinForms analyzer findings must all be fixed.

```
8    </PropertyGroup>
9  </Project>
```

Lines 8–9 Closes the property group and the project.

9.3 Files excluded from version control

`.gitignore · 9 lines`

Lists build outputs and personal IDE files that should never be committed or included in the source zip.

```
1 bin/
2 obj/
```

Lines 1–2 `bin/` and `obj/` are created by every build (compiled DLLs/EXEs and intermediate files).

```
3 publish/
```

Line 3 Output folder of the planned `dotnet publish` step (the standalone executable).

```
4 .vs/
5 *.user
```

Lines 4–5 Visual Studio's hidden per-user cache folder, and `*.user` per-user project settings.

```
6 TestResults/
```

Line 6 Test-run output written by `dotnet test` / Visual Studio's Test Explorer.

```
7 docs/guide-src/build/
8 __pycache__/
```

Lines 7–8 The HTML generated by the code-guide builder, and Python's compiled-module cache folders.

```
9 *.yahtzee.tmp
```

Line 9 A temporary file left behind only if the program crashed in the middle of saving.

9.4 Core library project

`src/Yahtzee.Core/Yahtzee.Core.csproj · 19 lines`

Defines the class library that contains **all game logic** and no user-interface code.

```
1  <Project Sdk="Microsoft.NET.Sdk">
2
3    <PropertyGroup>
```

Lines 1–3 Uses the standard .NET SDK project type.

```
4      <TargetFramework>net10.0</TargetFramework>
```

Line 4 Builds for plain `net10.0` (not `net10.0-windows`): the library uses no Windows-only APIs, so it is portable and can be unit tested anywhere.

```
5      <RootNamespace>Yahtzee.Core</RootNamespace>
```

Line 5 Default namespace for new files created in Visual Studio.

```
6      <!-- XML doc comments on public APIs feed the developer guide; missing ones fail the build. -->
7      <GenerateDocumentationFile>true</GenerateDocumentationFile>
```

Lines 6–7 Generates an XML documentation file from the `///` comments. Combined with warnings-as-errors this means every public type and member **must** have a documentation comment, which supports the "sufficient comments" marking criterion.

```
8    </PropertyGroup>
9
```

Line 8 Ends the property group.

```
10   <ItemGroup>
11     <InternalsVisibleTo Include="Yahtzee.Core.Tests" />
12   </ItemGroup>
13
```

Lines 10–12 `InternalsVisibleTo` lets the test project see members marked `internal` (useful for tests; currently no internal members are needed by tests).

```
14   <ItemGroup>
15     <!-- SQLite database engine for the games history (bundles the native SQLite library). -->
16     <PackageReference Include="Microsoft.Data.Sqlite" Version="10.0.12" />
17   </ItemGroup>
18
```

Lines 14–17 The library's only external dependency: **Microsoft.Data.Sqlite** 10.0.12, Microsoft's ADO.NET provider for SQLite. It brings in `SQLitePCLRaw`, which bundles the native SQLite engine, so nothing has to be installed separately. This dependency must be declared in the report.

```
19 </Project>
```

Line 19 End of the project file.

9.5 Windows Forms application project

`src/Yahtzee.WinForms/Yahtzee.WinForms.csproj · 17 lines`

Defines the executable that shows the game window.

```
1  <Project Sdk="Microsoft.NET.Sdk">
2
3    <PropertyGroup>
```

Lines 1–3 Standard .NET SDK project.

```
4      <OutputType>WinExe</OutputType>
```

Line 4 `WinExe` produces a Windows GUI executable (no console window appears).

```
5      <TargetFramework>net10.0-windows</TargetFramework>
```

Line 5 `net10.0-windows` is required for Windows Forms, which only exists on Windows.

```
6      <UseWindowsForms>true</UseWindowsForms>
```

Line 6 Pulls in the Windows Forms framework and its implicit usings.

```
7      <ApplicationHighDpiMode>PerMonitorV2</ApplicationHighDpiMode>
```

Line 7 `PerMonitorV2` tells Windows the app scales itself for each monitor's DPI, so text stays sharp at 125%/150% display scaling instead of being blurry-stretched.

```
8      <RootNamespace>Yahtzee.WinForms</RootNamespace>
```

Line 8 Default namespace.

```
9      <AssemblyName>Yahtzee</AssemblyName>
10     <Product>Yahtzee</Product>
```

Lines 9–10 The output file is called `Yahtzee.exe` (instead of `Yahtzee.WinForms.exe`); `Product` is the name shown in the file's properties.

```
11   </PropertyGroup>
12
```

Line 11 Ends the property group.

```
13   <ItemGroup>
14     <ProjectReference Include="..\Yahtzee.Core\Yahtzee.Core.csproj" />
15   </ItemGroup>
16
```

Lines 13–15 A project reference to `Yahtzee.Core`: the UI uses the engine, but the engine knows nothing about the UI.

```
17 </Project>
```

Line 17 End of the project file.

9.6 Unit-test project

`tests/Yahtzee.Core.Tests/Yahtzee.Core.Tests.csproj · 22 lines`

An xUnit test project that exercises the Core library.

```
1  <Project Sdk="Microsoft.NET.Sdk">
2
```

Line 1 Standard .NET SDK project.

```
3    <PropertyGroup>
```

Line 3 Start of the project settings.

```
4      <TargetFramework>net10.0</TargetFramework>
```

Line 4 Same framework as Core.

```
5      <IsPackable>false</IsPackable>
6    </PropertyGroup>
7
```

Lines 5–6 Prevents the test project from ever being packed as a NuGet package. Closes the property group.

```
8    <ItemGroup>
```

Line 8 Start of the NuGet package references (the only external dependencies in the solution so far, and only for tests).

```
9      <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.14.1" />
```

Line 9 The test platform host that lets `dotnet test` and Visual Studio discover and run tests.

```
10     <PackageReference Include="xunit" Version="2.9.3" />
```

Line 10 xUnit 2.9.3, the testing framework (`[Fact]`, `[Theory]`, `Assert`).

```
11     <PackageReference Include="xunit.runner.visualstudio" Version="3.1.4" />
12   </ItemGroup>
13
```

Lines 11–12 Adapter that connects xUnit to Visual Studio's Test Explorer and `dotnet test`.

```
14   <ItemGroup>
15     <Using Include="Xunit" />
16   </ItemGroup>
17
```

Lines 14–16 Adds a global `using Xunit;` to every test file, so the attribute and assertion types are always in scope.

```
18   <ItemGroup>
19     <ProjectReference Include="..\..\src\Yahtzee.Core\Yahtzee.Core.csproj" />
20   </ItemGroup>
21
```

Lines 18–20 References the Core project under test.

```
22 </Project>
```

Line 22 End of the project file.

9.7 Publish profile for the submission executable

`src/Yahtzee.WinForms/Properties/PublishProfiles/Standalone.pubxml · 21 lines`

Settings used by `dotnet publish src/Yahtzee.WinForms -p:PublishProfile=Standalone` (or Visual Studio's Publish command) to produce the single standalone `publish/Yahtzee.exe` that the specification requires.

```
1  <?xml version="1.0" encoding="utf-8"?>
```

Line 1 XML declaration.

```
2  <!--
3    Builds the standalone executable for submission: one Yahtzee.exe that runs on any 64-bit Windows 11 PC
4    without installing .NET (the runtime and the native SQLite library are bundled inside it).
5    Use:  dotnet publish src/Yahtzee.WinForms -p:PublishProfile=Standalone
6    or, in Visual Studio: right-click Yahtzee.WinForms > Publish > Standalone.
7  -->
```

Lines 2–7 Comment explaining what the profile produces and how to run it.

```
8  <Project>
9    <PropertyGroup>
```

Lines 8–9 An MSBuild file whose properties are applied on top of the project when this profile is selected.

```
10     <Configuration>Release</Configuration>
```

Line 10 Optimised Release build.

```
11     <RuntimeIdentifier>win-x64</RuntimeIdentifier>
```

Line 11 Target 64-bit Windows (`win-x64`), the lab machines' platform.

```
12     <SelfContained>true</SelfContained>
```

Line 12 Include the .NET runtime itself, so the PC does not need .NET installed.

```
13     <PublishSingleFile>true</PublishSingleFile>
```

Line 13 Pack everything into one `.exe`.

```
14     <IncludeNativeLibrariesForSelfExtract>true</IncludeNativeLibrariesForSelfExtract>
```

Line 14 Also pack native DLLs (the SQLite engine `e_sqlite3.dll`); they are extracted automatically at run time.

```
15     <EnableCompressionInSingleFile>true</EnableCompressionInSingleFile>
```

Line 15 Compress the bundled files: the exe is about 53 MB instead of about 150 MB.

```
16     <!-- Only the .exe is needed: no debug symbols or XML documentation files next to it. -->
17     <DebugType>none</DebugType>
18     <AllowedReferenceRelatedFileExtensions>-</AllowedReferenceRelatedFileExtensions>
```

Lines 16–18 No debug-symbol (`.pdb`) or documentation (`.xml`) files next to the exe; `-` is a placeholder meaning "no extensions".

```
19     <PublishDir>$(MSBuildThisFileDirectory)..\..\..\..\publish\</PublishDir>
```

Line 19 Output to the `publish/` folder at the repository root (four folders up from this file).

```
20   </PropertyGroup>
21 </Project>
```

Lines 20–21 End of the settings.

10. Line-by-line walkthrough: Yahtzee.Core: the game logic

10.1 The 13 scorecard categories

`src/Yahtzee.Core/Model/Category.cs · 64 lines`

An enumeration of the 13 boxes on the scorecard, plus small helper (extension) methods.

```
1  namespace Yahtzee.Core.Model;
2
```

Line 1 A *file-scoped namespace* (C# 10): everything in the file belongs to `Yahtzee.Core.Model` without an extra level of braces.

```
3  /// <summary>
4  /// The 13 scoring categories (boxes) on a Yahtzee scorecard, in the order shown in
5  /// Figure 1 of the coursework specification.
6  /// </summary>
```

Lines 3–6 XML documentation comment shown in IntelliSense and used in the developer guide.

```
7  /// <remarks>
8  /// The numeric values are stable and are used by the save-file format and the database,
9  /// so new members must not be inserted or reordered.
10 /// </remarks>
```

Lines 7–10 Important design note: the numeric values are **stable**. They will be written into save files and the database, so reordering the enum would corrupt old saves.

```
11 public enum Category
12 {
```

Lines 11–12 Declares the enumeration. An `enum` is a set of named integer constants.

```
13     /// <summary>Sum of dice showing 1.</summary>
14     Aces = 0,
15     /// <summary>Sum of dice showing 2.</summary>
16     Twos = 1,
17     /// <summary>Sum of dice showing 3.</summary>
18     Threes = 2,
19     /// <summary>Sum of dice showing 4.</summary>
20     Fours = 3,
21     /// <summary>Sum of dice showing 5.</summary>
22     Fives = 4,
23     /// <summary>Sum of dice showing 6.</summary>
24     Sixes = 5,
```

Lines 13–24 The upper-section categories `Aces` (0) to `Sixes` (5). Each has a doc comment and an explicit value. Their order matters: `Face()` below relies on `Aces`=0 … `Sixes`=5 so the face is value+1.

```
25     /// <summary>Sum of all dice, any combination.</summary>
26     Chance = 6,
27     /// <summary>Sum of all dice if at least three are the same.</summary>
28     ThreeOfAKind = 7,
29     /// <summary>Sum of all dice if at least four are the same.</summary>
30     FourOfAKind = 8,
```

Lines 25–30 `Chance`, `ThreeOfAKind` and `FourOfAKind`: categories that score the sum of all five dice (if their condition is met).

```
31     /// <summary>25 points for three of one value and two of another.</summary>
32     FullHouse = 9,
33     /// <summary>30 points for four sequential dice.</summary>
34     SmallStraight = 10,
35     /// <summary>40 points for five sequential dice.</summary>
36     LargeStraight = 11,
37     /// <summary>50 points for all five dice the same.</summary>
38     Yahtzee = 12,
```

Lines 31–38 The fixed-score categories: Full House (25), Small Straight (30), Large Straight (40) and Yahtzee (50). The order follows Figure 1 of the specification.

```
39 }
40
```

Line 39 End of the enum.

```
/// <summary>Helper members for <see cref="Category"/>.</summary>
public static class CategoryExtensions
{
```

Lines 41–43 A `static class` holding *extension methods* for `Category`. An enum cannot contain methods, so this is the idiomatic C# way to add behaviour to it.

```
    /// <summary>All categories in scorecard order.</summary>
    public static IReadOnlyList<Category> All { get; } = Enum.GetValues<Category>();

```

Lines 44–45 `All`: every category in declaration order, computed once by `Enum.GetValues<Category>()` and exposed read-only. Used to build the scorecard rows and to count rounds (13).

```
    /// <summary>Human-readable name for display in the GUI and reports.</summary>
    public static string DisplayName(this Category category) => category switch
    {
        Category.ThreeOfAKind => "Three of a Kind",
        Category.FourOfAKind => "Four of a Kind",
        Category.FullHouse => "Full House",
        Category.SmallStraight => "Small Straight",
        Category.LargeStraight => "Large Straight",
        _ => category.ToString(),
    };

```

Lines 47–56 `DisplayName` is an extension method (note `this Category`) so it is called as `category.DisplayName()`. It uses a **switch expression**: the five multi-word names get spaces; the discard pattern `_` falls back to the enum name for the rest ("Aces", "Chance", "Yahtzee" ...).

```
    /// <summary>
    /// For the upper-section categories (Aces to Sixes), the die face they count;
    /// otherwise <see langword="null"/>.
    /// </summary>
    public static int? Face(this Category category) =>
        category is >= Category.Aces and <= Category.Sixes ? (int)category + 1 : null;
```

Lines 58–63 `Face()` returns which die face an upper-section category counts (Aces→1 ... Sixes→6) or `null` for the other categories. `is >= Category.Aces and <= Category.Sixes` is a **relational pattern** combined with `and`. The return type `int?` is a nullable int.

```
}
```

Line 64 End of the static class.

10.2 Scoring rules

`src/Yahtzee.Core/Rules/Scorer.cs · 104 lines`

Pure functions implementing the **simplified rules of Appendix B**: given a category and five dice, return the score. "Pure" means no state and no side effects: the same input always gives the same output, which makes the rules easy to test.

```
1  using Yahtzee.Core.Model;
2
```

Line 1 Imports the namespace containing `Category`.

```
3  namespace Yahtzee.Core.Rules;
4
```

Line 3 The rules live in their own namespace, separate from the data model.

```
5  /// <summary>
6  /// Pure scoring functions implementing the simplified rules from Appendix B of the
7  /// coursework specification. There is no upper-section bonus, Yahtzee bonus or joker rule.
8  /// </summary>
```

Lines 5–8 Class documentation. States explicitly that there are no bonuses or joker rules (those belong to the full Yahtzee rules, which the specification says not to implement).

```
9  public static class Scorer
10 {
```

Lines 9–10 A `static` class cannot be instantiated; it only groups functions and constants.

```
11     /// <summary>Number of dice in play.</summary>
12     public const int DiceCount = 5;
13
```

Lines 11–12 `const` values are compile-time constants. Naming the magic numbers (5 dice) makes the code self-explanatory.

```
14     /// <summary>Lowest face value of a die.</summary>
15     public const int MinFace = 1;
16
```

Lines 14–15 Smallest die face (1).

```
17     /// <summary>Highest face value of a die.</summary>
18     public const int MaxFace = 6;
19
```

Lines 17–18 Largest die face (6).

```
20     /// <summary>Fixed score for a Full House.</summary>
21     public const int FullHouseScore = 25;
22
23     /// <summary>Fixed score for a Small Straight.</summary>
24     public const int SmallStraightScore = 30;
25
26     /// <summary>Fixed score for a Large Straight.</summary>
27     public const int LargeStraightScore = 40;
28
29     /// <summary>Fixed score for a Yahtzee.</summary>
30     public const int YahtzeeScore = 50;
31
```

Lines 20–30 The four fixed scores from Figure 1 (25, 30, 40, 50). The UI's help text reads these constants, so the numbers are written in exactly one place.

```
32     /// <summary>
33     /// Calculates the score the given dice would earn in <paramref name="category"/>.
34     /// Dice that do not meet the category's requirement score 0.
35     /// </summary>
36     /// <param name="category">The category to score.</param>
37     /// <param name="dice">Exactly five die values, each between 1 and 6.</param>
38     /// <exception cref="ArgumentException">The dice are not five values in the range 1–6.</exception>
```

Lines 32–38 Documentation for `Score`, including the exception it can throw.

```
39      public static int Score(Category category, IReadOnlyList<int> dice)
40      {
```

Lines 39–40 The main function. Parameters are a `Category` and `IReadOnlyList<int>`: an interface that accepts arrays, lists and collection expressions while promising not to modify them.

```
41          ValidateDice(dice);
```

Line 41 Rejects invalid input first (wrong number of dice or faces outside 1–6), so the rest of the method can assume valid dice.

```
42          var counts = CountFaces(dice);
43          int sum = dice.Sum();
44
```

Lines 42–43 Builds a **frequency table**: `counts[f]` = how many dice show face `f`. Almost every rule can be decided from this table. `sum` is the total of all dice (LINQ `Sum()`).

```
45          return category switch
46          {
```

Lines 45–46 A **switch expression** picks the formula for the category and returns its value.

```
47              >= Category.Aces and <= Category.Sixes => SumOfFace(counts, category.Face()!.Value),
```

Line 47 Upper section: one arm handles all six categories with a relational pattern; `SumOfFace` multiplies the count of that face by the face. The `!` after `Face()` tells the compiler the nullable result is definitely not null here.

```
48              Category.Chance => sum,
```

Line 48 Chance always scores the sum.

```
49              Category.ThreeOfAKind => counts.Max() >= 3 ? sum : 0,
50              Category.FourOfAKind => counts.Max() >= 4 ? sum : 0,
```

Lines 49–50 Three/Four of a Kind: "at least" N equal dice, so the *largest* count must be ≥ 3 (or ≥ 4). A Yahtzee therefore also counts, as Appendix B says "at least".

```
51              // Exactly a 3 + 2 split: a Yahtzee (5 of one value) is not a Full House.
52              Category.FullHouse => counts.Contains(3) && counts.Contains(2) ? FullHouseScore : 0,
```

Lines 51–52 Full House needs one face appearing exactly 3 times and another exactly 2 times. With 5 dice, five-of-a-kind has counts {5} and contains neither 3 nor 2, so **a Yahtzee is correctly rejected** (the note under Figure 1).

```
53              Category.SmallStraight => LongestRun(counts) >= 4 ? SmallStraightScore : 0,
54              Category.LargeStraight => LongestRun(counts) == 5 ? LargeStraightScore : 0,
```

Lines 53–54 Straights use `LongestRun`: the longest sequence of consecutive faces that are present. Small Straight needs a run ≥ 4 (so a large straight also qualifies), Large Straight needs a run of exactly 5.

```
55              Category.Yahtzee => counts.Contains(DiceCount) ? YahtzeeScore : 0,
```

Line 55 Yahtzee: some face appears 5 times.

```
56              _ => throw new ArgumentOutOfRangeException(nameof(category), category, "Unknown category."),
57          };
58      }
59
```

Lines 56–58 Defensive default: an undefined enum value (e.g. `(Category)99`) throws instead of silently scoring 0.

```
60      /// <summary>
61      /// Scores the dice against each of the given categories, for the score preview.
62      /// </summary>
63      /// <param name="dice">Exactly five die values, each between 1 and 6.</param>
64      /// <param name="categories">The categories to preview, typically the player's unfilled ones.</param>
65      /// <returns>A map from each category to the score it would earn.</returns>
```

Lines 60–65 Documentation for `Preview`.

```
66      public static IReadOnlyDictionary<Category, int> Preview(IReadOnlyList<int> dice, IEnumerable<Category> categories) =>
67          categories.ToDictionary(category => category, category => Score(category, dice));
68
```

Lines 66–67 Scores the dice in several categories at once and returns a dictionary category → points. This powers the **score preview**. It is an expression-bodied member (`=>`) using LINQ `ToDictionary`.

```
69      /// <summary>
70      /// Throws if <paramref name="dice"/> is not exactly five values in the range 1–6.
71      /// </summary>
72      /// <exception cref="ArgumentException">The dice are invalid.</exception>
```

Lines 69–72 Documentation for `ValidateDice`.

```
73      public static void ValidateDice(IReadOnlyList<int> dice)
74      {
75          ArgumentNullException.ThrowIfNull(dice);
```

Lines 73–75 Public because `DiceCup` reuses it when restoring saved dice. `ThrowIfNull` is the modern one-line null guard.

```
76          if (dice.Count != DiceCount)
77              throw new ArgumentException($"Expected {DiceCount} dice but got {dice.Count}.", nameof(dice));
```

Lines 76–77 Exactly five dice are required.

```
78          if (dice.Any(value => value is < MinFace or > MaxFace))
79              throw new ArgumentException($"Die values must be between {MinFace} and {MaxFace}.", nameof(dice));
80      }
81
```

Lines 78–80 Every value must be 1–6. `value is < MinFace or > MaxFace` is a pattern with `or`.

```
82      /// <summary>Returns how many dice show each face; index 0 is unused so index = face.</summary>
83      private static int[] CountFaces(IReadOnlyList<int> dice)
84      {
```

Lines 82–84 `CountFaces` builds the frequency table.

```
85          var counts = new int[MaxFace + 1];
86          foreach (int value in dice)
87              counts[value]++;
88          return counts;
89      }
90
```

Lines 85–89 An array of 7 integers (index 0 unused) so `counts[face]` can be indexed by the face directly. Each die increments its slot.

```
91      private static int SumOfFace(int[] counts, int face) => counts[face] * face;
92
```

Line 91 Upper-section score: (number of dice showing `face`) × `face`.

```
93      /// <summary>Length of the longest run of consecutive faces present at least once.</summary>
94      private static int LongestRun(int[] counts)
95      {
```

Lines 93–95 `LongestRun` scans faces 1→6.

```
96          int longest = 0, current = 0;
97          for (int face = MinFace; face <= MaxFace; face++)
98          {
```

Lines 96–98 `current` is the length of the run ending at the current face; `longest` is the best seen.

```
99              current = counts[face] > 0 ? current + 1 : 0;
100             longest = Math.Max(longest, current);
101         }
102         return longest;
103     }
104 }
```

Lines 99–104 If the face is present the run grows, otherwise it resets to 0. Duplicates do not matter because only presence is checked. Example: dice 2 3 4 5 5 → runs 0,1,2,3,4,0 → longest 4 → small straight.

10.3 Random-number abstraction

`src/Yahtzee.Core/Model/IDiceRoller.cs · 23 lines`

Separates *where die values come from* from the code that uses them, so tests can supply predictable values.

```
1  using Yahtzee.Core.Rules;
2
```

Line 1 Needed for the `Scorer.MinFace`/`MaxFace` constants.

```
3  namespace Yahtzee.Core.Model;
4
```

Line 3 Model namespace.

```
5  /// <summary>
6  /// Source of die rolls. Abstracted so that tests and AI simulations can use
7  /// deterministic values instead of real randomness.
8  /// </summary>
```

Lines 5–8 Explains why the abstraction exists (testability).

```
9  public interface IDiceRoller
10 {
```

Lines 9–10 An **interface** is a contract: any class that implements it must provide `RollDie()`.

```
11     /// <summary>Returns a value between 1 and 6 inclusive.</summary>
12     int RollDie();
13 }
14
```

Lines 11–13 The single method: return one die value 1–6.

```
15 /// <summary>An <see cref="IDiceRoller"/> backed by <see cref="Random"/>.</summary>
16 /// <param name="random">The random source; defaults to <see cref="Random.Shared"/>.</param>
```

Lines 15–16 Documentation for the real implementation.

```
17 public sealed class RandomDiceRoller(Random? random = null) : IDiceRoller
18 {
```

Lines 17–18 Uses a **primary constructor** (C# 12): the parameter `random` is declared on the class line. It is optional (`= null`). `sealed` means no class can inherit from it.

```
19     private readonly Random _random = random ?? Random.Shared;
20
```

Line 19 If no `Random` was supplied, use the thread-safe shared instance `Random.Shared` (the `??` null-coalescing operator).

```
21     /// <inheritdoc/>
22     public int RollDie() => _random.Next(Scorer.MinFace, Scorer.MaxFace + 1);
23 }
```

Lines 21–23 `Next(1, 7)` returns 1–6 (the upper bound is exclusive, hence `MaxFace + 1`).

10.4 The five dice and their held flags

`src/Yahtzee.Core/Model/DiceCup.cs · 88 lines`

Stores the five die values and which dice are held, and re-rolls the non-held ones. It deliberately does **not** count rolls; the three-roll limit is the game engine's job.

```
1  using Yahtzee.Core.Rules;
2
```

Line 1 For `Scorer.DiceCount` and `Scorer.ValidateDice`.

```
3  namespace Yahtzee.Core.Model;
4
```

Line 3 Model namespace.

```
5  /// <summary>
6  /// The five dice in play and which of them are held (set aside) for the next roll.
7  /// </summary>
```

Lines 5–7 Class summary.

```
8  /// <remarks>
9  /// This class only tracks dice; the limit of three rolls per turn is enforced by the game engine.
10 /// </remarks>
```

Lines 8–10 Documents the division of responsibility with `GameEngine`.

```
11 public sealed class DiceCup
12 {
```

Lines 11–12 Class declaration.

```
13     private readonly int[] _values = new int[Scorer.DiceCount];
14     private readonly bool[] _held = new bool[Scorer.DiceCount];
15     private readonly IDiceRoller _roller;
16
```

Lines 13–15 Two parallel fixed-size arrays: the face of each die and whether it is held. `readonly` means the array reference never changes (its contents can). The roller is stored for later rolls.

```
17     /// <summary>Creates a cup of unrolled dice.</summary>
18     /// <param name="roller">Source of die values.</param>
19     public DiceCup(IDiceRoller roller)
20     {
21         _roller = roller ?? throw new ArgumentNullException(nameof(roller));
22     }
23
```

Lines 17–22 Constructor for a fresh cup. A null roller is rejected using a *throw expression* (`?? throw ...`).

```
24     /// <summary>
25     /// Creates a cup restored to a previous state, for example when loading a saved game.
26     /// </summary>
27     /// <param name="roller">Source of die values.</param>
28     /// <param name="values">Five die values between 1 and 6.</param>
29     /// <param name="held">Five flags saying which dice are held.</param>
30     /// <exception cref="ArgumentException">The values or flags are invalid.</exception>
```

Lines 24–30 Documentation of the second, "restore" constructor.

```
31     public DiceCup(IDiceRoller roller, IReadOnlyList<int> values, IReadOnlyList<bool> held) : this(roller)
32     {
```

Lines 31–32 Used when loading a saved game. `: this(roller)` first runs the simple constructor (constructor chaining).

```
33         Scorer.ValidateDice(values);
34         ArgumentNullException.ThrowIfNull(held);
35         if (held.Count != Scorer.DiceCount)
36             throw new ArgumentException($"Expected {Scorer.DiceCount} held flags but got {held.Count}.", nameof(held));
37
```

Lines 33–36 Validates the saved values and the five held flags, so a corrupted save cannot create impossible dice.

```
38          for (int i = 0; i < Scorer.DiceCount; i++)
39          {
40              _values[i] = values[i];
41              _held[i] = held[i];
42          }
```

Lines 38–42 Copies the saved values and flags into the arrays.

```
43          HasRolled = true;
44      }
45
```

Lines 43–44 A restored cup always represents dice that have been rolled.

```
46      /// <summary>Whether the dice have been rolled since the cup was created or reset.</summary>
47      public bool HasRolled { get; private set; }
48
```

Lines 46–47 `HasRolled`: auto-property with a private setter (readable everywhere, changeable only here).

```
49      /// <summary>The current die values. Only meaningful once <see cref="HasRolled"/> is true.</summary>
50      public IReadOnlyList<int> Values => _values;
51
```

Lines 49–50 Exposes the array as `IReadOnlyList<int>` so callers can read but not modify it.

```
52      /// <summary>Which dice are held and will not change on the next roll.</summary>
53      public IReadOnlyList<bool> Held => _held;
54
```

Lines 52–53 Same for the held flags.

```
55      /// <summary>Rolls every die that is not held. The first roll always rolls all five.</summary>
56      public void Roll()
57      {
58          for (int i = 0; i < Scorer.DiceCount; i++)
59          {
```

Lines 55–59 `Roll()` loops over all five dice.

```
60              if (!HasRolled || !_held[i])
61                  _values[i] = _roller.RollDie();
62          }
63          HasRolled = true;
64      }
65
```

Lines 60–64 A die gets a new value if this is the first roll (holds are ignored before any roll) or it is not held. Then `HasRolled` becomes true.

```
66      /// <summary>Holds or releases the die at <paramref name="index"/>.</summary>
67      /// <exception cref="InvalidOperationException">The dice have not been rolled yet.</exception>
68      /// <exception cref="ArgumentOutOfRangeException"><paramref name="index"/> is not 0–4.</exception>
69      public void SetHeld(int index, bool held)
70      {
```

Lines 66–70 `SetHeld` documentation and signature.

```
71          if (!HasRolled)
72              throw new InvalidOperationException("Dice cannot be held before the first roll.");
```

Lines 71–72 You cannot hold dice that have not been rolled yet.

```
73          ArgumentOutOfRangeException.ThrowIfNegative(index);
74          ArgumentOutOfRangeException.ThrowIfGreaterThanOrEqual(index, Scorer.DiceCount);
75          _held[index] = held;
76      }
77
```

Lines 73–76 Index must be 0–4 (built-in .NET guard helpers), then the flag is stored.

```
78      /// <summary>Flips the held state of the die at <paramref name="index"/>.</summary>
79      public void ToggleHeld(int index) => SetHeld(index, !_held[index]);
80
```

Lines 78–79 `ToggleHeld` flips the flag by calling `SetHeld` with the opposite value (reusing its checks).

```
81      /// <summary>Releases all dice and marks the cup as unrolled, ready for the next turn.</summary>
82      public void Reset()
83      {
84          Array.Clear(_values);
85          Array.Clear(_held);
86          HasRolled = false;
87      }
88  }
```

Lines 81–88 `Reset()` prepares the cup for the next player's turn: clears values and holds and marks it unrolled.

10.5 One player's scorecard

`src/Yahtzee.Core/Model/ScoreCard.cs · 73 lines`

Stores which categories a player has filled and with what score, enforces "each category once", and computes totals and previews.

```
1  using Yahtzee.Core.Rules;
2
```

Line 1 For `Scorer`.

```
3  namespace Yahtzee.Core.Model;
4
```

Line 3 Model namespace.

```
5  /// <summary>
6  /// One player's scorecard: the 13 category boxes, each empty or holding a recorded score.
7  /// </summary>
```

Lines 5–7 Class summary.

```
8  public sealed class ScoreCard
9  {
```

Lines 8–9 Class declaration.

```
10     private readonly Dictionary<Category, int> _scores = [];
11
```

Line 10 The data structure: a dictionary from category to recorded score. An **absent key means the box is empty**, so no separate "filled" flags are needed. `[]` is a collection expression creating an empty dictionary.

```
12     /// <summary>Creates an empty scorecard.</summary>
13     public ScoreCard()
14     {
15     }
16
```

Lines 12–15 Parameterless constructor: new empty card.

```
17     /// <summary>
18     /// Creates a scorecard restored from previously recorded scores, for example when loading a saved game.
19     /// </summary>
20     /// <param name="scores">Recorded scores; categories that are absent are empty.</param>
21     /// <exception cref="ArgumentException">A score is negative.</exception>
```

Lines 17–21 Documentation of the restore constructor.

```
22     public ScoreCard(IReadOnlyDictionary<Category, int> scores)
23     {
24         ArgumentNullException.ThrowIfNull(scores);
```

Lines 22–24 Builds a card from saved scores (for loading games).

```
25         foreach (var (category, score) in scores)
26         {
```

Lines 25–26 **Deconstruction**: each `KeyValuePair` is split into two variables.

```
27             if (!Enum.IsDefined(category))
28                 throw new ArgumentException($"Unknown category {category}.", nameof(scores));
```

Lines 27–28 Rejects category numbers that do not exist (e.g. from a hand-edited save file).

```
29             if (score < 0)
30                 throw new ArgumentException($"Score for {category} cannot be negative.", nameof(scores));
```

Lines 29–30 Scores can never be negative.

```
31              _scores[category] = score;
32          }
33      }
34
```

Lines 31–33 Stores the validated score.

```
35      /// <summary>The recorded scores, keyed by category. Empty categories are absent.</summary>
36      public IReadOnlyDictionary<Category, int> Scores => _scores;
37
```

Lines 35–36 Read-only view of the dictionary.

```
38      /// <summary>The categories that have not been scored yet, in scorecard order.</summary>
39      public IEnumerable<Category> AvailableCategories =>
40          CategoryExtensions.All.Where(category => !_scores.ContainsKey(category));
41
```

Lines 38–40 LINQ `Where` filters the full category list to those not yet in the dictionary, keeping scorecard order.

```
42      /// <summary>Whether all 13 categories have been scored.</summary>
43      public bool IsComplete => _scores.Count == CategoryExtensions.All.Count;
44
```

Lines 42–43 The card is complete when it contains all 13 categories.

```
45      /// <summary>The total score: the sum of all recorded boxes.</summary>
46      public int Total => _scores.Values.Sum();
47
```

Lines 45–46 Total = sum of all recorded boxes (no bonuses in the simplified rules).

```
48      /// <summary>Whether <paramref name="category"/> has already been scored.</summary>
49      public bool IsFilled(Category category) => _scores.ContainsKey(category);
50
```

Lines 48–49 `IsFilled`: is the category a key in the dictionary?

```
51      /// <summary>The score recorded in <paramref name="category"/>, or <see langword="null"/> if empty.</summary>
52      public int? GetScore(Category category) => _scores.TryGetValue(category, out int score) ? score : null;
53
```

Lines 51–52 `GetScore` returns the score or `null` for an empty box, using `TryGetValue` and the conditional operator.

```
54      /// <summary>
55      /// Records the score for <paramref name="dice"/> in <paramref name="category"/>.
56      /// Dice that do not match the category score 0.
57      /// </summary>
58      /// <returns>The score that was recorded.</returns>
59      /// <exception cref="InvalidOperationException">The category has already been scored.</exception>
```

Lines 54–59 Documentation of `Record`.

```
60      public int Record(Category category, IReadOnlyList<int> dice)
61      {
```

Lines 60–61 Records a score for the given dice.

```
62          if (IsFilled(category))
63              throw new InvalidOperationException($"{category.DisplayName()} has already been scored.");
64
```

Lines 62–63 Enforces the rule "once a score has been entered for a category, it may not be used again".

```
65          int score = Scorer.Score(category, dice);
66          _scores[category] = score;
67          return score;
68      }
69
```

Lines 65–68 Delegates the arithmetic to `Scorer`, stores and returns it. Non-matching dice score 0.

```
70      /// <summary>Previews the score <paramref name="dice"/> would earn in each available category.</summary>
71      public IReadOnlyDictionary<Category, int> Preview(IReadOnlyList<int> dice) =>
72          Scorer.Preview(dice, AvailableCategories);
73  }
```

Lines 70–73 Preview for this player: scores only the categories still available.

10.6 A player

`src/Yahtzee.Core/Model/Player.cs · 49 lines`

A name, a scorecard and an optional AI strategy. Human and computer players share one class; the difference is whether a strategy is present.

```
1  using Yahtzee.Core.AI;
2
```

Line 1 For `IPlayerStrategy`.

```
3  namespace Yahtzee.Core.Model;
4
```

Line 3 Model namespace.

```
5  /// <summary>
6  /// A participant in the game: a name, a scorecard, and, for computer-controlled players, a strategy.
7  /// </summary>
```

Lines 5–7 Class summary.

```
8  public sealed class Player
9  {
```

Lines 8–9 Class declaration.

```
10     /// <summary>Longest allowed player name.</summary>
11     public const int MaxNameLength = 20;
12
```

Lines 10–11 Maximum name length (20) so names fit the scorecard columns; the new-game dialog uses it as the text box limit.

```
13     /// <summary>Creates a player.</summary>
14     /// <param name="name">Display name; surrounding whitespace is trimmed.</param>
15     /// <param name="strategy">The AI strategy, or <see langword="null"/> for a human player.</param>
16     /// <param name="scoreCard">An existing scorecard (when loading a save), or <see langword="null"/> for a new empty one.
</param>
17     /// <exception cref="ArgumentException">The name is blank or longer than <see cref="MaxNameLength"/>.</exception>
```

Lines 13–17 Constructor documentation.

```
18     public Player(string name, IPlayerStrategy? strategy = null, ScoreCard? scoreCard = null)
19     {
```

Lines 18–19 Optional parameters: `strategy` (null = human) and `scoreCard` (null = new empty card; a saved card when loading).

```
20         ArgumentNullException.ThrowIfNull(name);
21         name = name.Trim();
```

Lines 20–21 Rejects null, then trims surrounding spaces.

```
22         if (name.Length == 0)
23             throw new ArgumentException("Player name cannot be blank.", nameof(name));
24         if (name.Length > MaxNameLength)
25             throw new ArgumentException($"Player name cannot be longer than {MaxNameLength} characters.", nameof(name));
26
```

Lines 22–25 Rejects blank names and names longer than the limit.

```
27         Name = name;
28         Strategy = strategy;
29         ScoreCard = scoreCard ?? new ScoreCard();
30     }
31
```

Lines 27–30 Stores the values; `??` creates a new card when none was given.

```
32     /// <summary>The player's display name.</summary>
33     public string Name { get; }
34
```

Lines 32–33 Get-only auto-property: set once in the constructor, never changed (immutable).

```
35      /// <summary>The AI strategy, or <see langword="null"/> for a human player.</summary>
36      public IPlayerStrategy? Strategy { get; }
37
```

Lines 35–36 Nullable strategy.

```
38      /// <summary>Whether the player is computer-controlled.</summary>
39      public bool IsAi => Strategy is not null;
40
```

Lines 38–39 `IsAi` is derived from the strategy using the `is not null` pattern.

```
41      /// <summary>The player's scorecard.</summary>
42      public ScoreCard ScoreCard { get; }
43
```

Lines 41–42 The player's card.

```
44      /// <summary>The player's current total score.</summary>
45      public int Total => ScoreCard.Total;
46
```

Lines 44–45 Convenience shortcut to the card's total.

```
47      /// <inheritdoc/>
48      public override string ToString() => IsAi ? $"{Name} (AI: {Strategy!.Name})" : Name;
49  }
```

Lines 47–49 `ToString()` override used for debugging and display, e.g. "Bot (AI: Greedy)".

10.7 AI strategy contract

`src/Yahtzee.Core/AI/IPlayerStrategy.cs · 30 lines`

The interface every computer opponent will implement (strategies themselves are step 6). A strategy only *decides*; the engine applies the decision, so an AI can never break the rules.

```
1  using Yahtzee.Core.Model;
2
```

Line 1 For `Category` and `ScoreCard`.

```
3  namespace Yahtzee.Core.AI;
4
```

Line 3 AI namespace.

```
5  /// <summary>
6  /// Decision-making for a computer-controlled player. The game engine asks the strategy
7  /// what to do after each roll; the strategy never changes game state itself.
8  /// </summary>
```

Lines 5–8 Explains the decide-only design.

```
9  public interface IPlayerStrategy
10 {
```

Lines 9–10 Interface declaration.

```
11     /// <summary>Short display name of the strategy, also stored in save files.</summary>
12     string Name { get; }
13
```

Lines 11–12 `Name`: shown in the UI and stored in save files to recreate the right strategy.

```
14     /// <summary>
15     /// Decides which dice to keep before the next roll, or to stop rolling.
16     /// Only called when at least one roll remains in the turn.
17     /// </summary>
18     /// <param name="dice">The five current die values.</param>
19     /// <param name="rollsRemaining">Rolls left in this turn (1 or 2).</param>
20     /// <param name="scoreCard">The player's scorecard, for checking which categories are still open.</param>
21     /// <returns>
22     /// Five flags marking the dice to hold, or <see langword="null"/> to stop rolling and score now.
23     /// </returns>
```

Lines 14–23 Documentation for `ChooseHolds`: which dice to keep before the next roll, or stop.

```
24     IReadOnlyList<bool>? ChooseHolds(IReadOnlyList<int> dice, int rollsRemaining, ScoreCard scoreCard);
25
```

Line 24 Returns five flags (true = hold) or `null` meaning "stop rolling and score now".

```
26     /// <summary>Chooses which open category to score the final dice in.</summary>
27     /// <param name="dice">The five final die values.</param>
28     /// <param name="scoreCard">The player's scorecard; the result must be one of its available categories.</param>
29     Category ChooseCategory(IReadOnlyList<int> dice, ScoreCard scoreCard);
30 }
```

Lines 26–30 `ChooseCategory`: which open box to fill with the final dice.

10.8 Event data types

`src/Yahtzee.Core/Game/GameEvents.cs · 42 lines`

Small immutable **record** types carried by the engine's events, plus the result type of an AI step.

```
1  using Yahtzee.Core.Model;
2
```

Line 1 For `Player` and `Category`.

```
3  namespace Yahtzee.Core.Game;
4
```

Line 3 Game namespace.

```
5  /// <summary>Raised when a new turn begins.</summary>
6  /// <param name="Player">The player whose turn it is.</param>
7  /// <param name="Round">The round number, 1–13.</param>
8  public sealed record TurnStartedEventArgs(Player Player, int Round);
9
```

Lines 5–8 `record` with a primary constructor: one line defines properties `Player` and `Round`, a constructor, value-based equality and `ToString`. Value equality is what lets tests compare records with `Assert.Equal`.

```
10 /// <summary>Raised after the dice are rolled.</summary>
11 /// <param name="Player">The player who rolled.</param>
12 /// <param name="Dice">The five die values after the roll.</param>
13 /// <param name="RollsRemaining">Rolls left in this turn.</param>
14 public sealed record DiceRolledEventArgs(Player Player, IReadOnlyList<int> Dice, int RollsRemaining);
15
```

Lines 10–14 Who rolled, the five values, and rolls remaining (the UI shows "Player rolled 4 1 6 6 1").

```
16 /// <summary>Raised after a player records a score.</summary>
17 /// <param name="Player">The player who scored.</param>
18 /// <param name="Category">The category that was filled.</param>
19 /// <param name="Score">The points recorded.</param>
20 public sealed record CategoryScoredEventArgs(Player Player, Category Category, int Score);
21
```

Lines 16–20 Who scored, in which category, and how many points.

```
22 /// <summary>Raised once, when every player has filled all 13 categories.</summary>
23 /// <param name="Standings">Final results, ordered from highest to lowest score.</param>
24 /// <param name="CompletedAt">When the game finished.</param>
25 public sealed record GameOverEventArgs(IReadOnlyList<PlayerStanding> Standings, DateTimeOffset CompletedAt);
26
```

Lines 22–25 Final standings and completion time; the completion time will become the database's "date played".

```
27 /// <summary>One player's position in the results table.</summary>
28 /// <param name="Rank">1 for the winner. Tied players share a rank (1, 1, 3, ...).</param>
29 /// <param name="Name">The player's name.</param>
30 /// <param name="Score">The player's total score.</param>
31 /// <param name="IsAi">Whether the player was computer-controlled.</param>
32 public sealed record PlayerStanding(int Rank, string Name, int Score, bool IsAi);
33
```

Lines 27–32 One row of the results table: rank, name, score and whether the player was an AI.

```
34 /// <summary>The single action taken by <see cref="GameEngine.PlayAiStep"/>.</summary>
35 public enum AiStepResult
36 {
```

Lines 34–36 Enum returned by `PlayAiStep` so the UI knows whether the AI's turn has ended.

```
37     /// <summary>The AI rolled the dice (after choosing which to hold, if any).</summary>
38     Rolled,
39
40     /// <summary>The AI recorded a score, ending its turn.</summary>
41     Scored,
42 }
```

Lines 37–42 The two values: `Rolled` (turn continues) and `Scored` (turn over).

10.9 The game engine (state machine)

`src/Yahtzee.Core/Game/GameEngine.cs · 251 lines`

The heart of the program. It holds the game state, **enforces every turn rule**, and raises events so the UI can redraw. The UI never changes state directly; it only calls these methods.

```
1  using Yahtzee.Core.Model;
2  using Yahtzee.Core.Rules;
3
```

Lines 1–2 Imports the model and rules namespaces.

```
4  namespace Yahtzee.Core.Game;
5
```

Line 4 Game namespace.

```
6  /// <summary>
7  /// Runs a game of Yahtzee: whose turn it is, how many rolls are left, which dice are held,
8  /// and recording scores. The GUI calls the action methods and reacts to the events;
9  /// all rule enforcement happens here.
10 /// </summary>
```

Lines 6–10 Class summary: the engine owns the rules; the GUI only calls actions and listens to events.

```
11 /// <remarks>
12 /// Turn rules: a turn starts with no dice rolled. The player must roll at least once and may
13 /// roll up to <see cref="MaxRollsPerTurn"/> times, holding or releasing any dice between rolls.
14 /// After any roll the player may score one open category, which ends the turn and passes play
15 /// to the next player. The game ends when every player has filled all 13 categories.
16 /// <para>Saving and restoring games is in <c>GameEngine.Persistence.cs</c> (the other part of this partial class).</para>
17 /// </remarks>
```

Lines 11–17 Summary of the turn rules from Appendix B, as implemented below, and a pointer to the second file of the class.

```
18 public sealed partial class GameEngine
19 {
```

Lines 18–19 `partial` means the class is split over two files: this one (rules and play) and `GameEngine.Persistence.cs` (saving and restoring). The compiler joins them into one class, so both halves share the private state.

```
20     /// <summary>Maximum number of rolls in one turn.</summary>
21     public const int MaxRollsPerTurn = 3;
22
```

Lines 20–21 Three rolls per turn.

```
23     /// <summary>Maximum number of players in one game.</summary>
24     public const int MaxPlayers = 6;
25
```

Lines 23–24 The specification allows limiting the number of players; this game allows 6.

```
26     /// <summary>Number of rounds in a game: one per scorecard category.</summary>
27     public static int TotalRounds => CategoryExtensions.All.Count;
28
```

Lines 26–27 13 rounds, derived from the number of categories rather than hard-coded.

```
29     private readonly List<Player> _players;
30     private readonly DiceCup _dice;
31     private readonly TimeProvider _timeProvider;
32
```

Lines 29–31 The complete stored state, together with `CurrentPlayerIndex`, `RollsUsed` and `IsGameOver` below: the players, the dice, and the clock. Keeping state minimal makes saving simple.

```
33      /// <summary>Starts a new game with the first player to roll.</summary>
34      /// <param name="players">The players in turn order; 1 to <see cref="MaxPlayers"/>, with unique names.</param>
35      /// <param name="roller">Source of die values.</param>
36      /// <param name="timeProvider">Clock for timestamps; defaults to the system clock.</param>
37      /// <exception cref="ArgumentException">The player list is empty, too long, or has duplicate names.</exception>
```

Lines 33–37 Constructor documentation.

```
38      public GameEngine(IEnumerable<Player> players, IDiceRoller roller, TimeProvider? timeProvider = null)
39          : this(players, new DiceCup(roller), timeProvider, startedAt: null)
40      {
41      }
42
```

Lines 38–41 The public constructor for a **new** game: the players in turn order, the dice roller, and an optional `TimeProvider` (the .NET 8+ clock abstraction, injected so tests can control time). It does no work itself: `: this(...)` **chains** to the private constructor below, passing fresh dice and `startedAt: null` (meaning "now").

```
43      /// <summary>
44      /// Shared constructor: validates the players and stores the state. Used directly by
45      /// <see cref="Restore"/> to supply restored dice and the original start time.
46      /// </summary>
```

Lines 43–46 Documentation of the private constructor shared by new and restored games.

```
47      private GameEngine(IEnumerable<Player> players, DiceCup dice, TimeProvider? timeProvider, DateTimeOffset? startedAt)
48      {
49          ArgumentNullException.ThrowIfNull(players);
50
```

Lines 47–49 Private, so only the class itself (the public constructor and `Restore`) can call it. A restored game passes its saved dice and its original start time. Null players are rejected.

```
51          _players = [.. players];
```

Line 51 Copies the players into a new list with a collection expression and the **spread** operator `..`.

```
52          if (_players.Count is 0 or > MaxPlayers)
53              throw new ArgumentException($"A game needs between 1 and {MaxPlayers} players.", nameof(players));
```

Lines 52–53 1–6 players required (pattern `is 0 or > 6`).

```
54          if (_players.DistinctBy(player => player.Name, StringComparer.OrdinalIgnoreCase).Count() != _players.Count)
55              throw new ArgumentException("Player names must be unique.", nameof(players));
56
```

Lines 54–55 Names must be unique, ignoring case: if removing duplicates (LINQ `DistinctBy` with a case-insensitive comparer) shrinks the list, there was a duplicate.

```
57          _dice = dice;
58          _timeProvider = timeProvider ?? TimeProvider.System;
59          StartedAt = startedAt ?? _timeProvider.GetUtcNow();
60      }
61
```

Lines 57–60 Stores the dice (new, or restored from a save) and selects the system clock if none was given. `StartedAt` is the saved start time for a restored game, otherwise the current time.

```
62      /// <summary>Raised whenever any game state changes. Useful for refreshing the GUI and tracking unsaved changes.</summary>
63      public event EventHandler? StateChanged;
64
```

Lines 62–63 `StateChanged` uses the non-generic `EventHandler` because it carries no data. The UI subscribes to it to redraw (and, in step 4, to mark the game unsaved).

```
65      /// <summary>Raised when play passes to the next player. Not raised for the very first turn.</summary>
66      public event EventHandler<TurnStartedEventArgs>? TurnStarted;
67
68      /// <summary>Raised after each roll.</summary>
69      public event EventHandler<DiceRolledEventArgs>? DiceRolled;
70
71      /// <summary>Raised after a score is recorded.</summary>
72      public event EventHandler<CategoryScoredEventArgs>? CategoryScored;
73
74      /// <summary>Raised once when the game ends.</summary>
75      public event EventHandler<GameOverEventArgs>? GameOver;
76
```

Lines 65–75 The four typed events, each carrying one of the records from `GameEvents.cs`. `?` means no handler may be attached.

```
77      /// <summary>The players in turn order.</summary>
78      public IReadOnlyList<Player> Players => _players;
79
```

Lines 77–78 Read-only view of the players.

```
80      /// <summary>Index into <see cref="Players"/> of the player whose turn it is.</summary>
81      public int CurrentPlayerIndex { get; private set; }
82
```

Lines 80–81 Whose turn: an index into `Players`, changeable only inside the engine.

```
83      /// <summary>The player whose turn it is.</summary>
84      public Player CurrentPlayer => _players[CurrentPlayerIndex];
85
```

Lines 83–84 Convenience property.

```
86      /// <summary>When the game was started.</summary>
87      public DateTimeOffset StartedAt { get; }
88
```

Lines 86–87 Start timestamp (for saves).

```
89      /// <summary>
90      /// The current round, 1–13. Derived from the current player's scorecard, since every
91      /// player fills exactly one category per round.
92      /// </summary>
93      public int Round => Math.Min(CurrentPlayer.ScoreCard.Scores.Count + 1, TotalRounds);
94
```

Lines 89–93 `Round` is **calculated, not stored**: every player fills exactly one box per round, so the round equals the number of boxes the current player has filled plus one (capped at 13). There is nothing to keep in sync or save.

```
95      /// <summary>Rolls made so far in the current turn, 0–3.</summary>
96      public int RollsUsed { get; private set; }
97
```

Lines 95–96 `RollsUsed`: 0 at the start of a turn, up to 3.

```
98      /// <summary>Rolls left in the current turn.</summary>
99      public int RollsRemaining => MaxRollsPerTurn - RollsUsed;
100
```

Lines 98–99 Derived: 3 − RollsUsed. The UI shows "Rolls left: n of 3".

```
101     /// <summary>The five current die values. Only meaningful once <see cref="RollsUsed"/> is above 0.</summary>
102     public IReadOnlyList<int> Dice => _dice.Values;
103
```

Lines 101–102 Pass-through to the dice cup.

```
104     /// <summary>Which dice are held and will be kept on the next roll.</summary>
105     public IReadOnlyList<bool> Held => _dice.Held;
106
```

Lines 104–105 Pass-through to the dice cup.

```
107     /// <summary>Whether every player has filled their scorecard.</summary>
108     public bool IsGameOver { get; private set; }
109
```

Lines 107–108 Becomes true once, when the last box is filled.

```
110     /// <summary>Whether the current player may roll now.</summary>
111     public bool CanRoll => !IsGameOver && RollsRemaining > 0;
112
```

Lines 110–111 `CanRoll`: game not over and rolls remain. These `Can…` properties drive which buttons the UI enables.

```
113     /// <summary>
114     /// Whether the current player may hold or release dice now: after a roll, with another roll still to come.
115     /// </summary>
116     public bool CanHold => !IsGameOver && RollsUsed > 0 && RollsRemaining > 0;
117
```

Lines 113–116 `CanHold`: only after a roll *and* while another roll remains (holding after the third roll is pointless).

```
118     /// <summary>Whether the current player may record a score now: after at least one roll.</summary>
119     public bool CanScore => !IsGameOver && RollsUsed > 0;
120
```

Lines 118–119 `CanScore`: at least one roll made this turn.

```
121     /// <summary>Rolls every die that is not held.</summary>
122     /// <exception cref="InvalidOperationException">The game is over or no rolls remain this turn.</exception>
123     public void Roll()
124     {
```

Lines 121–124 `Roll()` documentation and signature.

```
125         if (!CanRoll)
126             throw new InvalidOperationException(IsGameOver ? "The game is over." : "No rolls remain this turn.");
127
```

Lines 125–126 Guard: refuse with a clear message if the game is over or no rolls remain.

```
128         _dice.Roll();
129         RollsUsed++;
130
```

Lines 128–129 Rolls the non-held dice and counts the roll.

```
131         DiceRolled?.Invoke(this, new DiceRolledEventArgs(CurrentPlayer, [.. Dice], RollsRemaining));
132         OnStateChanged();
133     }
134
```

Lines 131–133 Raises `DiceRolled` with a **copy** of the dice (`[.. Dice]`) so listeners cannot be confused by later changes, then `StateChanged`. `?.Invoke` only calls if someone is subscribed.

```
135     /// <summary>Holds or releases the die at <paramref name="index"/> (0–4).</summary>
136     /// <exception cref="InvalidOperationException">Dice cannot be held right now (see <see cref="CanHold"/>).</exception>
137     /// <exception cref="ArgumentOutOfRangeException"><paramref name="index"/> is not 0–4.</exception>
138     public void SetHeld(int index, bool held)
139     {
```

Lines 135–139 `SetHeld` documentation and signature.

```
140         ArgumentOutOfRangeException.ThrowIfNegative(index);
141         ArgumentOutOfRangeException.ThrowIfGreaterThanOrEqual(index, Scorer.DiceCount);
```

Lines 140–141 Validates the index (0–4) first, so a bad index gives a clear argument exception.

```
142         if (!CanHold)
143             throw new InvalidOperationException("Dice can only be held after a roll, while another roll remains.");
```

Lines 142–143 Rule check: holding is only allowed between rolls.

```
144         if (_dice.Held[index] == held)
145             return;
146
```

Lines 144–145 No change requested → return without raising events.

```
147         _dice.SetHeld(index, held);
148         OnStateChanged();
149     }
150
```

Lines 147–149 Applies the change and notifies listeners.

```
151     /// <summary>Flips the held state of the die at <paramref name="index"/> (0–4).</summary>
152     public void ToggleHeld(int index) => SetHeld(index, !Held[index]);
153
```

Lines 151–152 `ToggleHeld`: what a click on a die does.

```
154     /// <summary>
155     /// The score the current dice would earn in each of the current player's open categories.
156     /// Empty before the first roll of a turn.
157     /// </summary>
```

Lines 154–157 Preview documentation.

```
158     public IReadOnlyDictionary<Category, int> Preview() =>
159         CanScore ? CurrentPlayer.ScoreCard.Preview(Dice) : new Dictionary<Category, int>();
160
```

Lines 158–159 The current player's preview, or an empty dictionary before the first roll.

```
161     /// <summary>
162     /// Records the current dice in <paramref name="category"/> for the current player, ends the turn,
163     /// and passes play to the next player (or ends the game).
164     /// </summary>
165     /// <returns>The points recorded.</returns>
166     /// <exception cref="InvalidOperationException">
167     /// The dice have not been rolled this turn, the game is over, or the category is already filled.
168     /// </exception>
```

Lines 161–168 `Score` documentation.

```
169     public int Score(Category category)
170     {
```

Lines 169–170 Records a score and ends the turn.

```
171         if (!CanScore)
172             throw new InvalidOperationException(IsGameOver ? "The game is over." : "Roll the dice before scoring.");
173
```

Lines 171–172 Guard: must have rolled at least once.

```
174         var player = CurrentPlayer;
175         int points = player.ScoreCard.Record(category, Dice);
```

Lines 174–175 Remember who is scoring (the current player changes below) and record the score; `ScoreCard.Record` throws if the category is already filled, leaving the turn unchanged.

```
176         CategoryScored?.Invoke(this, new CategoryScoredEventArgs(player, category, points));
177
```

Line 176 Tell listeners what was scored.

```
178         if (_players.TrueForAll(p => p.ScoreCard.IsComplete))
179         {
180             IsGameOver = true;
181             GameOver?.Invoke(this, new GameOverEventArgs(GetStandings(), _timeProvider.GetUtcNow()));
182         }
```

Lines 178–182 If every scorecard is complete, the game is over: set the flag and raise `GameOver` with the standings and the time.

```
183         else
184         {
185             CurrentPlayerIndex = (CurrentPlayerIndex + 1) % _players.Count;
186             RollsUsed = 0;
187             _dice.Reset();
188             TurnStarted?.Invoke(this, new TurnStartedEventArgs(CurrentPlayer, Round));
189         }
190
```

Lines 183–189 Otherwise pass play to the next player: `% _players.Count` wraps from the last player back to the first. Rolls reset to 0, dice are cleared, and `TurnStarted` is raised.

```
191         OnStateChanged();
192         return points;
193     }
194
```

Lines 191–193 Finally `StateChanged` (always last, so the UI redraws a consistent state) and return the points.

```
195     /// <summary>
196     /// Plays one visible step of a computer-controlled player's turn: either a roll or the final score.
197     /// The GUI calls this repeatedly (with a delay) until it returns <see cref="AiStepResult.Scored"/>.
198     /// </summary>
199     /// <returns>Which action was taken.</returns>
200     /// <exception cref="InvalidOperationException">The game is over, the current player is human, or the strategy returned an
invalid choice.</exception>
```

Lines 195–200 `PlayAiStep` documentation: performs **one visible action** per call so the UI can show each AI roll.

```
201     public AiStepResult PlayAiStep()
202     {
```

Lines 201–202 Signature.

```
203         if (IsGameOver)
204             throw new InvalidOperationException("The game is over.");
```

Lines 203–204 Cannot act after the game ends.

```
205         var strategy = CurrentPlayer.Strategy
206             ?? throw new InvalidOperationException($"{CurrentPlayer.Name} is not an AI player.");
207
```

Lines 205–206 Gets the strategy or throws if the current player is human (throw expression).

```
208         if (RollsUsed == 0)
209         {
210             Roll();
211             return AiStepResult.Rolled;
212         }
213
```

Lines 208–212 Start of turn: the first action is always a roll.

```
214         if (RollsRemaining > 0)
215         {
```

Lines 214–215 Rolls remain: ask the strategy which dice to hold.

```
216             var holds = strategy.ChooseHolds([.. Dice], RollsRemaining, CurrentPlayer.ScoreCard);
217             if (holds is not null)
218             {
```

Lines 216–218 Passes copies of the dice; a non-null answer means "roll again".

```
219                 if (holds.Count != Scorer.DiceCount)
220                     throw new InvalidOperationException($"Strategy {strategy.Name} returned {holds.Count} hold flags.");
```

Lines 219–220 Protects against a faulty strategy returning the wrong number of flags.

```
                for (int i = 0; i < Scorer.DiceCount; i++)
                    _dice.SetHeld(i, holds[i]);
                Roll();
                return AiStepResult.Rolled;
            }
        }

```

Lines 221–226 Applies the holds directly to the cup, rolls, and reports `Rolled`. If the strategy returned null, or no rolls remain, execution falls through to scoring.

```
        var category = strategy.ChooseCategory([.. Dice], CurrentPlayer.ScoreCard);
```

Line 228 Ask which category to score.

```
        if (CurrentPlayer.ScoreCard.IsFilled(category))
            throw new InvalidOperationException($"Strategy {strategy.Name} chose {category.DisplayName()}, which is already
filled.");
```

Lines 229–230 Reject an illegal choice with a message naming the strategy.

```
        Score(category);
        return AiStepResult.Scored;
    }

```

Lines 231–233 Score (which also advances the turn) and report `Scored`.

```
    /// <summary>
    /// The players ordered by total score, highest first. Ties share a rank and keep turn order.
    /// </summary>
```

Lines 235–237 `GetStandings` documentation.

```
    public IReadOnlyList<PlayerStanding> GetStandings()
    {
```

Lines 238–239 Builds the results table.

```
        var ordered = _players.OrderByDescending(player => player.Total).ToList(); // stable sort keeps turn order on ties
```

Line 240 Sort players by total, highest first. LINQ's `OrderByDescending` is a **stable** sort, so tied players keep turn order.

```
        return ordered
            .Select(player => new PlayerStanding(
```

Lines 241–242 Project each player into a `PlayerStanding` record.

```
                Rank: ordered.FindIndex(other => other.Total == player.Total) + 1,
                player.Name,
                player.Total,
                player.IsAi))
            .ToList();
    }

```

Lines 243–248 Rank = position of the *first* player with the same total + 1, which gives standard competition ranking (scores 50, 0, 0 → ranks 1, 2, 2). Named argument `Rank:` improves readability.

```
    private void OnStateChanged() => StateChanged?.Invoke(this, EventArgs.Empty);
}
```

Lines 250–251 Helper raising `StateChanged`; called at the end of every mutating action.

10.10 Saving and restoring games (engine side)

`src/Yahtzee.Core/Game/GameEngine.Persistence.cs · 127 lines`

The second half of the `partial` class `GameEngine`. It turns a running game into a `GameSnapshot` and back. Restoring **validates** that the snapshot describes a game that could really exist, so a damaged or edited file cannot produce an impossible state.

```
1  using Yahtzee.Core.AI;
2  using Yahtzee.Core.Model;
3  using Yahtzee.Core.Persistence;
4  using Yahtzee.Core.Rules;
5
```

Lines 1–4 Imports for strategies, the model, the snapshot types and the scoring constants.

```
6  namespace Yahtzee.Core.Game;
7
```

Line 6 Same namespace as the other half of the class.

```
8  // Saving and restoring games. This half of the partial class can reach the engine's private state,
9  // which a restore needs to set; everything else about the engine is in GameEngine.cs.
```

Lines 8–9 Explains why this code lives in a second file of the same class: it needs the engine's private state.

```
10 public sealed partial class GameEngine
11 {
```

Lines 10–11 The second part of the partial class (the modifiers must match the first part).

```
12     /// <summary>Whether the game can be saved: any time before it ends.</summary>
13     public bool CanSave => !IsGameOver;
14
```

Lines 12–13 `CanSave`: any time until the game ends, including mid-turn ("after any roll").

```
15     /// <summary>Captures the complete game state, including mid-turn dice and holds.</summary>
16     /// <exception cref="InvalidOperationException">The game is over.</exception>
17     public GameSnapshot CreateSnapshot()
18     {
```

Lines 15–18 `CreateSnapshot` documentation.

```
19         if (!CanSave)
20             throw new InvalidOperationException("A finished game cannot be saved.");
21
```

Lines 19–20 A finished game is not saved (it is recorded in the history instead).

```
22         var players = _players
23             .Select(player => new PlayerSnapshot(
24                 player.Name,
```

Lines 22–24 For each player build a `PlayerSnapshot` with LINQ `Select`: the name...

```
25                 player.Strategy?.Name,
```

Line 25 ...the strategy name, or null for a human (the `?.` null-conditional operator)...

```
26                 CategoryExtensions.All.ToDictionary(category => category, player.ScoreCard.GetScore)))
27             .ToList();
28
```

Lines 26–27 ...and all 13 categories mapped to their score or null. `player.ScoreCard.GetScore` is passed as a method group.

```
29         bool rolled = RollsUsed > 0;
```

Line 29 Before the first roll the dice values are meaningless, so they are written as zeros.

```
30          return new GameSnapshot(
31              SaveGameSerializer.CurrentFormatVersion,
32              StartedAt,
33              players,
34              CurrentPlayerIndex,
35              RollsUsed,
```

Lines 30–35 Builds the record from the engine's state: format version, start time, players, whose turn, rolls used…

```
36              rolled ? [.. Dice] : [0, 0, 0, 0, 0],
37              rolled ? [.. Held] : [false, false, false, false, false]);
38      }
39
```

Lines 36–38 …and copies of the dice and held flags (collection expressions).

```
40      /// <summary>
41      /// Recreates a game from a snapshot, after checking that the snapshot describes a game that could
42      /// really have been reached by playing.
43      /// </summary>
44      /// <param name="snapshot">The saved state, usually from <see cref="SaveGameSerializer.LoadAsync"/>.</param>
45      /// <param name="roller">Source of die values for the rest of the game.</param>
46      /// <param name="createStrategy">Creates the AI strategy with the given saved name.</param>
47      /// <param name="timeProvider">Clock for timestamps; defaults to the system clock.</param>
48      /// <exception cref="InvalidSaveFileException">The snapshot is inconsistent or refers to an unknown strategy.</exception>
```

Lines 40–48 `Restore` documentation.

```
49      public static GameEngine Restore(GameSnapshot snapshot, IDiceRoller roller, Func<string, IPlayerStrategy> createStrategy,
50          TimeProvider? timeProvider = null)
51      {
```

Lines 49–51 A **static factory method**: it creates the engine, which a constructor could not do as clearly with this much validation. `createStrategy` is a delegate that turns a saved name into a strategy object.

```
52          ArgumentNullException.ThrowIfNull(snapshot);
53          ArgumentNullException.ThrowIfNull(roller);
54          ArgumentNullException.ThrowIfNull(createStrategy);
55
```

Lines 52–54 Programming errors (null arguments) throw normally.

```
56          if (snapshot.Players.Count is 0 or > MaxPlayers)
57              throw new InvalidSaveFileException($"A game must have between 1 and {MaxPlayers} players.");
```

Lines 56–57 Checks on the file's contents throw `InvalidSaveFileException` with a readable message: player count…

```
58          if (snapshot.CurrentPlayerIndex < 0 || snapshot.CurrentPlayerIndex >= snapshot.Players.Count)
59              throw new InvalidSaveFileException("The current player does not exist.");
```

Lines 58–59 …a current player that exists…

```
60          if (snapshot.RollsUsed is < 0 or > MaxRollsPerTurn)
61              throw new InvalidSaveFileException($"Rolls used must be between 0 and {MaxRollsPerTurn}.");
```

Lines 60–61 …a roll count of 0–3…

```
62          if (snapshot.Dice.Count != Scorer.DiceCount || snapshot.Held.Count != Scorer.DiceCount)
63              throw new InvalidSaveFileException($"Exactly {Scorer.DiceCount} dice and held flags are required.");
64
```

Lines 62–63 …and exactly five dice and five held flags.

```
65          GameEngine game;
66          try
67          {
```

Lines 65–67 The model classes report invalid data as `ArgumentException`s, so building the game is wrapped in a `try`.

```
68              var players = snapshot.Players.Select(saved => RestorePlayer(saved, createStrategy));
```

Line 68 Restore each player (lazily, as the constructor copies the list).

```
            var dice = snapshot.RollsUsed == 0
                ? new DiceCup(roller)
                : new DiceCup(roller, snapshot.Dice, snapshot.Held);
```

Lines 69–71 Fresh dice before the first roll, otherwise the restoring `DiceCup` constructor, which validates the values.

```
            game = new GameEngine(players, dice, timeProvider, snapshot.StartedAt);
        }
```

Lines 72–73 The private constructor checks the player count and duplicate names and keeps the original start time.

```
        catch (ArgumentException ex)
        {
            // Invalid names, scores or dice are reported by the model classes as ArgumentExceptions.
            throw new InvalidSaveFileException($"The save file contains invalid data: {ex.Message}", ex);
        }

```

Lines 74–78 Translate model errors into the save-file exception, keeping the original as the inner exception.

```
        game.CurrentPlayerIndex = snapshot.CurrentPlayerIndex;
        game.RollsUsed = snapshot.RollsUsed;
```

Lines 80–81 Private setters are reachable here because this is the same class.

```
        game.CheckTurnOrder();
        return game;
    }

```

Lines 82–84 Final consistency check, then the restored game is returned.

```
    private static Player RestorePlayer(PlayerSnapshot saved, Func<string, IPlayerStrategy> createStrategy)
    {
```

Lines 86–87 Rebuilds one player.

```
        var scores = saved.Scores
            .Where(entry => entry.Value is not null)
            .ToDictionary(entry => entry.Key, entry => entry.Value!.Value);

```

Lines 88–90 Keep only filled boxes (non-null values) and unwrap them from `int?` to `int`.

```
        IPlayerStrategy? strategy = null;
        if (saved.Strategy is not null)
        {
            try
            {
```

Lines 92–96 Humans have no strategy.

```
                strategy = createStrategy(saved.Strategy);
            }
```

Lines 97–98 Ask the catalog for the named strategy…

```
            catch (ArgumentException ex)
            {
                throw new InvalidSaveFileException($"Unknown AI strategy \"{saved.Strategy}\" for {saved.Name}.", ex);
            }
        }

```

Lines 99–103 …an unknown name (e.g. from an edited file) becomes a clear message.

```
        return new Player(saved.Name, strategy, new ScoreCard(scores));
    }

```

Lines 105–106 The `Player` and `ScoreCard` constructors validate the name and scores.

```
108     /// <summary>
109     /// Every player fills one box per turn, in turn order. So players before the current one must have
110     /// filled exactly one more box than the current player, and players after it the same number.
111     /// </summary>
```

Lines 108–111 The rule behind `CheckTurnOrder`.

```
112     private void CheckTurnOrder()
113     {
```

Lines 112–113 Validates that the scorecards agree with whose turn it is.

```
114         int filled = CurrentPlayer.ScoreCard.Scores.Count;
115         if (filled >= TotalRounds)
116             throw new InvalidSaveFileException("The game in this file has already finished.");
117
```

Lines 114–116 The current player has filled `filled` boxes; if that is 13 the game was already over.

```
118         for (int i = 0; i < _players.Count; i++)
119         {
```

Lines 118–119 Check every player.

```
120             int expected = i < CurrentPlayerIndex ? filled + 1 : filled;
```

Line 120 Players before the current one have already played this round (one more box).

```
121             if (_players[i].ScoreCard.Scores.Count != expected)
122                 throw new InvalidSaveFileException(
123                     $"{_players[i].Name} has {_players[i].ScoreCard.Scores.Count} scores filled, but it is " +
124                     $"{CurrentPlayer.Name}'s turn in round {filled + 1}, so {expected} were expected.");
125         }
126     }
127 }
```

Lines 121–127 Any mismatch means the file was edited or damaged; the message names the player and the numbers.

10.11 Save-file data types

`src/Yahtzee.Core/Persistence/GameSnapshot.cs · 32 lines`

Two records that define the save-file format. They are plain data (no behaviour) so that `System.Text.Json` can convert them directly.

```
1  using Yahtzee.Core.Model;
2
```

Line 1 For `Category`.

```
3  namespace Yahtzee.Core.Persistence;
4
```

Line 3 Persistence namespace.

```
5  /// <summary>
6  /// Everything needed to resume a game exactly where it was saved. This is the root object of the
7  /// <c>.yahtzee</c> save file; <see cref="SaveGameSerializer"/> converts it to and from JSON.
8  /// </summary>
```

Lines 5–8 Documentation of the root object of a save file.

```
9  /// <remarks>
10 /// The round number is not stored: it is derived from the scorecards when the game is restored.
11 /// </remarks>
```

Lines 9–11 The round is deliberately not stored (it is derived).

```
12 /// <param name="FormatVersion">Save-file format version; see <see cref="SaveGameSerializer.CurrentFormatVersion"/>.</param>
13 /// <param name="StartedAt">When the game was originally started.</param>
14 /// <param name="Players">The players in turn order.</param>
15 /// <param name="CurrentPlayerIndex">Index into <paramref name="Players"/> of the player whose turn it is.</param>
16 /// <param name="RollsUsed">Rolls already made in the current turn, 0–3.</param>
17 /// <param name="Dice">The five die values; all 0 when <paramref name="RollsUsed"/> is 0.</param>
18 /// <param name="Held">Which of the five dice are held.</param>
```

Lines 12–18 Documentation of each property.

```
19 public sealed record GameSnapshot(
```

Line 19 A positional record: the parameter list defines both the constructor and the properties. JSON uses the same names (in camelCase).

```
20     int FormatVersion,
21     DateTimeOffset StartedAt,
22     IReadOnlyList<PlayerSnapshot> Players,
23     int CurrentPlayerIndex,
24     int RollsUsed,
25     IReadOnlyList<int> Dice,
26     IReadOnlyList<bool> Held);
27
```

Lines 20–26 The fields: version, start time, players, current player, rolls used, dice, held flags. Read-only interfaces (`IReadOnlyList`) make snapshots effectively immutable.

```
28 /// <summary>One player's saved state.</summary>
29 /// <param name="Name">The player's name.</param>
30 /// <param name="Strategy">The AI strategy name, or <see langword="null"/> for a human player.</param>
31 /// <param name="Scores">Recorded scores by category; a <see langword="null"/> or missing entry is an empty box.</param>
```

Lines 28–31 Documentation of `PlayerSnapshot`.

```
32 public sealed record PlayerSnapshot(string Name, string? Strategy, IReadOnlyDictionary<Category, int?> Scores);
```

Line 32 Name, strategy name (null = human) and a dictionary of category → nullable score. In JSON the dictionary becomes an object such as `{ "Aces": 3, "Twos": null, … }`.

10.12 Save-file error type

`src/Yahtzee.Core/Persistence/InvalidSaveFileException.cs · 18 lines`

A dedicated exception type lets the UI catch "this file is bad" separately from programming errors.

```
1  namespace Yahtzee.Core.Persistence;
2
```

Line 1 Persistence namespace.

```
3  /// <summary>
4  /// Thrown when a save file cannot be read or describes a game that cannot exist
5  /// (for example, impossible dice or scorecards that disagree with the turn order).
6  /// </summary>
```

Lines 3–6 Documentation.

```
7  public sealed class InvalidSaveFileException : Exception
8  {
```

Lines 7–8 Derives from `Exception`.

```
9      /// <summary>Creates the exception with a message explaining what is wrong with the file.</summary>
10     public InvalidSaveFileException(string message) : base(message)
11     {
12     }
13
```

Lines 9–12 Constructor taking a message…

```
14     /// <summary>Creates the exception with a message and the error that caused it.</summary>
15     public InvalidSaveFileException(string message, Exception innerException) : base(message, innerException)
16     {
17     }
18 }
```

Lines 14–18 …and one that also keeps the underlying error (for example the JSON parser's), following the standard .NET pattern.

10.13 Save-file reading and writing

`src/Yahtzee.Core/Persistence/SaveGameSerializer.cs · 85 lines`

Converts snapshots to and from JSON text and reads/writes `.yahtzee` files.

```
1  using System.Text.Json;
2  using System.Text.Json.Serialization;
3
```

Lines 1–2 The built-in .NET JSON library (no external package needed).

```
4  namespace Yahtzee.Core.Persistence;
5
```

Line 4 Persistence namespace.

```
6  /// <summary>
7  /// Reads and writes <see cref="GameSnapshot"/>s as indented, human-readable JSON (<c>.yahtzee</c> files).
8  /// </summary>
```

Lines 6–8 Documentation.

```
9  /// <remarks>
10 /// This class only checks that the file is well-formed JSON of the right shape and version.
11 /// Whether the described game is actually possible is checked by <c>GameEngine.Restore</c>.
12 /// </remarks>
```

Lines 9–12 Division of work: this class checks the file's shape; `GameEngine.Restore` checks the game's logic.

```
13 public static class SaveGameSerializer
14 {
```

Lines 13–14 A static class: no state, just functions.

```
15     /// <summary>File extension used for save files, including the dot.</summary>
16     public const string FileExtension = ".yahtzee";
17
```

Lines 15–16 `.yahtzee`, used by the file dialogs.

```
18     /// <summary>The format version written by this program. Increase it whenever the format changes.</summary>
19     public const int CurrentFormatVersion = 1;
20
```

Lines 18–19 Version 1. If the format ever changes, the number goes up and old files can be recognised.

```
21     private static readonly JsonSerializerOptions Options = new()
22     {
```

Lines 21–22 One shared, reusable options object (creating one per call is slow).

```
23         WriteIndented = true,
24         PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
```

Lines 23–24 Human-readable, indented JSON with camelCase property names.

```
25         // A missing property or a null where the type does not allow one is an error, not a silent default.
26         RespectRequiredConstructorParameters = true,
27         RespectNullableAnnotations = true,
```

Lines 25–27 Strict reading (.NET 9+): a missing constructor parameter or a `null` for a non-nullable property is an error rather than being silently replaced by a default.

```
28         Converters = { new JsonStringEnumConverter() },
29     };
30
```

Lines 28–29 Writes categories by name ("FullHouse") instead of by number, so the file is readable.

```
31      /// <summary>Converts a snapshot to JSON text.</summary>
32      public static string Serialize(GameSnapshot snapshot)
33      {
34          ArgumentNullException.ThrowIfNull(snapshot);
35          return JsonSerializer.Serialize(snapshot, Options);
36      }
37
```

Lines 31–36 `Serialize`: one call to the library.

```
38      /// <summary>Parses JSON text produced by <see cref="Serialize"/>.</summary>
39      /// <exception cref="InvalidSaveFileException">The text is not a valid save of a supported version.</exception>
40      public static GameSnapshot Deserialize(string json)
41      {
42          ArgumentNullException.ThrowIfNull(json);
43
```

Lines 38–42 `Deserialize` documentation and signature.

```
44          GameSnapshot? snapshot;
```

Line 44 Nullable because the JSON text `null` deserializes to null.

```
45          try
46          {
47              snapshot = JsonSerializer.Deserialize<GameSnapshot>(json, Options);
48          }
```

Lines 45–48 Parse…

```
49          catch (JsonException ex)
50          {
51              throw new InvalidSaveFileException($"The file is not a valid Yahtzee save: {ex.Message}", ex);
52          }
53
```

Lines 49–52 …and turn any JSON error (bad syntax, wrong types, missing fields) into an `InvalidSaveFileException`.

```
54          if (snapshot is null)
55              throw new InvalidSaveFileException("The file is empty.");
```

Lines 54–55 The literal `null` is not a game.

```
56          if (snapshot.FormatVersion != CurrentFormatVersion)
57              throw new InvalidSaveFileException(
58                  $"The file uses save format version {snapshot.FormatVersion}; this program reads version
{CurrentFormatVersion}.");
59          return snapshot;
60      }
61
```

Lines 56–60 Refuse files from a different format version with a message saying which.

```
62      /// <summary>Writes a snapshot to <paramref name="path"/>, replacing any existing file.</summary>
63      /// <remarks>
64      /// The JSON is written to a temporary file first and then moved over the target, so a crash
65      /// part-way through never leaves a half-written save behind.
66      /// </remarks>
```

Lines 62–66 `SaveAsync` documentation, including the safe-write technique.

```
67      public static async Task SaveAsync(GameSnapshot snapshot, string path, CancellationToken cancellationToken = default)
68      {
69          ArgumentException.ThrowIfNullOrWhiteSpace(path);
```

Lines 67–69 Asynchronous: the UI thread is not blocked while the disk is written.

```
70          string json = Serialize(snapshot);
71          string tempPath = path + ".tmp";
72          await File.WriteAllTextAsync(tempPath, json, cancellationToken);
```

Lines 70–72 Write to `path.tmp` first…

```
73          File.Move(tempPath, path, overwrite: true);
74      }
75
```

Lines 73–74 ...then replace the real file in one step. If the program crashed mid-write, the old save would still be intact.

```
76      /// <summary>Reads a snapshot from <paramref name="path"/>.</summary>
77      /// <exception cref="InvalidSaveFileException">The file is not a valid save of a supported version.</exception>
78      /// <exception cref="IOException">The file could not be read.</exception>
```

Lines 76–78 `LoadAsync` documentation.

```
79      public static async Task<GameSnapshot> LoadAsync(string path, CancellationToken cancellationToken = default)
80      {
81          ArgumentException.ThrowIfNullOrWhiteSpace(path);
82          string json = await File.ReadAllTextAsync(path, cancellationToken);
83          return Deserialize(json);
84      }
85  }
```

Lines 79–85 Read the whole file asynchronously and parse it.

10.14 Games-history interface

`src/Yahtzee.Core/Data/IGameHistoryRepository.cs · 39 lines`

The **repository pattern**: an interface describing what the program needs from storage, independent of how it is stored.

```
1  using Yahtzee.Core.Game;
2
```

Line 1 For `PlayerStanding`.

```
3  namespace Yahtzee.Core.Data;
4
```

Line 3 Data namespace.

```
5  /// <summary>
6  /// Storage for completed games and high-score queries. The UI depends on this interface, not on
7  /// SQLite, so the storage engine could be replaced without touching the forms.
8  /// </summary>
```

Lines 5–8 Why an interface: the UI depends on this, not on SQLite.

```
9  public interface IGameHistoryRepository
10 {
```

Lines 9–10 The interface.

```
11     /// <summary>Records a finished game and every player's final score.</summary>
12     Task RecordGameAsync(CompletedGame game, CancellationToken cancellationToken = default);
13
```

Lines 11–12 Store a completed game. All methods return `Task`s (asynchronous) and accept an optional `CancellationToken`.

```
14     /// <summary>The highest individual scores across all games, best first.</summary>
15     /// <param name="count">How many scores to return.</param>
16     /// <param name="cancellationToken">Cancels the query.</param>
17     Task<IReadOnlyList<HighScore>> GetTopScoresAsync(int count, CancellationToken cancellationToken = default);
18
```

Lines 14–17 Top N overall (requirement: five highest overall scores).

```
19     /// <summary>The highest scores of one player (name compared ignoring case), best first.</summary>
20     /// <param name="playerName">The player's name.</param>
21     /// <param name="count">How many scores to return.</param>
22     /// <param name="cancellationToken">Cancels the query.</param>
23     Task<IReadOnlyList<HighScore>> GetTopScoresForPlayerAsync(string playerName, int count, CancellationToken cancellationToken =
default);
24
```

Lines 19–23 Top N for one player (requirement: five highest scores of a particular player).

```
25     /// <summary>Every player name that appears in the history, alphabetically.</summary>
26     Task<IReadOnlyList<string>> GetPlayerNamesAsync(CancellationToken cancellationToken = default);
27 }
28
```

Lines 25–27 Names for the high-score window's drop-down.

```
29 /// <summary>A finished game, as recorded in the history.</summary>
30 /// <param name="PlayedAt">When the game finished.</param>
31 /// <param name="Results">Each player's final standing.</param>
32 public sealed record CompletedGame(DateTimeOffset PlayedAt, IReadOnlyList<PlayerStanding> Results);
33
```

Lines 29–32 `CompletedGame` record: when it was played and the standings.

```
34 /// <summary>One row of a high-score table.</summary>
35 /// <param name="PlayerName">Who scored it.</param>
36 /// <param name="Score">The final score.</param>
37 /// <param name="IsAi">Whether the player was computer-controlled.</param>
38 /// <param name="PlayedAt">When the game finished.</param>
39 public sealed record HighScore(string PlayerName, int Score, bool IsAi, DateTimeOffset PlayedAt);
```

Lines 34–39 `HighScore` record: player, score, AI flag and date (the "could" item: include the date).

10.15 SQLite games history

`src/Yahtzee.Core/Data/SqliteGameHistoryRepository.cs · 169 lines`

Implements the repository with **SQLite**, a real database engine stored in a single file and needing no server. Requirement: "the games database should be kept using a database engine".

```
1  using System.Globalization;
2  using Microsoft.Data.Sqlite;
3
```

Lines 1–2 Culture-independent date formatting and the SQLite ADO.NET classes.

```
4  namespace Yahtzee.Core.Data;
5
```

Line 4 Data namespace.

```
6  /// <summary>
7  /// Keeps the games history in a SQLite database file. The tables are created automatically the first
8  /// time the database is used.
9  /// </summary>
```

Lines 6–9 Class documentation.

```
10 /// <remarks>
11 /// Schema:
12 /// <code>
13 /// Games(Id INTEGER PRIMARY KEY, PlayedAt TEXT NOT NULL)            -- ISO-8601 UTC timestamp
14 /// GameResults(Id INTEGER PRIMARY KEY, GameId INTEGER NOT NULL REFERENCES Games(Id),
15 ///             PlayerName TEXT NOT NULL COLLATE NOCASE, Score INTEGER NOT NULL, IsAi INTEGER NOT NULL)
16 /// </code>
17 /// All queries use parameters, never string concatenation, so player names cannot inject SQL.
18 /// </remarks>
```

Lines 10–18 The schema and the security rule (parameters only), documented for the developer guide.

```
19 public sealed class SqliteGameHistoryRepository : IGameHistoryRepository
20 {
```

Lines 19–20 Implements the interface.

```
21     private const string CreateSchemaSql = """
```

Line 21 The schema as a C# 11 **raw string literal** (`"""`), so the SQL can be written naturally.

```
22         CREATE TABLE IF NOT EXISTS Games (
23             Id       INTEGER PRIMARY KEY AUTOINCREMENT,
24             PlayedAt TEXT    NOT NULL
25         );
```

Lines 22–25 `Games`: one row per finished game with an auto-numbered id and the date as ISO-8601 text (SQLite has no date type; ISO text sorts correctly).

```
26         CREATE TABLE IF NOT EXISTS GameResults (
27             Id         INTEGER PRIMARY KEY AUTOINCREMENT,
28             GameId     INTEGER NOT NULL REFERENCES Games(Id) ON DELETE CASCADE,
29             PlayerName TEXT    NOT NULL COLLATE NOCASE,
30             Score      INTEGER NOT NULL,
31             IsAi       INTEGER NOT NULL
32         );
```

Lines 26–32 `GameResults`: one row per player per game. `REFERENCES Games(Id)` is a foreign key (enforced because the connection enables foreign keys); `COLLATE NOCASE` makes name comparisons case-insensitive.

```
33         CREATE INDEX IF NOT EXISTS IX_GameResults_Score ON GameResults (Score DESC);
34         CREATE INDEX IF NOT EXISTS IX_GameResults_Player ON GameResults (PlayerName, Score DESC);
35         """;
36
```

Lines 33–35 Indexes so the two high-score queries stay fast as the history grows. `IF NOT EXISTS` makes the whole script safe to run every time.

```
37      /// <summary>Columns shared by both high-score queries; ties go to whoever scored first.</summary>
38      private const string HighScoreSelect = """
39          SELECT r.PlayerName, r.Score, r.IsAi, g.PlayedAt
40          FROM GameResults r JOIN Games g ON g.Id = r.GameId
41          """;
42
```

Lines 37–41 The SELECT and JOIN common to both high-score queries, written once.

```
43      private readonly string _connectionString;
44      private bool _schemaCreated;
45
```

Lines 43–44 The connection string, and a flag so the schema is created only once per run.

```
46      /// <summary>Uses (and if necessary creates) the database file at <paramref name="databasePath"/>.</summary>
47      public SqliteGameHistoryRepository(string databasePath)
48      {
49          ArgumentException.ThrowIfNullOrWhiteSpace(databasePath);
```

Lines 46–49 Constructor: makes sure the folder exists (e.g. `%LOCALAPPDATA%\Yahtzee`)...

```
50          string? folder = Path.GetDirectoryName(Path.GetFullPath(databasePath));
51          if (folder is not null)
52              Directory.CreateDirectory(folder);
53
```

Lines 50–52 ...by taking the directory part of the full path...

```
54          _connectionString = new SqliteConnectionStringBuilder
55          {
56              DataSource = databasePath,
57              ForeignKeys = true,
58          }.ToString();
59      }
60
```

Lines 54–59 ...and builds the connection string with a builder (no hand-written string) with foreign keys on.

```
61      /// <summary>The default database location: <c>%LOCALAPPDATA%\Yahtzee\yahtzee.db</c>.</summary>
62      public static string DefaultDatabasePath { get; } = Path.Combine(
63          Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData), "Yahtzee", "yahtzee.db");
64
```

Lines 61–63 `%LOCALAPPDATA%\Yahtzee\yahtzee.db`: a per-user folder that is always writable.

```
65      /// <inheritdoc/>
66      public async Task RecordGameAsync(CompletedGame game, CancellationToken cancellationToken = default)
67      {
```

Lines 65–67 `RecordGameAsync`.

```
68          ArgumentNullException.ThrowIfNull(game);
69          await using var connection = await OpenAsync(cancellationToken);
```

Lines 68–69 `await using` disposes (closes) the connection asynchronously at the end of the method.

```
70          // One transaction: either the game and all of its results are stored, or nothing is.
71          await using var transaction = (SqliteTransaction)await connection.BeginTransactionAsync(cancellationToken);
72
```

Lines 70–71 A **transaction**: if anything fails part-way, nothing is stored, so a game never appears without its results.

```
73          using var insertGame = connection.CreateCommand();
74          insertGame.Transaction = transaction;
75          insertGame.CommandText = "INSERT INTO Games (PlayedAt) VALUES ($playedAt); SELECT last_insert_rowid();";
76          insertGame.Parameters.AddWithValue("$playedAt", game.PlayedAt.UtcDateTime.ToString("O", CultureInfo.InvariantCulture));
```

Lines 73–76 Insert the game row. `$playedAt` is a **parameter**: the value is sent separately from the SQL text, so it can never be interpreted as SQL.

```
77          long gameId = (long)(await insertGame.ExecuteScalarAsync(cancellationToken))!;
78
```

Line 77 `last_insert_rowid()` (run in the same command) returns the new game's id.

```
79          using var insertResult = connection.CreateCommand();
80          insertResult.Transaction = transaction;
81          insertResult.CommandText = """
82              INSERT INTO GameResults (GameId, PlayerName, Score, IsAi)
83              VALUES ($gameId, $name, $score, $isAi)
84              """;
```

Lines 79–84 One reusable insert command for the results…

```
85          var nameParameter = insertResult.Parameters.Add("$name", SqliteType.Text);
86          var scoreParameter = insertResult.Parameters.Add("$score", SqliteType.Integer);
87          var isAiParameter = insertResult.Parameters.Add("$isAi", SqliteType.Integer);
88          insertResult.Parameters.AddWithValue("$gameId", gameId);
89
```

Lines 85–88 …with its parameters created once; only their values change for each player.

```
90          foreach (var result in game.Results)
91          {
92              nameParameter.Value = result.Name;
93              scoreParameter.Value = result.Score;
94              isAiParameter.Value = result.IsAi ? 1 : 0;
95              await insertResult.ExecuteNonQueryAsync(cancellationToken);
96          }
97
```

Lines 90–96 Insert one row per player (AI flag stored as 1/0, since SQLite has no boolean type).

```
98          await transaction.CommitAsync(cancellationToken);
99      }
100
```

Lines 98–99 Make everything permanent at once.

```
101     /// <inheritdoc/>
102     public async Task<IReadOnlyList<HighScore>> GetTopScoresAsync(int count, CancellationToken cancellationToken = default)
103     {
```

Lines 101–103 `GetTopScoresAsync`.

```
104         ArgumentOutOfRangeException.ThrowIfNegativeOrZero(count);
105         await using var connection = await OpenAsync(cancellationToken);
106         using var command = connection.CreateCommand();
```

Lines 104–106 Validate, open, and build the command.

```
107         command.CommandText = $"{HighScoreSelect} ORDER BY r.Score DESC, g.PlayedAt ASC LIMIT $count";
108         command.Parameters.AddWithValue("$count", count);
109         return await ReadHighScoresAsync(command, cancellationToken);
110     }
111
```

Lines 107–110 Highest scores first; ties go to the earliest game; `LIMIT` keeps the top N. Only the constant part is interpolated; the count is a parameter.

```
112     /// <inheritdoc/>
113     public async Task<IReadOnlyList<HighScore>> GetTopScoresForPlayerAsync(string playerName, int count,
114         CancellationToken cancellationToken = default)
115     {
```

Lines 112–115 `GetTopScoresForPlayerAsync`.

```
116         ArgumentNullException.ThrowIfNull(playerName);
117         ArgumentOutOfRangeException.ThrowIfNegativeOrZero(count);
118         await using var connection = await OpenAsync(cancellationToken);
119         using var command = connection.CreateCommand();
```

Lines 116–119 Validate and open.

```
        // PlayerName is declared COLLATE NOCASE, so "alice" also finds "Alice".
        command.CommandText = $"{HighScoreSelect} WHERE r.PlayerName = $name ORDER BY r.Score DESC, g.PlayedAt ASC LIMIT $count";
        command.Parameters.AddWithValue("$name", playerName.Trim());
        command.Parameters.AddWithValue("$count", count);
        return await ReadHighScoresAsync(command, cancellationToken);
    }

```

Lines 120–125 Same query with a `WHERE` on the name parameter; thanks to the column's collation "alice" finds "Alice". The name is trimmed first.

```
    /// <inheritdoc/>
    public async Task<IReadOnlyList<string>> GetPlayerNamesAsync(CancellationToken cancellationToken = default)
    {
        await using var connection = await OpenAsync(cancellationToken);
        using var command = connection.CreateCommand();
```

Lines 127–131 `GetPlayerNamesAsync`.

```
        command.CommandText = "SELECT DISTINCT PlayerName FROM GameResults ORDER BY PlayerName";
```

Line 132 Each name once, alphabetically.

```
        var names = new List<string>();
        await using var reader = await command.ExecuteReaderAsync(cancellationToken);
        while (await reader.ReadAsync(cancellationToken))
            names.Add(reader.GetString(0));
        return names;
    }

```

Lines 133–138 Read the rows one by one with a data reader.

```
    /// <summary>Opens a connection, creating the tables the first time.</summary>
    private async Task<SqliteConnection> OpenAsync(CancellationToken cancellationToken)
    {
```

Lines 140–142 Helper used by every method.

```
        var connection = new SqliteConnection(_connectionString);
        await connection.OpenAsync(cancellationToken);
```

Lines 143–144 Open a new connection (the provider pools them, so this is cheap)…

```
        if (!_schemaCreated)
        {
            using var command = connection.CreateCommand();
            command.CommandText = CreateSchemaSql;
            await command.ExecuteNonQueryAsync(cancellationToken);
            _schemaCreated = true;
        }
        return connection;
    }

```

Lines 145–153 …and the first time, run the CREATE script.

```
    private static async Task<IReadOnlyList<HighScore>> ReadHighScoresAsync(SqliteCommand command, CancellationToken
cancellationToken)
    {
```

Lines 155–156 Converts query rows into `HighScore` records.

```
        var scores = new List<HighScore>();
        await using var reader = await command.ExecuteReaderAsync(cancellationToken);
        while (await reader.ReadAsync(cancellationToken))
        {
```

Lines 157–160 Read each row…

```
161             scores.Add(new HighScore(
162                 PlayerName: reader.GetString(0),
163                 Score: reader.GetInt32(1),
164                 IsAi: reader.GetInt64(2) != 0,
165                 PlayedAt: DateTimeOffset.Parse(reader.GetString(3), CultureInfo.InvariantCulture,
DateTimeStyles.AssumeUniversal)));
166         }
167         return scores;
168     }
169 }
```

Lines 161–169 ...using named arguments for clarity: name, score, 1/0 → bool, and the stored ISO text parsed back to a UTC date.

10.16 Shared AI judgement

`src/Yahtzee.Core/AI/ScoringHeuristics.cs · 35 lines`

"Par" values: what a sensible player typically scores in each box. Strategies use them to decide whether a score is a good use of a box.

```
1  using Yahtzee.Core.Model;
2  using Yahtzee.Core.Rules;
3
```

Lines 1–2 Imports.

```
4  namespace Yahtzee.Core.AI;
5
```

Line 4 AI namespace.

```
6  /// <summary>
7  /// Shared judgement used by the computer strategies: how good a score in a category is compared
8  /// with what a reasonable player usually gets there.
9  /// </summary>
```

Lines 6–9 Documentation.

```
10 internal static class ScoringHeuristics
11 {
```

Lines 10–11 `internal`: only the strategies in this library use it.

```
12     /// <summary>
13     /// "Par" for each category: a rough typical score for a sensible player. Scoring above par is a
14     /// good use of the box; scoring below par wastes it. These are hand-picked estimates, not exact values.
15     /// </summary>
```

Lines 12–15 What par means.

```
16     private static readonly Dictionary<Category, double> Par = new()
17     {
18         [Category.Aces] = 2, [Category.Twos] = 5, [Category.Threes] = 8.5, [Category.Fours] = 12,
19         [Category.Fives] = 15.5, [Category.Sixes] = 19, [Category.Chance] = 22, [Category.ThreeOfAKind] = 16,
20         [Category.FourOfAKind] = 7, [Category.FullHouse] = 15, [Category.SmallStraight] = 18,
21         [Category.LargeStraight] = 11, [Category.Yahtzee] = 9,
22     };
23
```

Lines 16–22 Hand-picked estimates using a dictionary initialiser. For example 22 for Chance: scoring 12 there wastes the box (10 below par), while 2 in Aces is exactly par.

```
24     /// <summary>How much better than par the dice would score in <paramref name="category"/>.</summary>
25     public static double ValueAbovePar(Category category, IReadOnlyList<int> dice) =>
26         Scorer.Score(category, dice) - Par[category];
27
```

Lines 24–26 Score minus par for one category.

```
28     /// <summary>The open category in which the dice score furthest above (or least below) par.</summary>
29     public static Category BestRelativeCategory(IReadOnlyList<int> dice, ScoreCard scoreCard) =>
30         scoreCard.AvailableCategories.MaxBy(category => ValueAbovePar(category, dice));
31
```

Lines 28–30 LINQ `MaxBy`: the open category with the best score-minus-par.

```
32     /// <summary>The open category with the lowest par: the cheapest box to sacrifice.</summary>
33     public static Category CheapestCategory(IEnumerable<Category> categories) =>
34         categories.MinBy(category => Par[category]);
35 }
```

Lines 32–35 `MinBy`: the cheapest box to give up when nothing scores.

10.17 Easy computer player

`src/Yahtzee.Core/AI/RandomStrategy.cs · 35 lines`

Satisfies the "should" requirement (even a trivial strategy is acceptable) and acts as a baseline in the tests.

```
1  using Yahtzee.Core.Model;
2  using Yahtzee.Core.Rules;
3
```

Lines 1–2 Imports.

```
4  namespace Yahtzee.Core.AI;
5
```

Line 4 AI namespace.

```
6  /// <summary>
7  /// The simplest possible opponent: holds random dice, sometimes stops early, and scores in a random
8  /// open category. Useful as a baseline to show that the other strategies really are better.
9  /// </summary>
10 /// <param name="random">Source of randomness; pass a seeded <see cref="Random"/> for repeatable play.</param>
```

Lines 6–10 Documentation.

```
11 public sealed class RandomStrategy(Random? random = null) : IPlayerStrategy
12 {
```

Lines 11–12 Primary constructor with an optional `Random`, so tests can pass a seeded one.

```
13     /// <summary>The name stored in save files for this strategy.</summary>
14     public const string StrategyName = "Random";
15
```

Lines 13–14 A `const` so the catalog and save files use exactly the same text.

```
16     private readonly Random _random = random ?? Random.Shared;
17
```

Line 16 Shared random generator if none was given.

```
18     /// <inheritdoc/>
19     public string Name => StrategyName;
20
```

Lines 18–19 `Name` property from the interface.

```
21     /// <inheritdoc/>
22     public IReadOnlyList<bool>? ChooseHolds(IReadOnlyList<int> dice, int rollsRemaining, ScoreCard scoreCard)
23     {
```

Lines 21–23 `ChooseHolds`:

```
24         if (_random.Next(3) == 0)
25             return null; // stop rolling one time in three
```

Lines 24–25 one time in three, stop rolling;

```
26         return [.. Enumerable.Range(0, Scorer.DiceCount).Select(_ => _random.Next(2) == 0)];
27     }
28
```

Lines 26–27 otherwise hold each die with 50% probability.

```
29     /// <inheritdoc/>
30     public Category ChooseCategory(IReadOnlyList<int> dice, ScoreCard scoreCard)
31     {
32         var open = scoreCard.AvailableCategories.ToList();
33         return open[_random.Next(open.Count)];
34     }
35 }
```

Lines 29–35 `ChooseCategory`: a random open category (always legal).

10.18 Medium computer player

`src/Yahtzee.Core/AI/GreedyStrategy.cs · 63 lines`

Simple rules that a beginner might follow. It plays much better than random (about 176 points on average in the tests).

```
1  using Yahtzee.Core.Model;
2  using Yahtzee.Core.Rules;
3
```

Lines 1–2 Imports.

```
4  namespace Yahtzee.Core.AI;
5
```

Line 4 AI namespace.

```
6  /// <summary>
7  /// A simple rule-based opponent. It chases straights when it has four in a row, otherwise collects as
8  /// many dice of one face as it can, and always scores the highest number of points available.
9  /// </summary>
```

Lines 6–9 Documentation.

```
10 public sealed class GreedyStrategy : IPlayerStrategy
11 {
```

Lines 10–11 No state, so no constructor parameters.

```
12     /// <summary>The name stored in save files for this strategy.</summary>
13     public const string StrategyName = "Greedy";
14
```

Lines 12–13 Saved name.

```
15     /// <inheritdoc/>
16     public string Name => StrategyName;
17
```

Lines 15–16 `Name`.

```
18     /// <inheritdoc/>
19     public IReadOnlyList<bool>? ChooseHolds(IReadOnlyList<int> dice, int rollsRemaining, ScoreCard scoreCard)
20     {
```

Lines 18–20 `ChooseHolds`:

```
21         bool IsOpenAndMade(Category category) => !scoreCard.IsFilled(category) && Scorer.Score(category, dice) > 0;
22
```

Line 21 A **local function**: true if the category is still open and the dice already score in it.

```
23         // Nothing can beat these, so stop rolling.
24         if (IsOpenAndMade(Category.Yahtzee) || IsOpenAndMade(Category.LargeStraight))
25             return null;
26
```

Lines 23–25 With a Yahtzee or large straight available, stop rolling.

```
27         // Four in a row with a straight still open: keep one die of each face in the run.
28         bool straightOpen = !scoreCard.IsFilled(Category.SmallStraight) || !scoreCard.IsFilled(Category.LargeStraight);
29         if (straightOpen && FindRunOfFour(dice) is int runStart)
30         {
31             var kept = new HashSet<int>();
32             return [.. dice.Select(value => value >= runStart && value < runStart + 4 && kept.Add(value))];
33         }
34
```

Lines 27–33 With four consecutive faces and a straight box open, keep one die of each face in the run. `kept.Add` returns false for a duplicate face, so a second copy of a face is re-rolled.

```
35          // Otherwise keep every die showing the most common face (the higher face wins a tie).
36          int target = dice.GroupBy(value => value)
37              .OrderByDescending(group => group.Count())
38              .ThenByDescending(group => group.Key)
39              .First().Key;
```

Lines 35–39 Otherwise group the dice by face, take the biggest group (higher face on a tie)...

```
40          bool[] holds = [.. dice.Select(value => value == target)];
41          return holds.All(held => held) ? null : holds; // holding all five would waste the roll
42      }
43
```

Lines 40–42 ...hold those dice; if that means all five, stop instead of wasting a roll.

```
44      /// <inheritdoc/>
45      public Category ChooseCategory(IReadOnlyList<int> dice, ScoreCard scoreCard)
46      {
```

Lines 44–46 `ChooseCategory`:

```
47          var open = scoreCard.AvailableCategories.ToList();
48          int best = open.Max(category => Scorer.Score(category, dice));
```

Lines 47–48 the highest number of points among open categories...

```
49          // Among the categories giving the most points, use up the one that is usually worth least.
50          return ScoringHeuristics.CheapestCategory(open.Where(category => Scorer.Score(category, dice) == best));
51      }
52
```

Lines 49–51 ...and, if several tie (often all 0), give up the one that is usually worth least.

```
53      /// <summary>The lowest face of a run of four consecutive faces in the dice, or <see langword="null"/>.</summary>
54      private static int? FindRunOfFour(IReadOnlyList<int> dice)
55      {
```

Lines 53–55 `FindRunOfFour`:

```
56          for (int start = 3; start >= 1; start--) // prefer the highest run
57          {
58              if (Enumerable.Range(start, 4).All(dice.Contains))
59                  return start;
60          }
61          return null;
62      }
63  }
```

Lines 56–63 try 3-4-5-6, then 2-3-4-5, then 1-2-3-4; `All(dice.Contains)` checks every face is present.

10.19 Hard computer player

`src/Yahtzee.Core/AI/ExpectedValueStrategy.cs · 98 lines`

Uses probability: it evaluates every possible choice exactly instead of following rules of thumb (about 228 points on average in the tests).

```
1  using Yahtzee.Core.Model;
2  using Yahtzee.Core.Rules;
3
```

Lines 1–2 Imports.

```
4  namespace Yahtzee.Core.AI;
5
```

Line 4 AI namespace.

```
6  /// <summary>
7  /// The strongest opponent. Before each roll it considers all 32 ways of holding dice and, for each,
8  /// averages the result over every possible outcome of the dice it would re-roll. It then makes the
9  /// choice with the best average, or stops if no re-roll is expected to improve on the current dice.
10 /// </summary>
```

Lines 6–10 The idea in one paragraph.

```
11 /// <remarks>
12 /// A result is valued as "points above par" in the best open category (see <see cref="ScoringHeuristics"/>),
13 /// so the strategy saves high-value boxes for good rolls. It looks one roll ahead at a time.
14 /// </remarks>
```

Lines 11–14 How a result is valued (points above par) and the limitation (one roll ahead).

```
15 public sealed class ExpectedValueStrategy : IPlayerStrategy
16 {
```

Lines 15–16 Class declaration.

```
17     /// <summary>The name stored in save files for this strategy.</summary>
18     public const string StrategyName = "ExpectedValue";
19
```

Lines 17–18 Saved name.

```
20     /// <summary>A re-roll must be expected to gain at least this much to be worth it.</summary>
21     private const double MinimumGain = 0.01;
22
```

Lines 20–21 A tiny threshold so that rounding noise never causes pointless re-rolls.

```
23     /// <inheritdoc/>
24     public string Name => StrategyName;
25
```

Lines 23–24 `Name`.

```
26     /// <inheritdoc/>
27     public IReadOnlyList<bool>? ChooseHolds(IReadOnlyList<int> dice, int rollsRemaining, ScoreCard scoreCard)
28     {
```

Lines 26–28 `ChooseHolds`:

```
29         var evaluator = new PositionEvaluator(scoreCard);
30         double bestValue = evaluator.Value(dice);
31         bool[]? bestHolds = null;
32
```

Lines 29–31 the value of stopping now is the starting "best"; no holds chosen yet.

```
33         // Each bit of the mask says whether that die is held; 2^5 = 32 possibilities.
34         for (int mask = 0; mask < 1 << Scorer.DiceCount; mask++)
35         {
```

Lines 33–35 Loop over the 32 numbers 0–31; bit *i* of `mask` says whether die *i* is held (`1 << 5` = 32).

```
36              bool[] holds = [.. Enumerable.Range(0, Scorer.DiceCount).Select(i => (mask & (1 << i)) != 0)];
```

Line 36 Turn the bits into five flags with the bitwise AND operator.

```
37              if (holds.All(held => held))
38                  continue; // holding everything is the same as stopping
39
```

Lines 37–38 Holding everything is the same as stopping, which is already the baseline.

```
40              double expected = evaluator.ExpectedValueAfterReroll(dice, holds);
41              if (expected > bestValue + MinimumGain)
42              {
43                  bestValue = expected;
44                  bestHolds = holds;
45              }
46          }
```

Lines 40–46 Average result of re-rolling the other dice; keep the best choice.

```
47          return bestHolds;
48      }
49
```

Lines 47–48 Null (stop) if nothing beats the current dice.

```
50      /// <inheritdoc/>
51      public Category ChooseCategory(IReadOnlyList<int> dice, ScoreCard scoreCard) =>
52          ScoringHeuristics.BestRelativeCategory(dice, scoreCard);
53
```

Lines 50–52 `ChooseCategory`: the open category furthest above par.

```
54      /// <summary>
55      /// Values dice positions for one scorecard. Positions are cached by their sorted dice, because the
56      /// order of the dice never affects scoring and there are only 252 different sorted hands.
57      /// </summary>
```

Lines 54–57 Why positions are cached: dice order never matters and there are only 252 sorted five-dice hands.

```
58      private sealed class PositionEvaluator(ScoreCard scoreCard)
59      {
```

Lines 58–59 A **nested private class** with a primary constructor; one is created per decision.

```
60          private readonly List<Category> _open = [.. scoreCard.AvailableCategories];
61          private readonly Dictionary<int, double> _cache = [];
62
```

Lines 60–61 The open categories (computed once) and the cache.

```
63          /// <summary>Value of finishing the turn with these dice: the best "points above par" available.</summary>
64          public double Value(IReadOnlyList<int> dice)
65          {
```

Lines 63–65 `Value`:

```
66              int[] sorted = [.. dice.Order()];
67              int key = sorted.Aggregate(0, (acc, value) => acc * 7 + value);
```

Lines 66–67 sort the dice and encode them as one number in base 7 (each digit 1–6) to use as the cache key;

```
68              if (!_cache.TryGetValue(key, out double value))
69              {
70                  value = _open.Max(category => ScoringHeuristics.ValueAbovePar(category, sorted));
71                  _cache[key] = value;
72              }
73              return value;
74          }
75
```

Lines 68–74 compute the best points-above-par only on a cache miss.

```
76          /// <summary>Average <see cref="Value"/> over every equally likely outcome of re-rolling the unheld dice.</summary>
77          public double ExpectedValueAfterReroll(IReadOnlyList<int> dice, IReadOnlyList<bool> holds)
78          {
```

Lines 76–78 `ExpectedValueAfterReroll`:

```
79              int[] rerolled = [.. Enumerable.Range(0, Scorer.DiceCount).Where(i => !holds[i])];
80              int outcomes = (int)Math.Pow(6, rerolled.Length);
81              int[] next = [.. dice];
82              double total = 0;
83
```

Lines 79–82 the indexes of the dice to re-roll; 6^k equally likely outcomes for k dice (at most 7776).

```
84              for (int outcome = 0; outcome < outcomes; outcome++)
85              {
```

Lines 84–85 Enumerate every outcome exactly (no random sampling)…

```
86                  // Treat the outcome number as base-6 digits: one digit per re-rolled die.
87                  int code = outcome;
88                  foreach (int index in rerolled)
89                  {
90                      next[index] = code % 6 + 1;
91                      code /= 6;
92                  }
```

Lines 86–92 …by reading the outcome number as base-6 digits, one per re-rolled die (digit + 1 = face)…

```
93                  total += Value(next);
94              }
95              return total / outcomes;
96          }
97      }
98  }
```

Lines 93–98 …sum the values and return the average (the **expected value**).

10.20 Strategy registry

`src/Yahtzee.Core/AI/StrategyCatalog.cs · 31 lines`

The one place that knows every strategy: used by the new-game dialog and when loading saved games.

```
1  namespace Yahtzee.Core.AI;
2
```

Line 1 AI namespace.

```
3  /// <summary>
4  /// The list of available computer strategies, looked up by the name stored in save files.
5  /// Adding a strategy means adding it here; the new-game dialog and save loading pick it up automatically.
6  /// </summary>
```

Lines 3–6 Documentation.

```
7  public static class StrategyCatalog
8  {
```

Lines 7–8 Static class.

```
9      /// <summary>Every strategy name, from weakest to strongest.</summary>
10     public static IReadOnlyList<string> Names { get; } =
11         [RandomStrategy.StrategyName, GreedyStrategy.StrategyName, ExpectedValueStrategy.StrategyName];
12
```

Lines 9–11 Names from weakest to strongest (the order of the drop-down).

```
13     /// <summary>Creates a new instance of the strategy called <paramref name="name"/>.</summary>
14     /// <exception cref="ArgumentException">No strategy has that name.</exception>
15     public static IPlayerStrategy Create(string name) => name switch
16     {
17         RandomStrategy.StrategyName => new RandomStrategy(),
18         GreedyStrategy.StrategyName => new GreedyStrategy(),
19         ExpectedValueStrategy.StrategyName => new ExpectedValueStrategy(),
20         _ => throw new ArgumentException($"Unknown AI strategy \"{name}\".", nameof(name)),
21     };
22
```

Lines 13–21 `Create`: a switch expression from name to a new object; an unknown name throws `ArgumentException`.

```
23     /// <summary>A friendly description of the strategy for the user interface.</summary>
24     public static string Describe(string name) => name switch
25     {
26         RandomStrategy.StrategyName => "Computer: easy (random)",
27         GreedyStrategy.StrategyName => "Computer: medium (greedy)",
28         ExpectedValueStrategy.StrategyName => "Computer: hard (expected value)",
29         _ => name,
30     };
31 }
```

Lines 23–31 `Describe`: the text shown in the drop-down, e.g. "Computer: hard (expected value)".

11. Line-by-line walkthrough: Yahtzee.WinForms: the user interface

11.1 Application entry point

`src/Yahtzee.WinForms/Program.cs · 21 lines`

The first code that runs when `Yahtzee.exe` starts. It creates the games-history database object and the main window.

```
1  using Yahtzee.Core.Data;
2  using Yahtzee.WinForms.Forms;
3
```

Lines 1–2 Imports the database classes and the namespace containing `MainForm`.

```
4  namespace Yahtzee.WinForms;
5
```

Line 4 Root namespace of the WinForms project.

```
6  internal static class Program
7  {
```

Lines 6–7 `internal`: visible only inside this project. `static`: never instantiated.

```
8      /// <summary>Application entry point.</summary>
9      /// <param name="args">
10     /// Optional <c>--database &lt;file&gt;</c> to use a different games-history database
11     /// (for example a fresh one for a demonstration); otherwise the default location is used.
12     /// </param>
```

Lines 8–12 Documentation, including the optional command-line argument.

```
13     [STAThread]
14     private static void Main(string[] args)
15     {
```

Lines 13–15 Windows Forms requires the UI thread to use the single-threaded apartment COM model (needed for dialogs, clipboard, drag-and-drop). `args` receives the command-line arguments.

```
16         ApplicationConfiguration.Initialize();
```

Line 16 Generated helper that applies the project settings: visual styles, default font, and the `PerMonitorV2` high-DPI mode from the `.csproj`.

```
17         string databasePath = args is ["--database", var path] ? path : SqliteGameHistoryRepository.DefaultDatabasePath;
```

Line 17 A **list pattern**: if the arguments are exactly `--database <file>`, use that file, otherwise the default `%LOCALAPPDATA%\Yahtzee\yahtzee.db`. Handy for a clean demo and used by the automated UI tests, so test games never pollute the real high scores.

```
18         var history = new SqliteGameHistoryRepository(databasePath);
```

Line 18 Creates the database object. The form only receives the `IGameHistoryRepository` interface (**dependency injection** through the constructor).

```
19         Application.Run(new MainForm(history));
20     }
21 }
```

Lines 19–21 Creates the main window and starts the Windows message loop; the method returns when the window is closed.

11.2 Shared colours and fonts

`src/Yahtzee.WinForms/UiTheme.cs · 25 lines`

One place for every colour and font, so all forms look consistent and the look can be changed in one file.

```
1  namespace Yahtzee.WinForms;
2
```

Line 1 Root namespace.

```
3  /// <summary>Shared colours and fonts so every form has a consistent look.</summary>
```

Line 3 Documentation.

```
4  internal static class UiTheme
5  {
```

Lines 4–5 A static class of `static readonly` fields (created once, never reassigned).

```
6      public static readonly Color Felt = Color.FromArgb(22, 94, 60);
7      public static readonly Color FeltText = Color.WhiteSmoke;
```

Lines 6–7 Dark green "card table felt" background of the dice area, and the light text drawn on it.

```
8      public static readonly Color DieFace = Color.White;
9      public static readonly Color DieBorder = Color.FromArgb(60, 60, 60);
10     public static readonly Color Pip = Color.FromArgb(30, 30, 30);
```

Lines 8–10 White die face, dark border, near-black pips.

```
11     public static readonly Color HeldFace = Color.FromArgb(255, 236, 160);
12     public static readonly Color HeldBorder = Color.FromArgb(230, 160, 0);
```

Lines 11–12 Held dice are drawn pale gold with a darker gold border, so held dice stand out (requirement: show which dice are set aside).

```
13     public static readonly Color Accent = Color.FromArgb(0, 102, 204);
```

Line 13 Blue used for non-zero preview scores.

```
14     public static readonly Color CurrentPlayerHeader = Color.FromArgb(255, 220, 120);
15     public static readonly Color HeaderBack = Color.FromArgb(235, 235, 235);
```

Lines 14–15 Gold header for the current player's column, and the neutral header grey.

```
16     public static readonly Color PreviewBack = Color.FromArgb(232, 243, 255);
17     public static readonly Color PreviewZero = Color.FromArgb(150, 150, 150);
```

Lines 16–17 Light blue background of the "This roll" preview column and grey for zero previews.

```
18     public static readonly Color TotalBack = Color.FromArgb(245, 245, 245);
19
```

Line 18 Background of the TOTAL row.

```
20     public static readonly Font Body = new("Segoe UI", 10F);
21     public static readonly Font Bold = new("Segoe UI", 10F, FontStyle.Bold);
```

Lines 20–21 Segoe UI (the Windows 11 UI font) in regular and bold.

```
22     public static readonly Font Heading = new("Segoe UI Semibold", 16F);
23     public static readonly Font Subheading = new("Segoe UI", 11F);
```

Lines 22–23 Larger fonts for the turn heading and subheadings.

```
24     public static readonly Font DieCaption = new("Segoe UI", 8F, FontStyle.Bold);
25 }
```

Lines 24–25 Small bold font for the HELD / WILL ROLL captions under each die.

11.3 Custom-drawn die

`src/Yahtzee.WinForms/Controls/DieControl.cs · 163 lines`

A reusable control that paints one die with pips, and a caption saying whether it is held or will be re-rolled. Painting the die ourselves (instead of using images) keeps it sharp at every display scale.

```
1  using System.ComponentModel;
2  using System.Drawing.Drawing2D;
3
```

Lines 1–2 `ComponentModel` provides the designer attributes; `Drawing2D` provides anti-aliasing and `GraphicsPath` for rounded corners.

```
4  namespace Yahtzee.WinForms.Controls;
5
```

Line 4 Controls namespace.

```
6  /// <summary>
7  /// A clickable, custom-drawn die. Held dice are drawn in gold with a "HELD" caption; dice that
8  /// will be re-rolled show "WILL ROLL", so it is always clear which dice the next roll affects.
9  /// </summary>
```

Lines 6–9 Class summary describing the visual states.

```
10 internal sealed class DieControl : Control
11 {
```

Lines 10–11 Inherits `Control`, the base class of every WinForms control; we supply the painting.

```
12     /// <summary>
13     /// Pip positions for each face on a 3x3 grid (column, row), indexed by face value.
14     /// Index 0 is unused.
15     /// </summary>
```

Lines 12–15 Explains the pip lookup table.

```
16     private static readonly (int Col, int Row)[][] PipLayout =
17     [
```

Lines 16–17 A **jagged array of tuples**: for each face value, the list of pip positions on a 3×3 grid (column, row from 0 to 2). Written with collection expressions.

```
18         [],
19         [(1, 1)],
20         [(0, 0), (2, 2)],
21         [(0, 0), (1, 1), (2, 2)],
22         [(0, 0), (2, 0), (0, 2), (2, 2)],
23         [(0, 0), (2, 0), (1, 1), (0, 2), (2, 2)],
24         [(0, 0), (2, 0), (0, 1), (2, 1), (0, 2), (2, 2)],
25     ];
26
```

Lines 18–25 Index 0 is unused (no face 0). Face 1: centre. Face 2: two opposite corners. Face 3: diagonal. Face 4: four corners. Face 5: corners + centre. Face 6: two columns of three.

```
27     private int _value;
28     private bool _held;
29     private bool _showRollCaption;
30
```

Lines 27–29 Backing fields for the three properties below.

```
31     public DieControl()
32     {
```

Lines 31–32 Constructor.

```
33         SetStyle(ControlStyles.AllPaintingInWmPaint | ControlStyles.OptimizedDoubleBuffer |
34                  ControlStyles.UserPaint | ControlStyles.ResizeRedraw | ControlStyles.SupportsTransparentBackColor, true);
```

Lines 33–34 Enables double buffering (no flicker), full custom painting, repainting on resize, and transparent backgrounds.

```
        BackColor = Color.Transparent;
        Cursor = Cursors.Hand;
        TabStop = true;
```

Lines 35–37 Transparent background so the green felt shows around the rounded die; hand cursor; can receive keyboard focus (Tab).

```
        Size = new Size(76, 96);
        Margin = new Padding(6);
    }

```

Lines 38–40 Default size in logical pixels (scaled automatically by the form's DPI scaling) and spacing between dice.

```
    /// <summary>The face shown, 1–6, or 0 before the first roll of a turn.</summary>
    [Browsable(false), DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]
    public int Value
    {
        get => _value;
        set { if (_value != value) { _value = value; Invalidate(); } }
    }

```

Lines 42–48 `Value` property (0 = not rolled yet, drawn as "?"). The attributes hide it from the designer and stop the designer serializing it (the WFO1000 analyzer requires this). The setter only repaints when the value really changes.

```
    /// <summary>Whether the die is set aside and will not be re-rolled.</summary>
    [Browsable(false), DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]
    public bool Held
    {
        get => _held;
        set { if (_held != value) { _held = value; Invalidate(); } }
    }

```

Lines 50–56 `Held` property, same pattern.

```
    /// <summary>Whether another roll is available, so non-held dice should be labelled "WILL ROLL".</summary>
    [Browsable(false), DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]
    public bool ShowRollCaption
    {
        get => _showRollCaption;
        set { if (_showRollCaption != value) { _showRollCaption = value; Invalidate(); } }
    }

```

Lines 58–64 `ShowRollCaption`: when true, non-held dice show "WILL ROLL".

```
    protected override void OnEnabledChanged(EventArgs e)
    {
        base.OnEnabledChanged(e);
        Cursor = Enabled ? Cursors.Hand : Cursors.Default;
        Invalidate();
    }

```

Lines 66–71 When enabled/disabled, switch the cursor and repaint (disabled dice look slightly faded).

```
    protected override void OnGotFocus(EventArgs e) { base.OnGotFocus(e); Invalidate(); }

    protected override void OnLostFocus(EventArgs e) { base.OnLostFocus(e); Invalidate(); }

```

Lines 73–75 Repaint when focus arrives or leaves, to draw or remove the focus rectangle.

```
77      /// <summary>Space or Enter toggles the die, like clicking it.</summary>
78      protected override void OnKeyDown(KeyEventArgs e)
79      {
80          base.OnKeyDown(e);
81          if (e.KeyCode is Keys.Space or Keys.Enter)
82          {
83              OnClick(EventArgs.Empty);
84              e.Handled = true;
85          }
86      }
87
```

Lines 77–86 Keyboard accessibility: Space/Enter on a focused die acts like a click.

```
88      protected override void OnPaint(PaintEventArgs e)
89      {
90          base.OnPaint(e);
91          var g = e.Graphics;
92          g.SmoothingMode = SmoothingMode.AntiAlias;
93
```

Lines 88–92 The painting routine. Anti-aliasing smooths the rounded corners and pips.

```
94          int captionHeight = LogicalToDeviceUnits(20);
95          int side = Math.Min(Width, Height - captionHeight) - 4;
96          var faceRect = new Rectangle((Width - side) / 2, 2, side, side);
97
```

Lines 94–96 Layout: caption height converted to device pixels for the current DPI; the face is the largest square that fits above it, centred.

```
98          DrawFace(g, faceRect);
99          DrawCaption(g, new Rectangle(0, faceRect.Bottom + 2, Width, captionHeight));
100     }
101
```

Lines 98–100 Draws the face, then the caption beneath it.

```
102     private void DrawFace(Graphics g, Rectangle rect)
103     {
104         using var path = RoundedRectangle(rect, rect.Width / 6);
```

Lines 102–104 Builds a rounded-rectangle outline (corner radius = 1/6 of the width). `using var` disposes the GDI object automatically.

```
105         Color face = _held ? UiTheme.HeldFace : UiTheme.DieFace;
106         if (!Enabled && _value != 0)
107             face = ControlPaint.Light(face, 0.1f);
108
```

Lines 105–107 Gold when held, white otherwise; slightly lightened when the control is disabled.

```
109         using (var fill = new SolidBrush(face))
110             g.FillPath(fill, path);
111         using (var border = new Pen(_held ? UiTheme.HeldBorder : UiTheme.DieBorder, _held ? 4f : 2f))
112             g.DrawPath(border, path);
113
```

Lines 109–112 Fills the shape and draws a thicker border for held dice.

```
114         if (Focused)
115         {
116             var focusRect = Rectangle.Inflate(rect, -6, -6);
117             ControlPaint.DrawFocusRectangle(g, focusRect);
118         }
119
```

Lines 114–118 Dotted focus rectangle for keyboard users.

```
120         if (_value == 0)
121         {
122             TextRenderer.DrawText(g, "?", UiTheme.Heading, rect, Color.Gray,
123                 TextFormatFlags.HorizontalCenter | TextFormatFlags.VerticalCenter);
124             return;
125         }
126
```

Lines 120–125 Before the first roll the die shows a grey "?" and nothing else.

```
127         float cell = rect.Width / 4f;
128         float pipRadius = rect.Width / 11f;
129         using var pipBrush = new SolidBrush(UiTheme.Pip);
```

Lines 127–129 Pip geometry: the 3×3 grid points sit at ¼, ½ and ¾ of the face; pip radius is 1/11 of the width.

```
130         foreach (var (col, row) in PipLayout[_value])
131         {
132             float cx = rect.Left + cell * (col + 1);
133             float cy = rect.Top + cell * (row + 1);
134             g.FillEllipse(pipBrush, cx - pipRadius, cy - pipRadius, pipRadius * 2, pipRadius * 2);
135         }
136     }
137
```

Lines 130–136 Looks up the pip positions for this face (tuple deconstruction) and draws a filled circle at each.

```
138     private void DrawCaption(Graphics g, Rectangle rect)
139     {
```

Lines 138–139 Draws the caption under the die.

```
140         (string text, Color color) = (_value, _held, _showRollCaption) switch
141         {
142             (0, _, _) => ("", Color.Empty),
143             (_, true, _) => ("HELD", UiTheme.HeldFace),
144             (_, false, true) => ("WILL ROLL", UiTheme.FeltText),
145             _ => ("", Color.Empty),
146         };
```

Lines 140–146 A switch expression on a **tuple** of three values with positional patterns: unrolled → nothing; held → "HELD" in gold; not held but another roll is available → "WILL ROLL"; otherwise (no rolls left) nothing.

```
147         if (text.Length > 0)
148             TextRenderer.DrawText(g, text, UiTheme.DieCaption, rect, color,
149                 TextFormatFlags.HorizontalCenter | TextFormatFlags.VerticalCenter);
150     }
151
```

Lines 147–150 Draws the caption centred.

```
152     private static GraphicsPath RoundedRectangle(Rectangle rect, int radius)
153     {
```

Lines 152–153 Helper building a rounded rectangle path.

```
154         int d = radius * 2;
155         var path = new GraphicsPath();
```

Lines 154–155 `d` is the diameter of each corner arc.

```
156         path.AddArc(rect.Left, rect.Top, d, d, 180, 90);
157         path.AddArc(rect.Right - d, rect.Top, d, d, 270, 90);
158         path.AddArc(rect.Right - d, rect.Bottom - d, d, d, 0, 90);
159         path.AddArc(rect.Left, rect.Bottom - d, d, d, 90, 90);
```

Lines 156–159 Four 90° arcs, one per corner, clockwise from top-left; GDI+ joins them with straight edges.

```
160         path.CloseFigure();
161         return path;
162     }
163 }
```

Lines 160–163 Closes the outline and returns it.

11.4 Player type choices

`src/Yahtzee.WinForms/Forms/PlayerTypeOption.cs · 26 lines`

The entries of the "Type" drop-down in the new-game dialog: Human, then one entry per computer strategy.

```
1  using Yahtzee.Core.AI;
2
```

Line 1 For `IPlayerStrategy` and `StrategyCatalog`.

```
3  namespace Yahtzee.WinForms.Forms;
4
```

Line 3 Forms namespace.

```
5  /// <summary>
6  /// One choice in the "Type" drop-down of the new-game dialog: a human, or an AI with a given strategy.
7  /// </summary>
8  /// <param name="Label">Text shown in the drop-down.</param>
9  /// <param name="CreateStrategy">Creates the AI strategy, or returns <see langword="null"/> for a human.</param>
```

Lines 5–9 Documentation, including the two record parameters.

```
10 internal sealed record PlayerTypeOption(string Label, Func<IPlayerStrategy?> CreateStrategy)
11 {
```

Lines 10–11 A record holding a label and a **factory delegate** `Func<IPlayerStrategy?>` that creates a fresh strategy (or null for a human) for each player.

```
12     /// <summary>A human player.</summary>
13     public static PlayerTypeOption Human { get; } = new("Human", () => null);
14
```

Lines 12–13 The Human option: its factory is a lambda returning null.

```
15     /// <summary>All selectable player types: Human, then one entry per strategy in <see cref="StrategyCatalog"/>.</summary>
16     public static IReadOnlyList<PlayerTypeOption> All { get; } =
17     [
```

Lines 15–17 The list shown in the drop-down, built with a collection expression…

```
18         Human,
```

Line 18 …whose first element is Human…

```
19         .. StrategyCatalog.Names.Select(name => new PlayerTypeOption(StrategyCatalog.Describe(name), () =>
StrategyCatalog.Create(name))),
20     ];
21
```

Lines 19–20 …followed (the **spread** `..`) by one option per strategy name: the label comes from `StrategyCatalog.Describe` (e.g. "Computer: hard (expected value)") and the factory calls `StrategyCatalog.Create`. A new strategy registered in the catalog appears here automatically.

```
22     /// <summary>Whether this option is a computer player.</summary>
23     public bool IsComputer => this != Human;
24
```

Lines 22–23 `IsComputer`: everything except Human. Records compare by value, and `Human` is a single shared instance, so `!=` works.

```
25     public override string ToString() => Label;
26 }
```

Lines 25–26 A ComboBox displays each item's `ToString()`, so this makes it show the label.

11.5 New-game dialog

`src/Yahtzee.WinForms/Forms/NewGameDialog.cs · 159 lines`

Lets the user choose 1–6 players, type each name and choose each player's type (human or one of three computer levels). Validates the input before the game starts.

```
1  using Yahtzee.Core.Game;
2  using Yahtzee.Core.Model;
3
```

Lines 1–2 For `GameEngine.MaxPlayers` and `Player`.

```
4  namespace Yahtzee.WinForms.Forms;
5
```

Line 4 Forms namespace.

```
6  /// <summary>
7  /// Dialog for setting up a new game: which seats are in play, each player's name, and whether
8  /// each player is a human or a computer opponent.
9  /// </summary>
```

Lines 6–9 Documentation.

```
10 internal sealed class NewGameDialog : Form
11 {
```

Lines 10–11 A `Form` used as a modal dialog.

```
12     /// <summary>The controls for one seat at the table.</summary>
13     private sealed record SeatRow(CheckBox Include, TextBox Name, ComboBox Type);
14
```

Lines 12–13 A **private nested record** grouping the three controls of one row (checkbox, name box, type box).

```
15     private readonly List<SeatRow> _seats = [];
16     private readonly ErrorProvider _errors = new() { BlinkStyle = ErrorBlinkStyle.NeverBlink };
17
```

Lines 15–16 All six rows, and an `ErrorProvider` that shows a red icon with a tooltip beside invalid names.

```
18     public NewGameDialog()
19     {
```

Lines 18–19 Constructor: builds the whole dialog in code.

```
20         SuspendLayout();
21         AutoScaleDimensions = new SizeF(96F, 96F);
22         AutoScaleMode = AutoScaleMode.Dpi;
```

Lines 20–22 Pauses layout while controls are added, and turns on DPI-based automatic scaling (96 DPI = 100%).

```
23         Text = "New Game";
24         Font = UiTheme.Body;
```

Lines 23–24 Window title and font.

```
25         FormBorderStyle = FormBorderStyle.FixedDialog;
26         MaximizeBox = false;
27         MinimizeBox = false;
28         ShowInTaskbar = false;
29         StartPosition = FormStartPosition.CenterParent;
```

Lines 25–29 Standard dialog appearance: fixed border, no minimize/maximize, no taskbar entry, centred on the main window.

```
30         AutoSize = true;
31         AutoSizeMode = AutoSizeMode.GrowAndShrink;
32         Padding = new Padding(16);
33
```

Lines 30–32 The dialog sizes itself to fit its content, with 16 px padding.

```
34          var layout = new TableLayoutPanel
35          {
36              AutoSize = true,
37              ColumnCount = 3,
38              Dock = DockStyle.Fill,
39          };
40
```

Lines 34–39 A 3-column table (Player / Name / Type) that lays the controls out in a grid.

```
41          var intro = new Label
42          {
43              Text = $"Choose up to {GameEngine.MaxPlayers} players. Players take turns in the order listed.",
44              AutoSize = true,
45              Margin = new Padding(3, 0, 3, 12),
46          };
47          layout.Controls.Add(intro, 0, 0);
48          layout.SetColumnSpan(intro, 3);
49
```

Lines 41–48 Instruction text spanning all three columns.

```
50          layout.Controls.Add(new Label { Text = "Player", Font = UiTheme.Bold, AutoSize = true }, 0, 1);
51          layout.Controls.Add(new Label { Text = "Name", Font = UiTheme.Bold, AutoSize = true }, 1, 1);
52          layout.Controls.Add(new Label { Text = "Type", Font = UiTheme.Bold, AutoSize = true }, 2, 1);
53
```

Lines 50–52 Bold column headings in row 1.

```
54          for (int seat = 0; seat < GameEngine.MaxPlayers; seat++)
55          {
```

Lines 54–55 One row per seat, 6 in total.

```
56              var row = CreateSeatRow(seat);
57              _seats.Add(row);
```

Lines 56–57 Builds the controls for the seat and remembers them.

```
58              layout.Controls.Add(row.Include, 0, seat + 2);
59              layout.Controls.Add(row.Name, 1, seat + 2);
60              layout.Controls.Add(row.Type, 2, seat + 2);
61          }
62
```

Lines 58–61 Places the three controls in their columns (rows start at 2, after the intro and headings).

```
63          var buttons = new FlowLayoutPanel
64          {
65              FlowDirection = FlowDirection.RightToLeft,
66              AutoSize = true,
67              Dock = DockStyle.Fill,
68              Margin = new Padding(0, 16, 0, 0),
69          };
```

Lines 63–69 Right-aligned button bar.

```
70          var cancel = new Button { Text = "Cancel", DialogResult = DialogResult.Cancel, AutoSize = true, MinimumSize = new
Size(90, 32) };
71          var start = new Button { Text = "Start Game", AutoSize = true, MinimumSize = new Size(110, 32) };
```

Lines 70–71 Cancel closes the dialog with `DialogResult.Cancel`; Start Game runs the validation instead of closing directly.

```
72          start.Click += (_, _) => TryAccept();
73          buttons.Controls.Add(cancel);
74          buttons.Controls.Add(start);
75          layout.Controls.Add(buttons, 0, GameEngine.MaxPlayers + 2);
76          layout.SetColumnSpan(buttons, 3);
77
```

Lines 72–76 Wires the click handler and places the button bar below the rows.

```
78          Controls.Add(layout);
79          AcceptButton = start;
80          CancelButton = cancel;
81          ResumeLayout(false);
82          PerformLayout();
83      }
84
```

Lines 78–83 Adds the table to the form; Enter = Start Game, Esc = Cancel; resumes layout.

```
85      /// <summary>The players chosen, in turn order. Only valid after the dialog returns <see cref="DialogResult.OK"/>.</summary>
86      public IReadOnlyList<Player> Players { get; private set; } = [];
87
```

Lines 85–86 Result property read by `MainForm` after OK. Initialised to an empty list.

```
88      private SeatRow CreateSeatRow(int seat)
89      {
90          bool firstSeat = seat == 0;
```

Lines 88–90 Creates the controls for one seat. Seat 1 is special because a game always has at least one player.

```
91          var include = new CheckBox
92          {
93              Text = $"Player {seat + 1}",
94              Checked = firstSeat,
95              Enabled = !firstSeat, // a game always has at least one player
96              AutoSize = true,
97              Anchor = AnchorStyles.Left,
98          };
```

Lines 91–98 "Player n" checkbox: seat 1 is always checked and cannot be unchecked.

```
99          var name = new TextBox
100         {
101             Text = $"Player {seat + 1}",
102             MaxLength = Player.MaxNameLength,
103             Width = 200,
104             Enabled = firstSeat,
105             Margin = new Padding(3, 3, 24, 3),
106         };
```

Lines 99–106 Name box with a default name and the 20-character limit from `Player.MaxNameLength`; disabled until the seat is included.

```
107         var type = new ComboBox
108         {
109             DropDownStyle = ComboBoxStyle.DropDownList,
110             Width = 250,
111             Enabled = firstSeat,
112         };
```

Lines 107–112 Drop-down list (no free typing), wide enough for the computer descriptions.

```
113         type.Items.AddRange([.. PlayerTypeOption.All]);
114         type.SelectedIndex = 0;
115
```

Lines 113–114 Fills it with `PlayerTypeOption.All` (spread into an array) and selects the first entry (Human).

```
116         // Keep default names meaningful: "Player 2" becomes "Computer 2" when switched to a computer, and back.
117         string humanName = $"Player {seat + 1}", computerName = $"Computer {seat + 1}";
118         type.SelectedIndexChanged += (_, _) =>
119         {
120             if (name.Text == humanName || name.Text == computerName)
121                 name.Text = ((PlayerTypeOption)type.SelectedItem!).IsComputer ? computerName : humanName;
122         };
123
```

Lines 116–122 If the name is still the default, switching the type renames the player "Computer n" (or back to "Player n"). A name the user typed is never overwritten. The two local strings are captured by the lambda (a **closure**).

```
124         include.CheckedChanged += (_, _) =>
125         {
126             name.Enabled = type.Enabled = include.Checked;
127             _errors.SetError(name, "");
128         };
```

Lines 124–128 Ticking/unticking a seat enables/disables its name and type and clears any error.

```
129         name.TextChanged += (_, _) => _errors.SetError(name, "");
130
```

Line 129 Editing a name clears its error icon.

```
131         return new SeatRow(include, name, type);
132     }
133
```

Lines 131–132 Returns the three controls as one record.

```
134     /// <summary>Validates the names and, if they are all valid, builds the players and closes the dialog.</summary>
135     private void TryAccept()
136     {
```

Lines 134–136 `TryAccept`: called by Start Game.

```
137         var included = _seats.Where(seat => seat.Include.Checked).ToList();
138         var seenNames = new HashSet<string>(StringComparer.OrdinalIgnoreCase);
139         bool valid = true;
140
```

Lines 137–139 Only checked seats count. A case-insensitive `HashSet` detects duplicate names.

```
141         foreach (var seat in included)
142         {
```

Lines 141–142 Check each included seat.

```
143             string name = seat.Name.Text.Trim();
144             string error = name.Length == 0 ? "Please enter a name."
145                 : !seenNames.Add(name) ? "Each player needs a different name."
146                 : "";
```

Lines 143–146 Chained conditional operators choose the error: blank name, duplicate name (`HashSet.Add` returns false for a duplicate), or none.

```
147             _errors.SetError(seat.Name, error);
148             valid &= error.Length == 0;
149         }
150
```

Lines 147–149 Show (or clear) the error icon; `valid` becomes false if any error exists.

```
151         if (!valid)
152             return;
153
```

Lines 151–152 Stay open so the user can fix the names.

```
154         Players = included
155             .Select(seat => new Player(seat.Name.Text, ((PlayerTypeOption)seat.Type.SelectedItem!).CreateStrategy()))
156             .ToList();
```

Lines 154–156 Builds a `Player` per seat, calling the selected option's factory to create its strategy.

```
157         DialogResult = DialogResult.OK;
158     }
159 }
```

Lines 157–159 Setting `DialogResult` closes the modal dialog and returns OK to the caller.

11.6 End-of-game results

`src/Yahtzee.WinForms/Forms/ResultsForm.cs · 103 lines`

Shows the required **summary of results**: names and scores ordered by score, with the winner (or a tie) announced.

```
1  using Yahtzee.Core.Game;
2
```

Line 1 For `PlayerStanding`.

```
3  namespace Yahtzee.WinForms.Forms;
4
```

Line 3 Forms namespace.

```
5  /// <summary>End-of-game summary: every player's name and score, highest first.</summary>
6  internal sealed class ResultsForm : Form
7  {
```

Lines 5–7 Documentation.

```
8      /// <param name="standings">Final standings, already sorted by score (see <see cref="GameEngine.GetStandings"/>).</param>
9      public ResultsForm(IReadOnlyList<PlayerStanding> standings)
10     {
```

Lines 8–10 The constructor receives the standings, already sorted by the engine.

```
11         SuspendLayout();
12         AutoScaleDimensions = new SizeF(96F, 96F);
13         AutoScaleMode = AutoScaleMode.Dpi;
```

Lines 11–13 Same dialog setup as the new-game dialog: DPI scaling…

```
14         Text = "Game Over";
15         Font = UiTheme.Body;
```

Lines 14–15 …title and font…

```
16         FormBorderStyle = FormBorderStyle.FixedDialog;
17         MaximizeBox = false;
18         MinimizeBox = false;
19         ShowInTaskbar = false;
20         StartPosition = FormStartPosition.CenterParent;
```

Lines 16–20 …fixed dialog style, centred on the main window…

```
21         AutoSize = true;
22         AutoSizeMode = AutoSizeMode.GrowAndShrink;
23         Padding = new Padding(16);
24
```

Lines 21–23 …sized to its content.

```
25         var layout = new TableLayoutPanel { AutoSize = true, ColumnCount = 1, Dock = DockStyle.Fill };
26
```

Line 25 Single-column vertical layout.

```
27         layout.Controls.Add(new Label
28         {
29             Text = DescribeWinner(standings),
30             Font = UiTheme.Heading,
31             ForeColor = UiTheme.Felt,
32             AutoSize = true,
33             MaximumSize = new Size(420, 0),
34             Margin = new Padding(3, 0, 3, 12),
35         });
36
```

Lines 27–35 Large green heading produced by `DescribeWinner`, wrapping at 420 px.

```
37          var table = new ListView
38          {
39              View = View.Details,
40              FullRowSelect = true,
41              HeaderStyle = ColumnHeaderStyle.Nonclickable,
42              Size = new Size(420, 40 + 28 * standings.Count),
43              MultiSelect = false,
44          };
```

Lines 37–44 A `ListView` in Details mode shows a simple table; its height grows with the number of players.

```
45          table.Columns.Add("Rank");
46          table.Columns.Add("Player");
47          table.Columns.Add("Score", -2, HorizontalAlignment.Right);
```

Lines 45–47 Three columns; `-2` means "fill the remaining width".

```
48          // Column widths are pixels and are not DPI-scaled, so share out the list's real width once it is known.
49          Load += (_, _) =>
50          {
51              int width = table.ClientSize.Width;
52              table.Columns[0].Width = width * 18 / 100;
53              table.Columns[1].Width = width * 57 / 100;
54              table.Columns[2].Width = width - table.Columns[0].Width - table.Columns[1].Width;
55          };
```

Lines 48–55 Column widths are not scaled by WinForms' automatic DPI scaling, so they are set in the `Load` event, when the list's real (scaled) width is known: 18% / 57% / remainder. This fixed an empty strip found during testing at 150% display scaling.

```
56          foreach (var standing in standings)
57          {
```

Lines 56–57 One row per player.

```
58              string name = standing.IsAi ? $"{standing.Name} (AI)" : standing.Name;
59              var item = new ListViewItem([Ordinal(standing.Rank), name, standing.Score.ToString()]);
```

Lines 58–59 Adds "(AI)" to computer players and builds the row cells (ordinal rank, name, score).

```
60              if (standing.Rank == 1)
61                  item.Font = UiTheme.Bold;
62              table.Items.Add(item);
63          }
```

Lines 60–63 Winners are shown in bold; the row is added.

```
64          layout.Controls.Add(table);
65
```

Line 64 Adds the table to the layout.

```
66          var close = new Button
67          {
68              Text = "Close",
69              DialogResult = DialogResult.OK,
70              AutoSize = true,
71              MinimumSize = new Size(90, 32),
72              Anchor = AnchorStyles.Right,
73              Margin = new Padding(3, 16, 3, 3),
74          };
```

Lines 66–74 Close button, right-aligned.

```
75          layout.Controls.Add(close);
76
```

Line 75 Adds the button.

```
77          Controls.Add(layout);
78          AcceptButton = close;
79          CancelButton = close;
80          ResumeLayout(false);
81          PerformLayout();
82      }
83
```

Lines 77–82 Enter and Esc both close; layout resumes.

```
84      private static string DescribeWinner(IReadOnlyList<PlayerStanding> standings)
85      {
```

Lines 84–85 Builds the heading text.

```
86          var winners = standings.Where(standing => standing.Rank == 1).ToList();
87          return winners switch
88          {
```

Lines 86–88 Everyone with rank 1, then a switch on the list using **list patterns** (C# 11).

```
89              [var only] when standings.Count == 1 => $"Final score: {only.Score} points",
90              [var only] => $"{only.Name} wins with {only.Score} points!",
```

Lines 89–90 `[var only]` matches a list with exactly one element: a single-player game shows "Final score", otherwise "X wins".

```
91              _ => $"It's a tie! {string.Join(" and ", winners.Select(w => w.Name))} share first place with {winners[0].Score}
points.",
92          };
93      }
94
```

Lines 91–93 More than one rank-1 player: announce the tie and list the names.

```
95      private static string Ordinal(int rank) => (rank % 100, rank % 10) switch
96      {
```

Lines 95–96 1 → "1st", 2 → "2nd" … using a tuple switch on the last two digits and last digit.

```
97          (11 or 12 or 13, _) => $"{rank}th",
98          (_, 1) => $"{rank}st",
99          (_, 2) => $"{rank}nd",
100         (_, 3) => $"{rank}rd",
101         _ => $"{rank}th",
102     };
103 }
```

Lines 97–103 11th, 12th, 13th are exceptions; otherwise the last digit decides st/nd/rd/th.

11.7 High-score tables

`src/Yahtzee.WinForms/Forms/HighScoresForm.cs · 168 lines`

Requirement: access to the **five highest overall scores** and the **five highest scores of a particular player**, here with the date of each (the "could" item). All data comes from `IGameHistoryRepository`; the form knows nothing about SQL.

```
1  using System.Data.Common;
2  using Yahtzee.Core.Data;
3
```

Lines 1–2 `DbException` (base class of every database error, including SQLite's) and the repository types.

```
4  namespace Yahtzee.WinForms.Forms;
5
```

Line 4 Forms namespace.

```
6  /// <summary>
7  /// High-score tables from the games history: the five best scores overall, and the five best scores of
8  /// a chosen player, each with the date it was achieved.
9  /// </summary>
```

Lines 6–9 Documentation.

```
10 internal sealed class HighScoresForm : Form
11 {
```

Lines 10–11 A modal dialog.

```
12     private const int TableSize = 5;
13
```

Line 12 Five rows per table, as the specification asks.

```
14     private readonly IGameHistoryRepository _history;
15     private readonly ListView _overallList = CreateScoreList();
16     private readonly ListView _playerList = CreateScoreList();
```

Lines 14–16 The repository, and two identical score lists created by the helper below.

```
17     private readonly ComboBox _playerBox = new()
18     {
19         Width = 220,
20         DropDownStyle = ComboBoxStyle.DropDown, // pick a known name or type one
21         AutoCompleteMode = AutoCompleteMode.SuggestAppend,
22         AutoCompleteSource = AutoCompleteSource.ListItems,
23     };
```

Lines 17–23 Player chooser: an editable drop-down (`DropDown` style) filled with every known name, with auto-complete, so the user can pick or type a name.

```
24     private readonly Label _messageLabel = new() { AutoSize = true, ForeColor = Color.DimGray };
25
```

Line 24 Grey label for "no games yet" and error messages.

```
26     public HighScoresForm(IGameHistoryRepository history)
27     {
```

Lines 26–27 Constructor.

```
28         _history = history;
29
```

Line 28 Stores the repository.

```
30          SuspendLayout();
31          AutoScaleDimensions = new SizeF(96F, 96F);
32          AutoScaleMode = AutoScaleMode.Dpi;
33          Text = "High Scores";
34          Font = UiTheme.Body;
35          FormBorderStyle = FormBorderStyle.FixedDialog;
36          MaximizeBox = false;
37          MinimizeBox = false;
38          ShowInTaskbar = false;
39          StartPosition = FormStartPosition.CenterParent;
40          AutoSize = true;
41          AutoSizeMode = AutoSizeMode.GrowAndShrink;
42          Padding = new Padding(16);
43
```

Lines 30–42 The usual dialog setup: DPI scaling, title, font, fixed border, centred, auto-sized.

```
44          var layout = new TableLayoutPanel { AutoSize = true, ColumnCount = 1, Dock = DockStyle.Fill };
45          layout.Controls.Add(Heading("Top 5 scores (all players)"));
46          layout.Controls.Add(_overallList);
47          layout.Controls.Add(Heading("Top 5 scores of a player"));
48
```

Lines 44–47 Vertical layout: heading, overall table, heading for the per-player table.

```
49          var showButton = new Button { Text = "Show", AutoSize = true, MinimumSize = new Size(80, 30) };
50          showButton.Click += async (_, _) => await LoadPlayerScoresAsync();
```

Lines 49–50 The Show button loads the chosen player's scores. The lambda is `async` so the window stays responsive.

```
51          _playerBox.SelectedIndexChanged += async (_, _) => await LoadPlayerScoresAsync();
```

Line 51 Choosing a name from the list loads it immediately.

```
52          var playerRow = new FlowLayoutPanel { AutoSize = true, WrapContents = false, Margin = new Padding(0, 0, 0, 6) };
53          playerRow.Controls.AddRange([new Label { Text = "Player:", AutoSize = true, Margin = new Padding(3, 6, 3, 3) },
54              _playerBox, showButton]);
55          layout.Controls.Add(playerRow);
```

Lines 52–55 "Player:" label, name box and Show button side by side.

```
56          layout.Controls.Add(_playerList);
57          layout.Controls.Add(_messageLabel);
58
```

Lines 56–57 The per-player table and the message label.

```
59          var close = new Button
60          {
61              Text = "Close",
62              DialogResult = DialogResult.OK,
63              AutoSize = true,
64              MinimumSize = new Size(90, 32),
65              Anchor = AnchorStyles.Right,
66              Margin = new Padding(3, 12, 3, 3),
67          };
68          layout.Controls.Add(close);
69
```

Lines 59–68 Close button, right-aligned.

```
70          Controls.Add(layout);
71          AcceptButton = showButton; // Enter in the name box shows that player's scores
72          CancelButton = close;
73          ResumeLayout(false);
74          PerformLayout();
75
```

Lines 70–74 Enter (while typing a name) presses Show; Esc closes.

```
76          Load += async (_, _) =>
77          {
78              SizeColumns(_overallList);
79              SizeColumns(_playerList);
80              await LoadOverallAsync();
81          };
82      }
83
```

Lines 76–82 When the form loads: size the columns for the real DPI-scaled width, then fetch the data asynchronously.

```
84      private static Label Heading(string text) =>
85          new() { Text = text, Font = UiTheme.Bold, AutoSize = true, Margin = new Padding(3, 10, 3, 4) };
86
```

Lines 84–85 Helper for the bold section headings.

```
87      private static ListView CreateScoreList()
88      {
89          var list = new ListView
90          {
91              View = View.Details,
92              FullRowSelect = true,
93              HeaderStyle = ColumnHeaderStyle.Nonclickable,
94              Size = new Size(520, 40 + 26 * TableSize),
95              MultiSelect = false,
96          };
```

Lines 87–96 Helper creating a details-view list tall enough for five rows.

```
97          list.Columns.Add("#");
98          list.Columns.Add("Player");
99          list.Columns.Add("Score", -2, HorizontalAlignment.Right);
100         list.Columns.Add("Date", -2);
101         return list;
102     }
103
```

Lines 97–102 Columns: rank, player, score (right-aligned), date.

```
104     /// <summary>Pixel column widths are not DPI-scaled, so share out the list's real width once it is known.</summary>
105     private static void SizeColumns(ListView list)
106     {
107         int width = list.ClientSize.Width;
```

Lines 104–107 Same DPI technique as `ResultsForm`: share the real width out as percentages; the date takes the rest.

```
108         int[] percent = [8, 42, 16];
109         for (int i = 0; i < percent.Length; i++)
110             list.Columns[i].Width = width * percent[i] / 100;
111         list.Columns[3].Width = width - list.Columns.Cast<ColumnHeader>().Take(3).Sum(column => column.Width);
112     }
113
```

Lines 108–112 8% / 42% / 16% for the first three columns, computed with integer arithmetic.

```
114     private async Task LoadOverallAsync()
115     {
116         try
117         {
```

Lines 114–117 Fills the player chooser and the overall table.

```
118             var names = await _history.GetPlayerNamesAsync();
119             _playerBox.Items.Clear();
120             _playerBox.Items.AddRange([.. names]);
121
```

Lines 118–120 Every name in the history, for the drop-down.

```
122             var scores = await _history.GetTopScoresAsync(TableSize);
123             Fill(_overallList, scores);
124             _messageLabel.Text = scores.Count == 0
125                 ? "No completed games yet. Finish a game and its scores will appear here."
126                 : "";
127         }
```

Lines 122–127 Top five overall; if there are none, explain how to get some.

```
128         catch (Exception ex) when (ex is DbException or IOException or UnauthorizedAccessException)
129         {
130             _messageLabel.Text = $"The games history could not be read: {ex.Message}";
131         }
132     }
133
```

Lines 128–132 An **exception filter** (`when`) catches only database and file errors; they are shown in the message label instead of crashing the program.

```
134     private async Task LoadPlayerScoresAsync()
135     {
```

Lines 134–135 Loads the per-player table.

```
136         string name = _playerBox.Text.Trim();
137         if (name.Length == 0)
138             return;
139         try
140         {
```

Lines 136–140 Uses whatever is in the box (picked or typed); an empty box does nothing.

```
141             var scores = await _history.GetTopScoresForPlayerAsync(name, TableSize);
142             Fill(_playerList, scores);
143             _messageLabel.Text = scores.Count == 0 ? $"No completed games found for \"{name}\"." : "";
144         }
```

Lines 141–144 Top five for that name (case-insensitive in the database); a message if none were found.

```
145         catch (Exception ex) when (ex is DbException or IOException or UnauthorizedAccessException)
146         {
147             _messageLabel.Text = $"The games history could not be read: {ex.Message}";
148         }
149     }
150
```

Lines 145–149 Same error handling as above.

```
151     private static void Fill(ListView list, IReadOnlyList<HighScore> scores)
152     {
```

Lines 151–152 Fills a list with score rows.

```
153         list.BeginUpdate();
154         list.Items.Clear();
```

Lines 153–154 `BeginUpdate` / `EndUpdate` stop the list repainting after every row.

```
155         for (int i = 0; i < scores.Count; i++)
156         {
157             var score = scores[i];
158             list.Items.Add(new ListViewItem(
159             [
160                 (i + 1).ToString(),
161                 score.IsAi ? $"{score.PlayerName} (AI)" : score.PlayerName,
162                 score.Score.ToString(),
163                 score.PlayedAt.ToLocalTime().ToString("d MMM yyyy, HH:mm"),
164             ]));
165         }
```

Lines 155–165 One row per score: position, name (with "(AI)"), score, and the date converted from UTC to the computer's local time.

```
166         list.EndUpdate();
167     }
168 }
```

Lines 166–168 Repaint once at the end.

11.8 The main game window

`src/Yahtzee.WinForms/Forms/MainForm.cs · 764 lines`

Builds the window, forwards user actions to the engine, redraws everything from engine state, and handles saving/loading, the unsaved-changes prompt and recording finished games. It contains **no game rules**.

```
1  using System.Data.Common;
2  using Yahtzee.Core.AI;
3  using Yahtzee.Core.Data;
4  using Yahtzee.Core.Game;
5  using Yahtzee.Core.Model;
6  using Yahtzee.Core.Persistence;
7  using Yahtzee.Core.Rules;
8  using Yahtzee.WinForms.Controls;
9
```

Lines 1–8 Imports: database errors, the Core namespaces (AI catalog, history, engine, model, save files, rules) and the custom die control.

```
10 namespace Yahtzee.WinForms.Forms;
11
```

Line 10 Forms namespace.

```
12 /// <summary>
13 /// The main game window. The left side is the dice table (whose turn, rolls left, dice, roll button);
14 /// the right side is the scorecard grid with a "This roll" score-preview column for the current player.
15 /// </summary>
```

Lines 12–15 Class summary: dice table on the left, scorecard with preview column on the right.

```
16 /// <remarks>
17 /// The form holds no game rules of its own: it calls <see cref="GameEngine"/> actions and redraws itself
18 /// from the engine's state whenever <see cref="GameEngine.StateChanged"/> fires. It also handles the
19 /// "outside the game" features: saving and loading, the unsaved-changes prompt, and the games history.
20 /// </remarks>
```

Lines 16–20 Design principle: the form only calls engine actions and redraws on `StateChanged`; it also owns the features outside the game itself.

```
21 internal sealed class MainForm : Form
22 {
```

Lines 21–22 The main window class.

```
23     /// <summary>Delay between visible steps of a computer player's turn.</summary>
24     private const int AiStepDelayMs = 800;
25
```

Lines 23–24 AI players perform one action every 800 ms so humans can follow what they do.

```
26     private const int CategoryColumn = 0;
27     private const int PreviewColumn = 1;
28     private const int FirstPlayerColumn = 2;
29
```

Lines 26–28 Named indexes of the grid columns: 0 = category name, 1 = "This roll" preview, 2+ = one column per player.

```
30     private const string SaveFileFilter = "Yahtzee saved games (*.yahtzee)|*.yahtzee|All files (*.*)|*.*";
31
```

Line 30 File-type filter for the Open/Save dialogs: "description|pattern" pairs separated by `|`.

```
32     private readonly DieControl[] _dice = new DieControl[Scorer.DiceCount];
```

Line 32 The five die controls.

```
33      private readonly Label _turnLabel = new();
34      private readonly Label _roundLabel = new();
35      private readonly Label _rollsLabel = new();
36      private readonly Label _hintLabel = new();
37      private readonly Button _rollButton = new();
38      private readonly Button _newGameButton = new();
39      private readonly Button _openButton = new();
```

Lines 33–39 Labels for turn, round, rolls left and hints, and the three buttons.

```
40      private readonly DataGridView _scoreGrid = new();
41      private readonly ToolStripStatusLabel _statusLabel = new();
```

Lines 40–41 The scorecard grid and the status-bar label.

```
42      private readonly System.Windows.Forms.Timer _aiTimer = new() { Interval = AiStepDelayMs };
```

Line 42 A UI-thread timer for AI steps. The full name is needed because `System.Threading.Timer` also exists.

```
43      private readonly ToolStripMenuItem _saveItem = new("&Save", null, null, Keys.Control | Keys.S);
44      private readonly ToolStripMenuItem _saveAsItem = new("Save &As...");
```

Lines 43–44 The Save (Ctrl+S) and Save As menu items are fields because they are enabled/disabled as the game changes.

```
45      private readonly IGameHistoryRepository _history;
46
```

Line 45 The games history, received from `Program`.

```
47      private GameEngine? _game;
48
```

Line 47 The current game, or null before the first game starts.

```
49      /// <summary>File the current game was last saved to or loaded from, or null if it has never been saved.</summary>
50      private string? _savePath;
51
```

Lines 49–50 Where Save writes without asking; null means "never saved", so Save behaves like Save As.

```
52      /// <summary>
53      /// Unsaved-changes tracking: <see cref="_changeCount"/> goes up on every engine change, and
54      /// <see cref="_savedChangeCount"/> remembers its value at the last save. A counter (rather than a
55      /// true/false flag) stays correct even if an AI move happens while a save is being written.
56      /// </summary>
57      private int _changeCount;
58      private int _savedChangeCount;
59
```

Lines 52–58 **Unsaved-changes tracking.** Every engine change increments `_changeCount`; a save records the count it wrote in `_savedChangeCount`. The game is unsaved when they differ. A counter is used instead of a true/false flag because saving is asynchronous: if an AI move happens while the file is being written, the counts still differ afterwards, so the game is correctly still unsaved.

```
60      /// <summary>Number of dialogs currently open; computer players wait while any are showing.</summary>
61      private int _modalDepth;
62
```

Lines 60–61 While this is above zero the AI timer is held, so the computer never plays behind a dialog.

```
63      /// <summary>Set once the user has dealt with unsaved changes, so the next close really closes.</summary>
64      private bool _closeConfirmed;
65
```

Lines 63–64 Used by the closing logic further down.

```
66      /// <param name="history">Where completed games are recorded and high scores are read from.</param>
67      public MainForm(IGameHistoryRepository history)
68      {
```

Lines 66–68 Constructor; the history object is injected.

```
69          _history = history;
```

Line 69 Stores it.

```
70          SuspendLayout();
71          AutoScaleDimensions = new SizeF(96F, 96F);
72          AutoScaleMode = AutoScaleMode.Dpi;
```

Lines 70–72 Pause layout and enable DPI scaling.

```
73          Text = "Yahtzee";
74          Font = UiTheme.Body;
75          StartPosition = FormStartPosition.CenterScreen;
76          Size = new Size(1180, 760);
77          MinimumSize = new Size(1000, 700);
78
```

Lines 73–77 Title, font, centred start position, default and minimum window size.

```
79          var mainLayout = new TableLayoutPanel { Dock = DockStyle.Fill, ColumnCount = 2, RowCount = 1 };
80          mainLayout.ColumnStyles.Add(new ColumnStyle(SizeType.AutoSize));
81          mainLayout.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 100));
82          mainLayout.Controls.Add(BuildDiceTable(), 0, 0);
83          mainLayout.Controls.Add(BuildScoreGrid(), 1, 0);
84
```

Lines 79–83 Two columns: the dice table sizes to its content; the scorecard takes the remaining width.

```
85          // Dock order: the fill control must be added before the edge-docked menu and status bar.
86          Controls.Add(mainLayout);
87          Controls.Add(BuildStatusBar());
88          Controls.Add(BuildMenu());
89
```

Lines 85–88 WinForms docks controls in reverse order of adding: adding the fill panel first, then the status bar and menu, gives a menu at the top, status bar at the bottom and the game area in between.

```
90          _aiTimer.Tick += (_, _) => PlayAiStep();
91
```

Line 90 Each timer tick performs one AI step.

```
92          ResumeLayout(false);
93          PerformLayout();
94          RefreshUi();
95      }
96
```

Lines 92–95 Resume layout and draw the initial (welcome) state.

```
97      // ------------------------------------------------------------------
98      // Layout
99      // ------------------------------------------------------------------
100
```

Lines 97–99 Section: building the layout.

```
101     private MenuStrip BuildMenu()
102     {
```

Lines 101–102 Creates the menu bar.

```
103         _saveItem.Click += async (_, _) => await SaveGameAsync(saveAs: false);
104         _saveAsItem.Click += async (_, _) => await SaveGameAsync(saveAs: true);
105
```

Lines 103–104 Save and Save As call the same method. The handlers are `async` lambdas, so the UI stays responsive while the file is written.

```
106         var gameMenu = new ToolStripMenuItem("&Game");
107         gameMenu.DropDownItems.AddRange(
108         [
109             new ToolStripMenuItem("&New Game...", null, async (_, _) => await NewGameAsync(), Keys.Control | Keys.N),
110             new ToolStripMenuItem("&Open Saved Game...", null, async (_, _) => await OpenGameAsync(), Keys.Control | Keys.O),
111             _saveItem,
112             _saveAsItem,
113             new ToolStripSeparator(),
114             new ToolStripMenuItem("&High Scores...", null, (_, _) => ShowHighScores(), Keys.Control | Keys.H),
115             new ToolStripSeparator(),
116             new ToolStripMenuItem("E&xit", null, (_, _) => Close(), Keys.Alt | Keys.F4),
117         ]);
118
```

Lines 106–117 Game menu, built with a collection expression: New Game (Ctrl+N), Open Saved Game (Ctrl+O), Save (Ctrl+S), Save As, High Scores (Ctrl+H), Exit (Alt+F4). The `&` marks the Alt-key mnemonic letter.

```
119         var helpMenu = new ToolStripMenuItem("&Help");
120         helpMenu.DropDownItems.Add(new ToolStripMenuItem("&How to Play", null, (_, _) => ShowHowToPlay(), Keys.F1));
121
```

Lines 119–120 Help → How to Play (F1).

```
122         var menu = new MenuStrip();
123         menu.Items.AddRange([gameMenu, helpMenu]);
124         MainMenuStrip = menu;
125         return menu;
126     }
127
```

Lines 122–126 Adds both menus and registers the strip as the form's main menu.

```
128     private StatusStrip BuildStatusBar()
129     {
130         var status = new StatusStrip();
131         _statusLabel.Spring = true;
132         _statusLabel.TextAlign = ContentAlignment.MiddleLeft;
133         status.Items.Add(_statusLabel);
134         return status;
135     }
136
```

Lines 128–135 A status bar whose single label stretches (`Spring`) to show messages such as "Ann scored 25 in Full House."

```
137     private Control BuildDiceTable()
138     {
```

Lines 137–138 Builds the green left-hand panel.

```
139         var panel = new FlowLayoutPanel
140         {
141             Dock = DockStyle.Fill,
142             FlowDirection = FlowDirection.TopDown,
143             WrapContents = false,
144             AutoSize = true,
145             BackColor = UiTheme.Felt,
146             Padding = new Padding(24),
147             Margin = new Padding(0),
148         };
149
```

Lines 139–148 Controls stacked top-to-bottom on the felt colour.

```
150         StyleFeltLabel(_turnLabel, UiTheme.Heading);
151         StyleFeltLabel(_roundLabel, UiTheme.Subheading);
152         StyleFeltLabel(_rollsLabel, new Font(UiTheme.Subheading, FontStyle.Bold));
153         _rollsLabel.ForeColor = UiTheme.HeldFace;
154         _rollsLabel.Margin = new Padding(3, 3, 3, 16);
155
```

Lines 150–154 Styles the turn, round and rolls labels; "Rolls left" is bold gold so it is prominent.

```
156         var diceRow = new FlowLayoutPanel { AutoSize = true, WrapContents = false, BackColor = Color.Transparent };
```

Line 156 A horizontal row for the five dice.

```
157         for (int i = 0; i < _dice.Length; i++)
158         {
```

Lines 157–158 Creates the five dice.

```
159             int index = i; // captured by the click handler
```

Line 159 Copies the loop variable so each click handler captures its own index (a classic closure pitfall avoided).

```
160             var die = new DieControl { AccessibleName = $"Die {i + 1}" };
161             die.Click += (_, _) => ToggleDie(index);
162             _dice[i] = die;
163             diceRow.Controls.Add(die);
164         }
165
```

Lines 160–164 Creates the die with an accessible name ("Die 1" …) for screen readers, wires its click to `ToggleDie`, and adds it.

```
166         _rollButton.Size = new Size(220, 48);
167         _rollButton.Font = UiTheme.Subheading;
168         _rollButton.Margin = new Padding(3, 16, 3, 8);
169         _rollButton.BackColor = Color.WhiteSmoke;
170         _rollButton.Click += (_, _) => RollDice();
171
```

Lines 166–170 The roll button: size, font, spacing, and click → `RollDice`.

```
172         StyleFeltLabel(_hintLabel, UiTheme.Body);
173         _hintLabel.MaximumSize = new Size(440, 0);
174
```

Lines 172–173 The hint text wraps at 440 px.

```
175         _newGameButton.Text = "Start a New Game";
176         _newGameButton.Size = new Size(220, 48);
177         _newGameButton.Font = UiTheme.Subheading;
178         _newGameButton.BackColor = Color.WhiteSmoke;
179         _newGameButton.Margin = new Padding(3, 16, 3, 3);
180         _newGameButton.Click += async (_, _) => await NewGameAsync();
181
```

Lines 175–180 "Start a New Game" button, shown before the first game and after a game ends.

```
182         _openButton.Text = "Open a Saved Game";
183         _openButton.Size = new Size(220, 48);
184         _openButton.Font = UiTheme.Subheading;
185         _openButton.BackColor = Color.WhiteSmoke;
186         _openButton.Margin = new Padding(3, 8, 3, 3);
187         _openButton.Click += async (_, _) => await OpenGameAsync();
188
```

Lines 182–187 "Open a Saved Game" button, shown at the same times.

```
189         panel.Controls.AddRange([_turnLabel, _roundLabel, _rollsLabel, diceRow, _rollButton, _hintLabel, _newGameButton,
_openButton]);
190         return panel;
191     }
192
```

Lines 189–191 Adds all controls to the panel in display order.

```
193     private static void StyleFeltLabel(Label label, Font font)
194     {
195         label.AutoSize = true;
196         label.Font = font;
197         label.ForeColor = UiTheme.FeltText;
198         label.BackColor = Color.Transparent;
199     }
200
```

Lines 193–199 Helper: auto-sized label with the given font, light text, transparent background.

```
201     private Control BuildScoreGrid()
202     {
203         var grid = _scoreGrid;
```

Lines 201–203 Configures the scorecard grid.

```
204         grid.Dock = DockStyle.Fill;
205         grid.Margin = new Padding(12);
```

Lines 204–205 Fills its table cell, with a margin.

```
206         grid.ReadOnly = true;
207         grid.AllowUserToAddRows = false;
208         grid.AllowUserToDeleteRows = false;
209         grid.AllowUserToResizeRows = false;
210         grid.AllowUserToResizeColumns = false;
211         grid.AllowUserToOrderColumns = false;
```

Lines 206–211 A display only: no editing, adding, deleting, resizing or reordering by the user.

```
212         grid.RowHeadersVisible = false;
213         grid.MultiSelect = false;
214         grid.TabStop = false;
215         grid.BackgroundColor = SystemColors.Window;
216         grid.BorderStyle = BorderStyle.None;
```

Lines 212–216 No row-header column, single selection, not a tab stop, plain white background, no border.

```
217         grid.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;
218         grid.ColumnHeadersHeightSizeMode = DataGridViewColumnHeadersHeightSizeMode.DisableResizing;
```

Lines 217–218 Columns share the width by `FillWeight`; header height is fixed.

```
219         grid.EnableHeadersVisualStyles = false; // lets us colour the current player's header
220         grid.ColumnHeadersDefaultCellStyle.BackColor = UiTheme.HeaderBack;
221         grid.ColumnHeadersDefaultCellStyle.Font = UiTheme.Bold;
222         grid.ColumnHeadersDefaultCellStyle.Alignment = DataGridViewContentAlignment.MiddleCenter;
```

Lines 219–222 Turning off visual styles is required to colour individual column headers (the current player's header turns gold).

```
223         grid.DefaultCellStyle.Alignment = DataGridViewContentAlignment.MiddleCenter;
224         grid.ShowCellToolTips = true;
225
```

Lines 223–224 Centred cells; tooltips enabled (category rules and "Click to score…").

```
226         // The grid is a display, not a selectable table: hide the selection highlight.
227         grid.SelectionChanged += (_, _) => grid.ClearSelection();
```

Lines 226–227 Selection highlighting would look like a bug, so any selection is cleared immediately.

```
228         grid.CellClick += OnScoreCellClick;
```

Line 228 Clicking a cell may score a category.

```
229         grid.CellMouseEnter += (_, e) => grid.Cursor = IsScorableCell(e.RowIndex, e.ColumnIndex) ? Cursors.Hand :
Cursors.Default;
230         grid.CellMouseLeave += (_, _) => grid.Cursor = Cursors.Default;
231
```

Lines 229–230 Shows a hand cursor over clickable preview cells only.

```
232         return grid;
233     }
234
```

Lines 232–233 Returns the configured grid.

```
235     /// <summary>Creates the grid's columns (one per player) and rows (one per category plus a total).</summary>
236     private void BuildScoreGridFor(GameEngine game)
237     {
```

Lines 235–237 `BuildScoreGridFor`: called for each new or loaded game because the number of players varies.

```
238         var grid = _scoreGrid;
239         grid.Columns.Clear();
240         grid.Rows.Clear();
241
```

Lines 238–240 Removes the previous game's columns and rows.

```
242         // Pixel heights are not scaled automatically, so convert them for the monitor's DPI.
243         grid.ColumnHeadersHeight = LogicalToDeviceUnits(40);
244         grid.RowTemplate.Height = LogicalToDeviceUnits(32);
245
```

Lines 242–244 Row and header heights are raw pixels, which automatic scaling does not touch. `LogicalToDeviceUnits` converts 40/32 logical pixels for the monitor's DPI. This was a real bug found in testing at 150% scaling.

```
246         grid.Columns.Add(new DataGridViewTextBoxColumn
247         {
248             HeaderText = "Category",
249             FillWeight = 160,
250             SortMode = DataGridViewColumnSortMode.NotSortable,
251             DefaultCellStyle = { Alignment = DataGridViewContentAlignment.MiddleLeft, Font = UiTheme.Bold, Padding = new
Padding(8, 0, 0, 0) },
252         });
```

Lines 246–252 Category column: wider, left-aligned bold text. The nested object initialiser `DefaultCellStyle = { … }` sets properties on the existing style object.

```
253         grid.Columns.Add(new DataGridViewTextBoxColumn
254         {
255             HeaderText = "This roll",
256             FillWeight = 90,
257             SortMode = DataGridViewColumnSortMode.NotSortable,
258             DefaultCellStyle = { BackColor = UiTheme.PreviewBack },
259         });
```

Lines 253–259 "This roll" preview column with a light blue background.

```
260         foreach (var player in game.Players)
261         {
262             grid.Columns.Add(new DataGridViewTextBoxColumn
263             {
264                 HeaderText = player.IsAi ? $"{player.Name} (AI)" : player.Name,
265                 FillWeight = 100,
266                 SortMode = DataGridViewColumnSortMode.NotSortable,
267             });
268         }
269
```

Lines 260–268 One column per player, headed by the name (with "(AI)" for computer players).

```
270         foreach (var category in CategoryExtensions.All)
271         {
```

Lines 270–271 One row per category.

```
272             int row = grid.Rows.Add();
273             grid.Rows[row].Tag = category;
```

Lines 272–273 The row's `Tag` stores its `Category`, so a click can be mapped back to a category.

```
274             var nameCell = grid.Rows[row].Cells[CategoryColumn];
275             nameCell.Value = category.DisplayName();
276             nameCell.ToolTipText = DescribeCategory(category);
277         }
278
```

Lines 274–277 The first cell shows the display name, with the rule as a tooltip.

```
279         int totalRow = grid.Rows.Add();
280         grid.Rows[totalRow].Cells[CategoryColumn].Value = "TOTAL";
281         grid.Rows[totalRow].DefaultCellStyle.BackColor = UiTheme.TotalBack;
282         grid.Rows[totalRow].DefaultCellStyle.Font = UiTheme.Bold;
283         grid.Rows[totalRow].Cells[PreviewColumn].Style.BackColor = UiTheme.TotalBack;
284     }
285
```

Lines 279–284 The final TOTAL row (no Tag), in bold on grey.

```
286     // ---------------------------------------------------------------
287     // Game lifecycle
288     // ---------------------------------------------------------------
289
```

Lines 286–288 Section: game lifecycle.

```
290     private bool IsGameInProgress => _game is { IsGameOver: false };
291
```

Line 290 **Property pattern**: true when a game exists and is not over.

```
292     private bool IsHumanTurn => _game is { IsGameOver: false, CurrentPlayer.IsAi: false };
293
```

Line 292 Nested property pattern `CurrentPlayer.IsAi: false`: a human is on turn in a running game.

```
294     /// <summary>A game is running and has changed since it was last saved (or was never saved).</summary>
295     private bool HasUnsavedChanges => IsGameInProgress && _changeCount != _savedChangeCount;
296
```

Lines 294–295 `HasUnsavedChanges`: a running game whose change counter moved since the last save. A finished game never counts as unsaved (it is already in the history).

```
297     private async Task NewGameAsync()
298     {
```

Lines 297–298 Starts a new game (menu, Ctrl+N, or button).

```
299         if (!await ConfirmUnsavedChangesAsync("starting a new game"))
300             return;
301
```

Lines 299–300 First deal with any unsaved changes; the user may cancel.

```
302         using var dialog = new NewGameDialog();
303         if (ShowModal(() => dialog.ShowDialog(this)) != DialogResult.OK)
304             return;
305
```

Lines 302–304 Shows the dialog (through `ShowModal`, which pauses the AI); Cancel aborts. `using` disposes the dialog afterwards.

```
306         StartGame(new GameEngine(dialog.Players, new RandomDiceRoller()), savePath: null);
307         SetStatus($"New game started with {dialog.Players.Count} player(s). {_game!.CurrentPlayer.Name} goes first.");
308     }
309
```

Lines 306–308 Creates the engine with real random dice and starts it; the status bar announces who goes first.

```
310     /// <summary>Makes <paramref name="game"/> the current game, replacing any previous one.</summary>
311     /// <param name="game">The new or loaded game.</param>
312     /// <param name="savePath">The file it was loaded from, or null for a new game.</param>
313     private void StartGame(GameEngine game, string? savePath)
314     {
```

Lines 310–314 `StartGame`: switches the window to a new or loaded engine.

```
315         if (_game is not null)
316         {
317             _game.StateChanged -= OnGameStateChanged;
318             _game.DiceRolled -= OnDiceRolled;
319             _game.CategoryScored -= OnCategoryScored;
320             _game.GameOver -= OnGameOver;
321         }
322
```

Lines 315–321 Unsubscribes from the old engine's events so it can be garbage-collected and cannot affect the UI any more.

```
323         _game = game;
324         game.StateChanged += OnGameStateChanged;
325         game.DiceRolled += OnDiceRolled;
326         game.CategoryScored += OnCategoryScored;
327         game.GameOver += OnGameOver;
```

Lines 323–327 Subscribes to the new engine's events.

```
328         _savePath = savePath;
329         _changeCount = _savedChangeCount = 0; // a new or just-loaded game has nothing unsaved
330
```

Lines 328–329 Remembers the file (null for a new game) and resets both counters: a new or just-loaded game has nothing unsaved.

```
331         BuildScoreGridFor(game);
332         RefreshUi();
333         _rollButton.Focus();
334     }
335
```

Lines 331–334 Builds the grid for these players, draws everything, and focuses the Roll button.

```
336     // ---------------------------------------------------------------
337     // Saving, loading and the unsaved-changes prompt
338     // ---------------------------------------------------------------
339
```

Lines 336–338 Section: saving, loading and the unsaved-changes prompt (requirements: save after any roll; offer to save before exiting).

```
340     /// <summary>
341     /// If the game has unsaved changes, asks whether to save before <paramref name="action"/>.
342     /// Returns true when it is fine to continue: nothing to save, saved successfully, or "No" (don't save).
343     /// </summary>
```

Lines 340–343 Documentation of `ConfirmUnsavedChangesAsync`.

```
344     private async Task<bool> ConfirmUnsavedChangesAsync(string action)
345     {
```

Lines 344–345 Returns `Task<bool>`: true = carry on, false = the user cancelled.

```
346         if (!HasUnsavedChanges)
347             return true;
348
```

Lines 346–347 Nothing unsaved → no question at all. This is what makes the prompt appear *only* when needed.

```
349         var answer = ShowModal(() => MessageBox.Show(this,
350             $"The current game has unsaved changes.\n\nDo you want to save it before {action}?",
351             "Save Game?", MessageBoxButtons.YesNoCancel, MessageBoxIcon.Question));
```

Lines 349–351 Yes / No / Cancel question naming the action ("exiting", "starting a new game"…).

```
352         return answer switch
353         {
354             DialogResult.Yes => await SaveGameAsync(saveAs: false),
355             DialogResult.No => true,
356             _ => false, // Cancel, or the message box was closed
357         };
358     }
359
```

Lines 352–358 Yes → save (and continue only if the save succeeded); No → continue without saving; Cancel → stop.

```
360     /// <summary>
361     /// Saves the current game. Uses the file it was last saved to unless <paramref name="saveAs"/> is set
362     /// or it has never been saved, in which case the user picks a file.
363     /// </summary>
364     /// <returns>True if the game was saved.</returns>
```

Lines 360–364 Documentation of `SaveGameAsync`.

```
365     private async Task<bool> SaveGameAsync(bool saveAs)
366     {
```

Lines 365–366 Returns true if the game was saved.

```
367         if (_game is not { CanSave: true } game)
368             return false;
369
```

Lines 367–368 A negated property pattern that also declares `game`: nothing to save (no game, or it is over) → false.

```
370         string? path = saveAs ? null : _savePath;
371         if (path is null)
372         {
```

Lines 370–372 Save reuses the known file; Save As (or a never-saved game) asks for one.

```
373             using var dialog = new SaveFileDialog
374             {
375                 Title = "Save Game",
376                 Filter = SaveFileFilter,
377                 DefaultExt = SaveGameSerializer.FileExtension,
378                 AddExtension = true,
379                 FileName = _savePath is null ? $"Yahtzee {DateTime.Now:yyyy-MM-dd HH-mm}" : Path.GetFileName(_savePath),
380             };
381             if (ShowModal(() => dialog.ShowDialog(this)) != DialogResult.OK)
382                 return false;
383             path = dialog.FileName;
384         }
385
```

Lines 373–384 Standard Windows save dialog with the .yahtzee filter and a suggested name such as "Yahtzee 2026-10-08 17-14"; cancelling returns false.

```
386         var snapshot = game.CreateSnapshot();
387         int changesSaved = _changeCount; // remember which state was written, in case play continues meanwhile
```

Lines 386–387 Takes the snapshot *now* and remembers which change count it represents.

```
388         try
389         {
390             await SaveGameSerializer.SaveAsync(snapshot, path);
391         }
```

Lines 388–391 Writes the file asynchronously (the UI stays responsive).

```
392         catch (Exception ex) when (ex is IOException or UnauthorizedAccessException)
393         {
394             ShowError($"The game could not be saved.\n\n{ex.Message}");
395             return false;
396         }
397
```

Lines 392–396 Disk full, read-only folder, no permission…: explain and report failure.

```
398         _savePath = path;
399         _savedChangeCount = changesSaved;
400         SetStatus($"Game saved to {path}.");
401         UpdateTitleAndMenus();
402         return true;
403     }
404
```

Lines 398–403 Success: remember the file, mark that state as saved, update status, title and menus.

```
405     private async Task OpenGameAsync()
406     {
```

Lines 405–406 Opens a saved game (menu, Ctrl+O, or button).

```
407         if (!await ConfirmUnsavedChangesAsync("opening another game"))
408             return;
409 
```

Lines 407–408 Protect the current game first.

```
410         using var dialog = new OpenFileDialog { Title = "Open Saved Game", Filter = SaveFileFilter };
411         if (ShowModal(() => dialog.ShowDialog(this)) != DialogResult.OK)
412             return;
413 
```

Lines 410–412 Standard open dialog with the same filter.

```
414         GameEngine game;
415         try
416         {
```

Lines 414–416 Declared outside the `try` so it can be used after it.

```
417             var snapshot = await SaveGameSerializer.LoadAsync(dialog.FileName);
```

Line 417 Read and parse the file...

```
418             game = GameEngine.Restore(snapshot, new RandomDiceRoller(), StrategyCatalog.Create);
419         }
```

Lines 418–419 ...then rebuild the game, validating it. `StrategyCatalog.Create` is passed as a **method group** (a delegate) to turn saved strategy names back into AI objects.

```
420         catch (Exception ex) when (ex is InvalidSaveFileException or IOException or UnauthorizedAccessException)
421         {
422             ShowError($"The game could not be opened.\n\n{ex.Message}");
423             return;
424         }
425 
```

Lines 420–424 A damaged, impossible or unreadable file gives a clear message and leaves the current game untouched.

```
426         StartGame(game, dialog.FileName);
427         SetStatus($"Loaded {Path.GetFileName(dialog.FileName)}. It is {game.CurrentPlayer.Name}'s turn.");
428     }
429 
```

Lines 426–428 Switch to the loaded game; AI players continue automatically if it is their turn.

```
430     /// <summary>Offers to save unsaved changes before the window closes.</summary>
431     protected override void OnFormClosing(FormClosingEventArgs e)
432     {
```

Lines 430–432 `OnFormClosing` runs whenever the window is about to close (Exit, Alt+F4, the X button).

```
433         base.OnFormClosing(e);
```

Line 433 Let the base class raise the normal event first.

```
434         if (_closeConfirmed || !HasUnsavedChanges || e.CloseReason == CloseReason.WindowsShutDown)
435             return;
436 
```

Lines 434–435 Close straight away if the user already answered, there is nothing unsaved, or Windows is shutting down.

```
437         // Saving is asynchronous but this event must decide immediately, so cancel this close,
438         // ask (and maybe save) afterwards, and then close again unless the user cancelled.
439         e.Cancel = true;
```

Lines 437–439 Otherwise **cancel this close**: the event must decide synchronously, but saving is asynchronous.

```
440         BeginInvoke(async () =>
441         {
442             if (await ConfirmUnsavedChangesAsync("exiting"))
443             {
444                 _closeConfirmed = true;
445                 Close();
446             }
447         });
448     }
449
```

Lines 440–448 Ask (and possibly save) just after the event finishes; if the user did not cancel, set the flag and close again, which now passes straight through.

```
450     private void ShowHighScores()
451     {
452         using var form = new HighScoresForm(_history);
453         ShowModal(() => form.ShowDialog(this));
454     }
455
```

Lines 450–454 Opens the high-score window (Ctrl+H).

```
456     // ------------------------------------------------------------------
457     // Player actions
458     // ------------------------------------------------------------------
459
```

Lines 456–458 Section: player actions. Each method checks the same conditions the engine enforces, so clicks at the wrong time are simply ignored.

```
460     private void RollDice()
461     {
462         if (IsHumanTurn && _game!.CanRoll)
463             _game.Roll();
464     }
465
```

Lines 460–464 Roll button / R key.

```
466     private void ToggleDie(int index)
467     {
468         if (IsHumanTurn && _game!.CanHold)
469             _game.ToggleHeld(index);
470     }
471
```

Lines 466–470 Die click / 1–5 keys.

```
472     private void OnScoreCellClick(object? sender, DataGridViewCellEventArgs e)
473     {
474         if (!IsScorableCell(e.RowIndex, e.ColumnIndex))
475             return;
476
```

Lines 472–475 Grid click: ignore anything that is not a scorable preview cell.

```
477         var category = (Category)_scoreGrid.Rows[e.RowIndex].Tag!;
478         int points = Scorer.Score(category, _game!.Dice);
```

Lines 477–478 Reads the category from the row's Tag and computes the points.

```
479         if (points == 0)
480         {
481             var answer = ShowModal(() => MessageBox.Show(this,
482                 $"Your dice don't fit {category.DisplayName()}, so it will score 0.\n\nScore 0 in {category.DisplayName()}?",
483                 "Score zero?", MessageBoxButtons.YesNo, MessageBoxIcon.Question, MessageBoxDefaultButton.Button2));
484             if (answer != DialogResult.Yes)
485                 return;
486         }
487
```

Lines 479–486 Scoring 0 is legal but is often a mis-click, so it asks for confirmation (default No).

```
488         _game.Score(category);
489     }
490
```

Lines 488–489 Records the score; the engine's events then redraw the window.

```
491     /// <summary>Whether clicking this grid cell would score a category for the current human player.</summary>
492     private bool IsScorableCell(int row, int column) =>
493         IsHumanTurn
494         && _game!.CanScore
495         && column == PreviewColumn
496         && row >= 0
497         && _scoreGrid.Rows[row].Tag is Category category
498         && !_game.CurrentPlayer.ScoreCard.IsFilled(category);
499
```

Lines 491–498 `IsScorableCell`: human turn, at least one roll, the preview column, a category row (`is Category category` pattern also extracts the value), and the box not yet filled.

```
500     /// <summary>Keyboard shortcuts: 1–5 hold or release a die, R rolls.</summary>
501     protected override bool ProcessCmdKey(ref Message msg, Keys keyData)
502     {
```

Lines 500–502 `ProcessCmdKey` sees key presses before the focused control does.

```
503         int? dieIndex = keyData switch
504         {
505             >= Keys.D1 and <= Keys.D5 => (int)(keyData - Keys.D1),
506             >= Keys.NumPad1 and <= Keys.NumPad5 => (int)(keyData - Keys.NumPad1),
507             _ => null,
508         };
509
```

Lines 503–508 Maps keys 1–5 (main row or numeric keypad) to die index 0–4 using relational patterns on the `Keys` enum.

```
510         if (dieIndex is int index && IsHumanTurn && _game!.CanHold)
511         {
512             ToggleDie(index);
513             return true;
514         }
```

Lines 510–514 If a die key was pressed and holding is allowed, toggle it and mark the key as handled.

```
515         if (keyData == Keys.R && _rollButton.Enabled)
516         {
517             RollDice();
518             return true;
519         }
```

Lines 515–519 R rolls when the Roll button is enabled.

```
520         return base.ProcessCmdKey(ref msg, keyData);
521     }
522
```

Lines 520–521 All other keys get normal processing (menus, Tab, ...).

```
523     // ---------------------------------------------------------------------
524     // Computer players
525     // ---------------------------------------------------------------------
526
```

Lines 523–525 Section: computer players.

```
527     /// <summary>
528     /// Starts or stops the AI timer depending on whether a computer player is on turn and no dialog is open.
529     /// </summary>
```

Lines 527–529 `UpdateAiTimer` documentation.

```
530     private void UpdateAiTimer()
531     {
```

Lines 530–531 Called after every redraw.

```
532         bool aiTurn = _game is { IsGameOver: false, CurrentPlayer.IsAi: true } && _modalDepth == 0;
533         if (aiTurn && !_aiTimer.Enabled)
534             _aiTimer.Start();
535         else if (!aiTurn)
536             _aiTimer.Stop();
537     }
538
```

Lines 532–537 Run the timer only while an AI is on turn in a running game *and* no dialog is open.

```
539     private void PlayAiStep()
540     {
```

Lines 539–540 Timer tick handler.

```
541         _aiTimer.Stop();
542         if (_game is not { IsGameOver: false, CurrentPlayer.IsAi: true })
543             return;
544
```

Lines 541–543 Stops the timer during the step and re-checks that an AI is still on turn.

```
545         try
546         {
547             _game.PlayAiStep(); // raises StateChanged, which restarts the timer if the AI is still on turn
548         }
```

Lines 545–548 One engine step; the resulting `StateChanged` → `RefreshUi` → `UpdateAiTimer` restarts the timer if the AI continues.

```
549         catch (InvalidOperationException ex)
550         {
551             ShowError($"The computer player made an invalid move:\n\n{ex.Message}");
552         }
553     }
554
```

Lines 549–553 A buggy strategy is reported in a message box instead of crashing the program.

```
555     // ------------------------------------------------------------------
556     // Engine events
557     // ------------------------------------------------------------------
558
```

Lines 555–557 Section: engine event handlers.

```
559     private void OnGameStateChanged(object? sender, EventArgs e)
560     {
561         _changeCount++;
562         RefreshUi();
563     }
564
```

Lines 559–563 Any change → count it (for unsaved-changes tracking) and redraw everything.

```
565     private void OnDiceRolled(object? sender, DiceRolledEventArgs e) =>
566         SetStatus($"{e.Player.Name} rolled {string.Join("  ", e.Dice)}.");
567
```

Lines 565–566 Status message with the rolled values.

```
568     private void OnCategoryScored(object? sender, CategoryScoredEventArgs e) =>
569         SetStatus($"{e.Player.Name} scored {e.Score} in {e.Category.DisplayName()}.");
570
```

Lines 568–569 Status message with the scored category and points.

```
571     /// <summary>Shows the results and records the game in the games history.</summary>
572     /// <remarks><c>async void</c> is acceptable here because this is an event handler and its errors are caught.</remarks>
573     private async void OnGameOver(object? sender, GameOverEventArgs e)
574     {
```

Lines 571–574 Documentation, including why `async void` is acceptable for an event handler.

```
575         // Show the results after the engine finishes raising events and the board has been redrawn.
576         BeginInvoke(() =>
577         {
578             using var results = new ResultsForm(e.Standings);
579             ShowModal(() => results.ShowDialog(this));
580         });
581
```

Lines 575–580 `BeginInvoke` queues the results dialog to run after the current event handling finishes, so the board shows the final scores behind the dialog.

```
582         try
583         {
```

Lines 582–583 Meanwhile, record the finished game in the database…

```
584             await _history.RecordGameAsync(new CompletedGame(e.CompletedAt, e.Standings));
585             SetStatus("Game over. The result has been added to the games history (Game > High Scores).");
586         }
```

Lines 584–586 …as a `CompletedGame` record (completion time + standings), then confirm in the status bar.

```
587         catch (Exception ex) when (ex is DbException or IOException or UnauthorizedAccessException)
588         {
589             SetStatus($"Game over, but the result could not be saved to the games history: {ex.Message}");
590         }
591     }
592
```

Lines 587–591 If the database cannot be written, the game still ends normally and the status bar explains the problem.

```
593     // ------------------------------------------------------------------
594     // Rendering
595     // ------------------------------------------------------------------
596
```

Lines 593–595 Section: rendering.

```
597     /// <summary>Redraws everything from the current game state.</summary>
598     private void RefreshUi()
599     {
```

Lines 597–599 `RefreshUi`: the single method that turns engine state into what is on screen.

```
600         if (_game is null)
601         {
602             ShowWelcome();
603             return;
604         }
605
```

Lines 600–604 No game yet → welcome screen.

```
606         var game = _game;
607         var player = game.CurrentPlayer;
608         bool humanTurn = IsHumanTurn;
609
```

Lines 606–608 Local shortcuts.

```
610         _turnLabel.Text = game.IsGameOver ? "Game over"
611             : player.IsAi ? $"{player.Name}'s turn (computer)"
612             : $"{player.Name}'s turn";
```

Lines 610–612 Requirement: **whose turn it is** (marked "(computer)" for AI).

```
613         _roundLabel.Text = game.IsGameOver
614             ? $"All {GameEngine.TotalRounds} rounds played"
615             : $"Round {game.Round} of {GameEngine.TotalRounds}";
```

Lines 613–615 "Round n of 13".

```
616         _rollsLabel.Text = game.IsGameOver ? "" : $"Rolls left: {game.RollsRemaining} of {GameEngine.MaxRollsPerTurn}";
617
```

Line 616 Requirement: **how many rolls are available**.

```
618         for (int i = 0; i < _dice.Length; i++)
619         {
620             var die = _dice[i];
```

Lines 618–620 Updates each die control.

```
621             die.Value = game.RollsUsed > 0 ? game.Dice[i] : 0;
622             die.Held = game.RollsUsed > 0 && game.Held[i];
623             die.ShowRollCaption = game.CanHold;
624             die.Enabled = humanTurn && game.CanHold;
625         }
626
```

Lines 621–625 Requirement: **which dice will be rolled and which are set aside**. Value (0 shows "?" before the first roll), held state, the WILL ROLL caption while rolls remain, and clickability only for a human who may hold.

```
627         int diceToRoll = game.RollsUsed == 0 ? Scorer.DiceCount : game.Held.Count(held => !held);
```

Line 627 How many dice the next roll will change (all five on the first roll).

```
628         _rollButton.Visible = !game.IsGameOver;
629         _rollButton.Enabled = humanTurn && game.CanRoll && diceToRoll > 0;
```

Lines 628–629 Roll button hidden after the game; enabled only for a human with rolls left and at least one unheld die.

```
630         _rollButton.Text = game.RollsUsed == 0 ? "Roll Dice (R)"
631             : diceToRoll == 0 ? "All dice held"
632             : $"Re-roll {diceToRoll} {(diceToRoll == 1 ? "die" : "dice")} (R)";
633
```

Lines 630–632 Button text tells exactly what will happen: "Roll Dice", "Re-roll 3 dice", or "All dice held".

```
634         _hintLabel.Text = game switch
635         {
636             { IsGameOver: true } => "The game is over. Start a new game from the Game menu.",
637             _ when !humanTurn => $"{player.Name} is playing...",
638             { RollsUsed: 0 } => "Roll the dice to start your turn.",
639             { CanRoll: true } => "Click dice (or press 1-5) to hold them, then re-roll. " +
640                                  "Or click a score in the \"This roll\" column to end your turn.",
641             _ => "No rolls left. Click a score in the \"This roll\" column.",
642         };
```

Lines 634–642 A switch expression with property patterns and a `when` guard chooses the hint for the situation.

```
643         _newGameButton.Visible = _openButton.Visible = game.IsGameOver;
644
```

Line 643 Offer a new game or a saved one once this game is over (chained assignment).

```
645         RefreshScoreGrid(game);
646         UpdateTitleAndMenus();
647         UpdateAiTimer();
648     }
649
```

Lines 645–648 Redraw the scorecard, the title and menus, and start/stop the AI timer.

```
650     private void ShowWelcome()
651     {
```

Lines 650–651 Initial screen before any game.

```
652         _turnLabel.Text = "Welcome to Yahtzee";
653         _roundLabel.Text = "No game in progress";
654         _rollsLabel.Text = "";
```

Lines 652–654 Welcome texts.

```
655         foreach (var die in _dice)
656         {
657             die.Value = 0;
658             die.Held = false;
659             die.Enabled = false;
660         }
```

Lines 655–660 Five disabled "?" dice.

```
661         _rollButton.Visible = false;
662         _hintLabel.Text = "Start a new game to choose the players. Up to " +
663                           $"{GameEngine.MaxPlayers} players can take turns on this computer.";
664         _newGameButton.Visible = _openButton.Visible = true;
665         UpdateTitleAndMenus();
666         SetStatus("Ready.");
667     }
668
```

Lines 661–667 Hide Roll, explain how to start, show both start buttons.

```
669     /// <summary>Shows the file name and an unsaved-changes marker in the title, and enables Save when possible.</summary>
670     private void UpdateTitleAndMenus()
671     {
```

Lines 669–671 `UpdateTitleAndMenus` documentation.

```
672         string name = _savePath is null ? "new game" : Path.GetFileNameWithoutExtension(_savePath);
673         Text = _game is null ? "Yahtzee" : $"Yahtzee - {name}{(HasUnsavedChanges ? " (unsaved changes)" : "")}";
674         _saveItem.Enabled = _saveAsItem.Enabled = _game is { CanSave: true };
675     }
676
```

Lines 672–675 The title shows the save file's name (or "new game") and "(unsaved changes)" when appropriate, so the user always knows whether the game is safe; Save/Save As are enabled only while a game can be saved.

```
677     private void RefreshScoreGrid(GameEngine game)
678     {
```

Lines 677–678 Fills the grid from the engine.

```
679         var preview = game.Preview();
680         int currentColumn = FirstPlayerColumn + game.CurrentPlayerIndex;
681
```

Lines 679–680 Requirement: **score preview** for every open category after each roll.

```
682         foreach (DataGridViewRow row in _scoreGrid.Rows)
683         {
```

Lines 682–683 Visit every row.

```
684             var previewCell = row.Cells[PreviewColumn];
685             if (row.Tag is Category category)
686             {
```

Lines 684–686 For category rows (identified by the Tag pattern) ...

```
687                 bool hasPreview = preview.TryGetValue(category, out int points);
688                 previewCell.Value = hasPreview ? points.ToString() : "";
```

Lines 687–688 ... show the preview value, or blank for filled categories / before rolling.

```
689                 previewCell.Style.ForeColor = points > 0 ? UiTheme.Accent : UiTheme.PreviewZero;
690                 previewCell.Style.Font = points > 0 ? UiTheme.Bold : UiTheme.Body;
691                 previewCell.ToolTipText = hasPreview && IsHumanTurn ? $"Click to score {points} in {category.DisplayName()}" :
""";
692
```

Lines 689–691 Non-zero previews in bold blue, zeros in grey; tooltip "Click to score ..." for humans.

```
693                 for (int p = 0; p < game.Players.Count; p++)
694                     row.Cells[FirstPlayerColumn + p].Value = game.Players[p].ScoreCard.GetScore(category)?.ToString() ?? "";
695             }
```

Lines 693–695 Each player's recorded score, or blank (`?.` and `??` handle the null of an empty box).

```
696             else
697             {
698                 for (int p = 0; p < game.Players.Count; p++)
699                     row.Cells[FirstPlayerColumn + p].Value = game.Players[p].Total.ToString();
700             }
701         }
702
```

Lines 696–701 TOTAL row: each player's total.

```
703         for (int column = FirstPlayerColumn; column < _scoreGrid.Columns.Count; column++)
704         {
705             bool isCurrent = column == currentColumn && !game.IsGameOver;
706             _scoreGrid.Columns[column].HeaderCell.Style.BackColor = isCurrent ? UiTheme.CurrentPlayerHeader : UiTheme.HeaderBack;
707             _scoreGrid.Columns[column].DefaultCellStyle.BackColor = isCurrent ? Color.FromArgb(255, 250, 230) :
SystemColors.Window;
708         }
```

Lines 703–708 Highlights the current player's column header (gold) and cells (cream).

```
709         _scoreGrid.Columns[PreviewColumn].HeaderText = game.IsGameOver ? "This roll" : $"This roll ({game.CurrentPlayer.Name})";
710     }
711
```

Lines 709–710 Preview header names whose roll it is.

```
712     private void SetStatus(string message) => _statusLabel.Text = message;
713
```

Line 712 Writes a message to the status bar.

```
714     /// <summary>
715     /// Runs <paramref name="show"/> (which opens a dialog) with computer players paused, so the game
716     /// cannot move on while the user is reading a message or choosing a file.
717     /// </summary>
```

Lines 714–717 `ShowModal` documentation.

```
718     private T ShowModal<T>(Func<T> show)
719     {
```

Lines 718–719 A **generic method**: it runs any dialog-showing function and returns whatever that function returns (`DialogResult` here), with the AI paused around it.

```
720         _modalDepth++;
721         _aiTimer.Stop();
```

Lines 720–721 Count the open dialog and stop the timer…

```
722         try
723         {
724             return show();
725         }
```

Lines 722–725 …show the dialog…

```
726         finally
727         {
728             _modalDepth--;
729             UpdateAiTimer();
730         }
731     }
732
```

Lines 726–731 …and, even if showing it threw an exception, uncount it and let the AI continue if appropriate (`finally` always runs).

```
733     private void ShowError(string message) =>
734         ShowModal(() => MessageBox.Show(this, message, "Yahtzee", MessageBoxButtons.OK, MessageBoxIcon.Error));
735
```

Lines 733–734 Error message box, also through `ShowModal`.

```
736     // ------------------------------------------------------------------
737     // Help
738     // ------------------------------------------------------------------
739
```

Lines 736–738 Section: help.

```
740     private static string DescribeCategory(Category category) => category switch
741     {
```

Lines 740–741 One-sentence rule for each category, used in tooltips and the How to Play box.

```
742         >= Category.Aces and <= Category.Sixes => $"Sum of all dice showing {category.Face()}.",
743         Category.Chance => "Sum of all dice, any combination.",
744         Category.ThreeOfAKind => "At least three dice the same: sum of all dice.",
745         Category.FourOfAKind => "At least four dice the same: sum of all dice.",
746         Category.FullHouse => $"Three of one number and two of another: {Scorer.FullHouseScore} points. (Five of a kind does not
count.)",
747         Category.SmallStraight => $"Four sequential dice: {Scorer.SmallStraightScore} points.",
748         Category.LargeStraight => $"Five sequential dice: {Scorer.LargeStraightScore} points.",
749         Category.Yahtzee => $"All five dice the same: {Scorer.YahtzeeScore} points.",
750         _ => "",
751     };
752
```

Lines 742–751 Switch expression over the categories, reusing the `Scorer` constants for fixed scores.

```
753     private void ShowHowToPlay()
754     {
```

Lines 753–754 Help → How to Play.

```
755         string categories = string.Join("\n", CategoryExtensions.All.Select(c => $"  {c.DisplayName()}: {DescribeCategory(c)}"));
```

Line 755 Builds the list of category rules with LINQ `Select` and `string.Join`.

```
756         ShowModal(() => MessageBox.Show(this,
757             "Each turn, roll the dice up to three times. Between rolls, click dice (or press 1-5) to hold them; " +
758             "held dice are gold and will not be re-rolled.\n\n" +
759             "After any roll, click a score in the \"This roll\" column to record it. Each category can be used " +
760             "once, and dice that don't fit a category score 0 in it.\n\n" +
761             $"The game lasts {GameEngine.TotalRounds} rounds. The highest total wins.\n\n" + categories,
762             "How to Play", MessageBoxButtons.OK, MessageBoxIcon.Information));
763     }
764 }
```

Lines 756–764 Shows the rules summary.

12. Line-by-line walkthrough: Yahtzee.Core.Tests: the unit tests

12.1 Scripted dice for tests

`tests/Yahtzee.Core.Tests/FixedDiceRoller.cs · 13 lines`

A **test double**: an `IDiceRoller` that returns values from a fixed list, making every test repeatable.

```
1  using Yahtzee.Core.Model;
2
```

Line 1 Model namespace for `IDiceRoller`.

```
3  namespace Yahtzee.Core.Tests;
4
```

Line 3 Test namespace.

```
5  /// <summary>Test double that returns a predetermined sequence of die values.</summary>
6  internal sealed class FixedDiceRoller(params int[] values) : IDiceRoller
7  {
```

Lines 5–7 `params int[] values` lets tests write `new FixedDiceRoller(1, 2, 3, 4, 5)`.

```
8      private readonly Queue<int> _values = new(values);
9
```

Line 8 A queue hands out the values first-in, first-out.

```
10     public int RollDie() => _values.Count > 0
11         ? _values.Dequeue()
12         : throw new InvalidOperationException("FixedDiceRoller ran out of values.");
13 }
```

Lines 10–13 Returns the next value, or throws if a test rolled more dice than it scripted, which exposes mistakes in tests.

12.2 Other test doubles

`tests/Yahtzee.Core.Tests/TestDoubles.cs · 37 lines`

A controllable clock and a scripted AI strategy.

```
1  using Yahtzee.Core.AI;
2  using Yahtzee.Core.Model;
3
```

Lines 1–2 Imports.

```
4  namespace Yahtzee.Core.Tests;
5
```

Line 4 Test namespace.

```
6  /// <summary>Clock that always returns a fixed time, which tests can move forward.</summary>
7  internal sealed class ManualTimeProvider(DateTimeOffset now) : TimeProvider
8  {
```

Lines 6–8 `ManualTimeProvider` derives from .NET's `TimeProvider` so the engine can use it like the real clock.

```
9      public DateTimeOffset Now { get; set; } = now;
10
```

Line 9 Tests can set the current time.

```
11     public override DateTimeOffset GetUtcNow() => Now;
12 }
13
```

Lines 11–12 The engine asks for the time; it gets `Now`.

```
14 /// <summary>
15 /// Strategy test double: holds whatever <see cref="HoldsToReturn"/> says and scores the first open category.
16 /// </summary>
```

Lines 14–16 Documentation.

```
17 internal sealed class ScriptedStrategy : IPlayerStrategy
18 {
```

Lines 17–18 A fake AI used to test `PlayAiStep` without real AI logic.

```
19     public string Name => "Scripted";
20
```

Line 19 Strategy name.

```
21     /// <summary>Returned from every <see cref="ChooseHolds"/> call; null means "stop rolling".</summary>
22     public IReadOnlyList<bool>? HoldsToReturn { get; set; }
23
```

Lines 21–22 The holds to return (null = stop rolling).

```
24     /// <summary>If set, returned from <see cref="ChooseCategory"/> instead of the first open category.</summary>
25     public Category? CategoryToReturn { get; set; }
26
```

Lines 24–25 Optional fixed category choice.

```
27     public int ChooseHoldsCalls { get; private set; }
28
```

Line 27 Counts calls so tests can check how often the engine asked.

```
29      public IReadOnlyList<bool>? ChooseHolds(IReadOnlyList<int> dice, int rollsRemaining, ScoreCard scoreCard)
30      {
31          ChooseHoldsCalls++;
32          return HoldsToReturn;
33      }
34
```

Lines 29–33 Counts the call and returns the scripted holds.

```
35      public Category ChooseCategory(IReadOnlyList<int> dice, ScoreCard scoreCard) =>
36          CategoryToReturn ?? scoreCard.AvailableCategories.First();
37  }
```

Lines 35–37 Returns the scripted category or the first open one.

12.3 Scoring rule tests

`tests/Yahtzee.Core.Tests/ScorerTests.cs · 96 lines`

Each `[Theory]` runs once per `[InlineData]` row (category/dice/expected score). The cases include the examples from Figure 1 and the tricky edges of the simplified rules.

```
1  using Yahtzee.Core.Model;
2  using Yahtzee.Core.Rules;
3
```

Lines 1–2 Imports.

```
4  namespace Yahtzee.Core.Tests;
5
```

Line 4 Test namespace.

```
6  public class ScorerTests
7  {
```

Lines 6–7 Test class (xUnit creates a new instance per test).

```
8      [Theory]
9      [InlineData(Category.Aces, new[] { 1, 1, 1, 4, 5 }, 3)]
10     [InlineData(Category.Aces, new[] { 2, 3, 4, 5, 6 }, 0)]
11     [InlineData(Category.Twos, new[] { 2, 2, 2, 5, 6 }, 6)]
12     [InlineData(Category.Threes, new[] { 3, 3, 3, 3, 1 }, 12)]
13     [InlineData(Category.Fours, new[] { 4, 4, 5, 5, 6 }, 8)]
14     [InlineData(Category.Fives, new[] { 1, 1, 2, 2, 5 }, 5)]
15     [InlineData(Category.Sixes, new[] { 3, 3, 6, 6, 6 }, 18)]
16     [InlineData(Category.Sixes, new[] { 6, 6, 6, 6, 6 }, 30)]
17     public void UpperSection_SumsMatchingFace(Category category, int[] dice, int expected) =>
18         Assert.Equal(expected, Scorer.Score(category, dice));
19
```

Lines 8–18 **UpperSection_SumsMatchingFace**: Aces 1 1 1 4 5 → 3, Aces with no 1s → 0, Twos 6, Threes 12, Fours 8, Fives 5, Sixes 18, and five sixes → 30.

```
20     [Theory]
21     [InlineData(new[] { 1, 1, 3, 4, 5 }, 14)]
22     [InlineData(new[] { 6, 6, 6, 6, 6 }, 30)]
23     [InlineData(new[] { 1, 1, 1, 1, 1 }, 5)]
24     public void Chance_SumsAllDice(int[] dice, int expected) =>
25         Assert.Equal(expected, Scorer.Score(Category.Chance, dice));
26
```

Lines 20–25 **Chance_SumsAllDice**: Chance is always the sum (14, 30, 5).

```
27     [Theory]
28     [InlineData(new[] { 2, 3, 4, 4, 4 }, 17)]
29     [InlineData(new[] { 4, 4, 4, 4, 1 }, 17)] // four of a kind also counts
30     [InlineData(new[] { 5, 5, 5, 5, 5 }, 25)] // a Yahtzee also counts
31     [InlineData(new[] { 2, 2, 3, 3, 4 }, 0)]
32     public void ThreeOfAKind_RequiresAtLeastThreeSame(int[] dice, int expected) =>
33         Assert.Equal(expected, Scorer.Score(Category.ThreeOfAKind, dice));
34
```

Lines 27–33 **ThreeOfAKind_RequiresAtLeastThreeSame**: Three 4s → 17; four or five of a kind also count ("at least"); two pairs → 0.

```
35     [Theory]
36     [InlineData(new[] { 4, 5, 5, 5, 5 }, 24)]
37     [InlineData(new[] { 3, 3, 3, 3, 3 }, 15)] // a Yahtzee also counts
38     [InlineData(new[] { 5, 5, 5, 4, 4 }, 0)]
39     public void FourOfAKind_RequiresAtLeastFourSame(int[] dice, int expected) =>
40         Assert.Equal(expected, Scorer.Score(Category.FourOfAKind, dice));
41
```

Lines 35–40 **FourOfAKind_RequiresAtLeastFourSame**: Four 5s → 24; a Yahtzee also counts; three of a kind → 0.

```
42      [Theory]
43      [InlineData(new[] { 2, 2, 5, 5, 5 }, 25)]
44      [InlineData(new[] { 5, 2, 5, 2, 5 }, 25)] // order does not matter
45      [InlineData(new[] { 2, 2, 5, 5, 6 }, 0)]  // two pair
46      [InlineData(new[] { 2, 5, 5, 5, 5 }, 0)]  // four of a kind
47      public void FullHouse_RequiresThreeAndTwo(int[] dice, int expected) =>
48          Assert.Equal(expected, Scorer.Score(Category.FullHouse, dice));
49
```

Lines 42–48 **FullHouse_RequiresThreeAndTwo**: 3+2 → 25 in any order; two pair → 0; four of a kind → 0.

```
50      [Fact]
51      public void FullHouse_RejectsYahtzee() =>
52          Assert.Equal(0, Scorer.Score(Category.FullHouse, [4, 4, 4, 4, 4]));
53
```

Lines 50–52 **FullHouse_RejectsYahtzee**: The specification's explicit note: five 4s are *not* a full house.

```
54      [Theory]
55      [InlineData(new[] { 1, 2, 3, 4, 6 }, 30)]
56      [InlineData(new[] { 2, 3, 4, 5, 5 }, 30)] // duplicate inside the run
57      [InlineData(new[] { 6, 4, 3, 1, 5 }, 30)] // unsorted 3-4-5-6
58      [InlineData(new[] { 1, 2, 3, 4, 5 }, 30)] // a large straight is also a small straight
59      [InlineData(new[] { 1, 2, 3, 5, 6 }, 0)]  // gap breaks the run
60      [InlineData(new[] { 1, 1, 2, 2, 3 }, 0)]
61      public void SmallStraight_RequiresFourSequential(int[] dice, int expected) =>
62          Assert.Equal(expected, Scorer.Score(Category.SmallStraight, dice));
63
```

Lines 54–62 **SmallStraight_RequiresFourSequential**: 1-2-3-4, 2-3-4-5 with a duplicate, unsorted 3-4-5-6, and a large straight all score 30; a gap → 0.

```
64      [Theory]
65      [InlineData(new[] { 1, 2, 3, 4, 5 }, 40)]
66      [InlineData(new[] { 6, 5, 4, 3, 2 }, 40)]
67      [InlineData(new[] { 1, 2, 3, 4, 6 }, 0)]
68      [InlineData(new[] { 1, 2, 3, 4, 4 }, 0)]
69      public void LargeStraight_RequiresFiveSequential(int[] dice, int expected) =>
70          Assert.Equal(expected, Scorer.Score(Category.LargeStraight, dice));
71
```

Lines 64–70 **LargeStraight_RequiresFiveSequential**: 1-5 and 2-6 → 40; four in a row → 0.

```
72      [Theory]
73      [InlineData(new[] { 3, 3, 3, 3, 3 }, 50)]
74      [InlineData(new[] { 3, 3, 3, 3, 2 }, 0)]
75      public void Yahtzee_RequiresFiveSame(int[] dice, int expected) =>
76          Assert.Equal(expected, Scorer.Score(Category.Yahtzee, dice));
77
```

Lines 72–76 **Yahtzee_RequiresFiveSame**: Five 3s → 50; four 3s → 0.

```
78      [Theory]
79      [InlineData(new[] { 1, 2, 3, 4 })]
80      [InlineData(new[] { 1, 2, 3, 4, 5, 6 })]
81      [InlineData(new[] { 0, 2, 3, 4, 5 })]
82      [InlineData(new[] { 1, 2, 3, 4, 7 })]
83      public void Score_RejectsInvalidDice(int[] dice) =>
84          Assert.Throws<ArgumentException>(() => Scorer.Score(Category.Chance, dice));
85
```

Lines 78–84 **Score_RejectsInvalidDice**: Four dice, six dice, a 0 and a 7 all throw `ArgumentException`.

```
86      [Fact]
87      public void Preview_ScoresOnlyRequestedCategories()
88      {
89          var preview = Scorer.Preview([2, 2, 5, 5, 5], [Category.FullHouse, Category.Fives, Category.Yahtzee]);
90
91          Assert.Equal(3, preview.Count);
92          Assert.Equal(25, preview[Category.FullHouse]);
93          Assert.Equal(15, preview[Category.Fives]);
94          Assert.Equal(0, preview[Category.Yahtzee]);
95      }
96  }
```

Lines 86–96 **Preview_ScoresOnlyRequestedCategories**: Preview returns exactly the requested categories with correct scores (Arrange/Act/Assert layout).

12.4 Scorecard tests

`tests/Yahtzee.Core.Tests/ScoreCardTests.cs · 89 lines`

Recording, the once-per-category rule, totals and restoring.

```
1  using Yahtzee.Core.Model;
2
```

Line 1 Imports.

```
3  namespace Yahtzee.Core.Tests;
4
```

Line 3 Test namespace.

```
5  public class ScoreCardTests
6  {
```

Lines 5–6 Test class.

```
7      [Fact]
8      public void NewCard_IsEmpty()
9      {
10         var card = new ScoreCard();
11
12         Assert.Equal(13, card.AvailableCategories.Count());
13         Assert.False(card.IsComplete);
14         Assert.Equal(0, card.Total);
15         Assert.Null(card.GetScore(Category.Chance));
16     }
17
```

Lines 7–16 **NewCard_IsEmpty**: 13 open categories, not complete, total 0, empty box returns null.

```
18     [Fact]
19     public void Record_StoresScoreAndRemovesCategory()
20     {
21         var card = new ScoreCard();
22
23         int score = card.Record(Category.Sixes, [6, 6, 6, 1, 2]);
24
25         Assert.Equal(18, score);
26         Assert.Equal(18, card.GetScore(Category.Sixes));
27         Assert.True(card.IsFilled(Category.Sixes));
28         Assert.DoesNotContain(Category.Sixes, card.AvailableCategories);
29     }
30
```

Lines 18–29 **Record_StoresScoreAndRemovesCategory**: Recording 18 in Sixes stores it and removes Sixes from the open list.

```
31     [Fact]
32     public void Record_NonMatchingDiceScoresZero()
33     {
34         var card = new ScoreCard();
35
36         Assert.Equal(0, card.Record(Category.Yahtzee, [1, 2, 3, 4, 6]));
37         Assert.True(card.IsFilled(Category.Yahtzee));
38     }
39
```

Lines 31–38 **Record_NonMatchingDiceScoresZero**: A failed Yahtzee scores 0 but still fills the box.

```
40     [Fact]
41     public void Record_SameCategoryTwice_Throws()
42     {
43         var card = new ScoreCard();
44         card.Record(Category.Chance, [1, 2, 3, 4, 5]);
45
46         Assert.Throws<InvalidOperationException>(() => card.Record(Category.Chance, [6, 6, 6, 6, 6]));
47     }
48
```

Lines 40–47 **Record_SameCategoryTwice_Throws**: Using a category twice is rejected.

```
49      [Fact]
50      public void FillingAllCategories_CompletesCardAndSumsTotal()
51      {
52          var card = new ScoreCard();
53          int[] dice = [5, 5, 5, 5, 5];
54
55          foreach (var category in CategoryExtensions.All)
56              card.Record(category, dice);
57
58          Assert.True(card.IsComplete);
59          Assert.Empty(card.AvailableCategories);
60          // Fives 25 + Chance 25 + 3oak 25 + 4oak 25 + Yahtzee 50; everything else 0.
61          Assert.Equal(150, card.Total);
62      }
63
```

Lines 49–62 **FillingAllCategories_CompletesCardAndSumsTotal**: Filling all 13 with five 5s completes the card; total 150 is worked out in the comment.

```
64      [Fact]
65      public void Preview_ExcludesFilledCategories()
66      {
67          var card = new ScoreCard();
68          card.Record(Category.FullHouse, [2, 2, 3, 3, 3]);
69
70          var preview = card.Preview([2, 2, 3, 3, 3]);
71
72          Assert.Equal(12, preview.Count);
73          Assert.False(preview.ContainsKey(Category.FullHouse));
74          Assert.Equal(9, preview[Category.Threes]);
75      }
76
```

Lines 64–75 **Preview_ExcludesFilledCategories**: After filling Full House the preview has 12 entries and no Full House.

```
77      [Fact]
78      public void RestoreConstructor_KeepsScores()
79      {
80          var card = new ScoreCard(new Dictionary<Category, int> { [Category.Aces] = 3, [Category.Yahtzee] = 50 });
81
82          Assert.Equal(53, card.Total);
83          Assert.Equal(11, card.AvailableCategories.Count());
84      }
85
```

Lines 77–84 **RestoreConstructor_KeepsScores**: A restored card keeps its scores (total 53, 11 open).

```
86      [Fact]
87      public void RestoreConstructor_RejectsNegativeScore() =>
88          Assert.Throws<ArgumentException>(() => new ScoreCard(new Dictionary<Category, int> { [Category.Aces] = -1 }));
89  }
```

Lines 86–89 **RestoreConstructor_RejectsNegativeScore**: Negative saved scores are rejected.

12.5 Dice cup tests

`tests/Yahtzee.Core.Tests/DiceCupTests.cs · 90 lines`

Rolling, holding, resetting and restoring dice with scripted values.

```
1  using Yahtzee.Core.Model;
2
```

Line 1 Imports.

```
3  namespace Yahtzee.Core.Tests;
4
```

Line 3 Test namespace.

```
5  public class DiceCupTests
6  {
```

Lines 5–6 Test class.

```
7      [Fact]
8      public void FirstRoll_RollsAllFiveDice()
9      {
10         var cup = new DiceCup(new FixedDiceRoller(1, 2, 3, 4, 5));
11
12         cup.Roll();
13
14         Assert.True(cup.HasRolled);
15         Assert.Equal(new[] { 1, 2, 3, 4, 5 }, cup.Values);
16     }
17
```

Lines 7–16 **FirstRoll_RollsAllFiveDice**: The first roll takes five values in order.

```
18     [Fact]
19     public void Reroll_KeepsHeldDice()
20     {
21         var cup = new DiceCup(new FixedDiceRoller(1, 2, 3, 4, 5, 6, 6, 6));
22         cup.Roll();
23         cup.SetHeld(1, true);
24         cup.SetHeld(3, true);
25
26         cup.Roll();
27
28         Assert.Equal(new[] { 6, 2, 6, 4, 6 }, cup.Values);
29     }
30
```

Lines 18–29 **Reroll_KeepsHeldDice**: With dice 2 and 4 held, the second roll replaces only dice 1, 3 and 5 (with 6s).

```
31     [Fact]
32     public void ToggleHeld_ReleasesPreviouslyHeldDie()
33     {
34         var cup = new DiceCup(new FixedDiceRoller(1, 2, 3, 4, 5));
35         cup.Roll();
36
37         cup.ToggleHeld(2);
38         cup.ToggleHeld(2);
39
40         Assert.False(cup.Held[2]);
41     }
42
```

Lines 31–41 **ToggleHeld_ReleasesPreviouslyHeldDie**: Toggling twice releases the die again.

```
43     [Fact]
44     public void Hold_BeforeFirstRoll_Throws()
45     {
46         var cup = new DiceCup(new FixedDiceRoller());
47
48         Assert.Throws<InvalidOperationException>(() => cup.SetHeld(0, true));
49     }
50
```

Lines 43–49 **Hold_BeforeFirstRoll_Throws**: Cannot hold unrolled dice.

```
51      [Theory]
52      [InlineData(-1)]
53      [InlineData(5)]
54      public void Hold_InvalidIndex_Throws(int index)
55      {
56          var cup = new DiceCup(new FixedDiceRoller(1, 2, 3, 4, 5));
57          cup.Roll();
58
59          Assert.Throws<ArgumentOutOfRangeException>(() => cup.SetHeld(index, true));
60      }
61
```

Lines 51–60 **Hold_InvalidIndex_Throws**: Indexes −1 and 5 are out of range.

```
62      [Fact]
63      public void Reset_ClearsHoldsAndRolledState()
64      {
65          var cup = new DiceCup(new FixedDiceRoller(1, 2, 3, 4, 5, 6, 6, 6, 6, 6));
66          cup.Roll();
67          cup.SetHeld(0, true);
68
69          cup.Reset();
70          cup.Roll();
71
72          Assert.All(cup.Held, held => Assert.False(held));
73          Assert.Equal(new[] { 6, 6, 6, 6, 6 }, cup.Values);
74      }
75
```

Lines 62–74 **Reset_ClearsHoldsAndRolledState**: After reset, holds are cleared and the next roll replaces all five dice.

```
76      [Fact]
77      public void RestoreConstructor_KeepsValuesAndHolds()
78      {
79          var cup = new DiceCup(new FixedDiceRoller(), [3, 3, 3, 1, 2], [true, true, true, false, false]);
80
81          Assert.True(cup.HasRolled);
82          Assert.Equal(new[] { 3, 3, 3, 1, 2 }, cup.Values);
83          Assert.Equal(new[] { true, true, true, false, false }, cup.Held);
84      }
85
```

Lines 76–84 **RestoreConstructor_KeepsValuesAndHolds**: A restored cup has the saved values and holds and counts as rolled.

```
86      [Fact]
87      public void RestoreConstructor_RejectsInvalidValues() =>
88          Assert.Throws<ArgumentException>(() =>
89              new DiceCup(new FixedDiceRoller(), [0, 3, 3, 1, 2], [false, false, false, false, false]));
90  }
```

Lines 86–90 **RestoreConstructor_RejectsInvalidValues**: A saved 0 is rejected.

12.6 Player tests

`tests/Yahtzee.Core.Tests/PlayerTests.cs · 38 lines`

Name validation and AI detection.

```
1  using Yahtzee.Core.Model;
2
```

Line 1 Imports.

```
3  namespace Yahtzee.Core.Tests;
4
```

Line 3 Test namespace.

```
5  public class PlayerTests
6  {
```

Lines 5–6 Test class.

```
7      [Fact]
8      public void Constructor_TrimsName()
9      {
10         var player = new Player("  Alice  ");
11
12         Assert.Equal("Alice", player.Name);
13         Assert.False(player.IsAi);
14         Assert.Equal(0, player.Total);
15     }
16
```

Lines 7–15 **Constructor_TrimsName**: Spaces are trimmed; a new player is human with 0 points.

```
17     [Theory]
18     [InlineData("")]
19     [InlineData("   ")]
20     [InlineData("ThisNameIsWayTooLongToFit")]
21     public void Constructor_RejectsInvalidName(string name) =>
22         Assert.Throws<ArgumentException>(() => new Player(name));
23
```

Lines 17–22 **Constructor_RejectsInvalidName**: Empty, blank and 25-character names are rejected.

```
24     [Fact]
25     public void PlayerWithStrategy_IsAi() =>
26         Assert.True(new Player("Bot", new ScriptedStrategy()).IsAi);
27
```

Lines 24–26 **PlayerWithStrategy_IsAi**: A player with a strategy is an AI.

```
28     [Fact]
29     public void Constructor_KeepsGivenScoreCard()
30     {
31         var card = new ScoreCard(new Dictionary<Category, int> { [Category.Chance] = 20 });
32
33         var player = new Player("Alice", scoreCard: card);
34
35         Assert.Same(card, player.ScoreCard);
36         Assert.Equal(20, player.Total);
37     }
38 }
```

Lines 28–38 **Constructor_KeepsGivenScoreCard**: A supplied scorecard is used as-is (same object).

12.7 Game engine tests

`tests/Yahtzee.Core.Tests/GameEngineTests.cs · 353 lines`

The largest test class: construction rules, rolling and holding, scoring and turn order, game over, and AI steps.

```
1  using Yahtzee.Core.Game;
2  using Yahtzee.Core.Model;
3
```

Lines 1–2 Imports.

```
4  namespace Yahtzee.Core.Tests;
5
```

Line 4 Test namespace.

```
6  public class GameEngineTests
7  {
```

Lines 6–7 Test class.

```
8      private static readonly DateTimeOffset Start = new(2026, 10, 6, 12, 0, 0, TimeSpan.Zero);
9
```

Line 8 A fixed start time for the fake clock.

```
10     /// <summary>A roller with enough fives for any game in these tests.</summary>
11     private static FixedDiceRoller Fives() => new([.. Enumerable.Repeat(5, 1000)]);
12
```

Lines 10–11 `Fives()`: a roller returning 1000 fives (spread of `Enumerable.Repeat` into the params array).

```
13     private static GameEngine NewGame(IDiceRoller roller, params string[] names) =>
14         new(names.Select(name => new Player(name)), roller, new ManualTimeProvider(Start));
15
```

Lines 13–14 Helper creating a game of human players with the given names.

```
16     // ---- Construction ----
17
```

Line 16 Section comment: tests of the constructor's rules.

```
18     [Fact]
19     public void NewGame_StartsWithFirstPlayerUnrolled()
20     {
21         var game = NewGame(Fives(), "Alice", "Bob");
22
23         Assert.Equal("Alice", game.CurrentPlayer.Name);
24         Assert.Equal(1, game.Round);
25         Assert.Equal(0, game.RollsUsed);
26         Assert.Equal(3, game.RollsRemaining);
27         Assert.True(game.CanRoll);
28         Assert.False(game.CanHold);
29         Assert.False(game.CanScore);
30         Assert.False(game.IsGameOver);
31         Assert.Equal(Start, game.StartedAt);
32     }
33
```

Lines 18–32 **NewGame_StartsWithFirstPlayerUnrolled**: A new game: first player on turn, round 1, 0 rolls used, may roll but not hold or score, start time recorded.

```
34     [Fact]
35     public void Constructor_RejectsNoPlayers() =>
36         Assert.Throws<ArgumentException>(() => NewGame(Fives()));
37
```

Lines 34–36 **Constructor_RejectsNoPlayers**: Zero players → exception.

```
38      [Fact]
39      public void Constructor_RejectsTooManyPlayers() =>
40          Assert.Throws<ArgumentException>(() => NewGame(Fives(), "A", "B", "C", "D", "E", "F", "G"));
41
```

Lines 38–40 **Constructor_RejectsTooManyPlayers**: Seven players → exception.

```
42      [Fact]
43      public void Constructor_RejectsDuplicateNamesIgnoringCase() =>
44          Assert.Throws<ArgumentException>(() => NewGame(Fives(), "Alice", "alice"));
45
```

Lines 42–44 **Constructor_RejectsDuplicateNamesIgnoringCase**: "Alice" and "alice" → exception.

```
46      // ---- Rolling and holding ----
47
```

Line 46 Section comment.

```
48      [Fact]
49      public void Roll_UsesUpRollsAndRaisesEvent()
50      {
51          var game = NewGame(new FixedDiceRoller(1, 2, 3, 4, 5), "Alice");
52          DiceRolledEventArgs? rolled = null;
53          game.DiceRolled += (_, e) => rolled = e;
54
55          game.Roll();
56
57          Assert.Equal(1, game.RollsUsed);
58          Assert.Equal(2, game.RollsRemaining);
59          Assert.NotNull(rolled);
60          Assert.Equal(new[] { 1, 2, 3, 4, 5 }, rolled.Dice);
61          Assert.Equal(2, rolled.RollsRemaining);
62      }
63
```

Lines 48–62 **Roll_UsesUpRollsAndRaisesEvent**: A roll uses a roll and raises `DiceRolled` with the values and rolls remaining.

```
64      [Fact]
65      public void Roll_FourthTimeInTurn_Throws()
66      {
67          var game = NewGame(Fives(), "Alice");
68          game.Roll();
69          game.Roll();
70          game.Roll();
71
72          Assert.False(game.CanRoll);
73          Assert.Throws<InvalidOperationException>(game.Roll);
74      }
75
```

Lines 64–74 **Roll_FourthTimeInTurn_Throws**: After three rolls `CanRoll` is false and a fourth roll throws.

```
76      [Fact]
77      public void HeldDice_AreKeptOnReroll()
78      {
79          var game = NewGame(new FixedDiceRoller(6, 6, 1, 2, 3, 6, 4, 5), "Alice");
80          game.Roll();
81          game.SetHeld(0, true);
82          game.ToggleHeld(1);
83
84          game.Roll();
85
86          Assert.Equal(new[] { 6, 6, 6, 4, 5 }, game.Dice);
87      }
88
```

Lines 76–87 **HeldDice_AreKeptOnReroll**: Holding the two 6s and re-rolling gives 6 6 6 4 5.

```
89      [Fact]
90      public void Hold_BeforeFirstRoll_Throws()
91      {
92          var game = NewGame(Fives(), "Alice");
93
94          Assert.Throws<InvalidOperationException>(() => game.SetHeld(0, true));
95      }
96
```

Lines 89–95 **Hold_BeforeFirstRoll_Throws**: Holding before rolling is refused.

```
97      [Fact]
98      public void Hold_AfterThirdRoll_Throws()
99      {
100         var game = NewGame(Fives(), "Alice");
101         game.Roll();
102         game.Roll();
103         game.Roll();
104
105         Assert.False(game.CanHold);
106         Assert.Throws<InvalidOperationException>(() => game.SetHeld(0, true));
107     }
108
```

Lines 97–107 **Hold_AfterThirdRoll_Throws**: Holding when no rolls remain is refused.

```
109     [Theory]
110     [InlineData(-1)]
111     [InlineData(5)]
112     public void Hold_InvalidIndex_Throws(int index)
113     {
114         var game = NewGame(Fives(), "Alice");
115         game.Roll();
116
117         Assert.Throws<ArgumentOutOfRangeException>(() => game.SetHeld(index, true));
118     }
119
```

Lines 109–118 **Hold_InvalidIndex_Throws**: Bad indexes give `ArgumentOutOfRangeException`.

```
120     // ---- Scoring and turn order ----
121
```

Line 120 Section comment.

```
122     [Fact]
123     public void Score_BeforeRolling_Throws()
124     {
125         var game = NewGame(Fives(), "Alice");
126
127         Assert.Throws<InvalidOperationException>(() => game.Score(Category.Chance));
128     }
129
```

Lines 122–128 **Score_BeforeRolling_Throws**: Cannot score without rolling.

```
130     [Fact]
131     public void Preview_IsEmptyBeforeRollAndCoversOpenCategoriesAfter()
132     {
133         var game = NewGame(new FixedDiceRoller(2, 2, 3, 3, 3), "Alice");
134         Assert.Empty(game.Preview());
135
136         game.Roll();
137         var preview = game.Preview();
138
139         Assert.Equal(13, preview.Count);
140         Assert.Equal(25, preview[Category.FullHouse]);
141         Assert.Equal(13, preview[Category.Chance]);
142     }
143
```

Lines 130–142 **Preview_IsEmptyBeforeRollAndCoversOpenCategoriesAfter**: Preview empty before rolling; after rolling 2 2 3 3 3 it has 13 entries, Full House 25, Chance 13.

```
144     [Fact]
145     public void Score_RecordsPointsAndPassesTurnToNextPlayer()
146     {
147         var game = NewGame(Fives(), "Alice", "Bob");
148         TurnStartedEventArgs? turn = null;
149         CategoryScoredEventArgs? scored = null;
150         game.TurnStarted += (_, e) => turn = e;
151         game.CategoryScored += (_, e) => scored = e;
152         game.Roll();
153         game.SetHeld(0, true);
154 
155         int points = game.Score(Category.Yahtzee);
156 
157         Assert.Equal(50, points);
158         Assert.Equal(50, game.Players[0].ScoreCard.GetScore(Category.Yahtzee));
159         Assert.Equal(new CategoryScoredEventArgs(game.Players[0], Category.Yahtzee, 50), scored);
160         Assert.Equal("Bob", game.CurrentPlayer.Name);
161         Assert.Equal(new TurnStartedEventArgs(game.Players[1], 1), turn);
162         Assert.Equal(0, game.RollsUsed);
163         Assert.All(game.Held, held => Assert.False(held));
164     }
165 
```

Lines 144–164 **Score_RecordsPointsAndPassesTurnToNextPlayer**: Scoring records 50, raises the right events (compared as records), passes the turn to Bob and resets rolls and holds.

```
166     [Fact]
167     public void Round_AdvancesAfterEveryPlayerHasHadATurn()
168     {
169         var game = NewGame(Fives(), "Alice", "Bob");
170 
171         game.Roll();
172         game.Score(Category.Aces);
173         Assert.Equal(1, game.Round); // Bob's first turn
174 
175         game.Roll();
176         game.Score(Category.Aces);
177         Assert.Equal(2, game.Round); // back to Alice
178         Assert.Equal("Alice", game.CurrentPlayer.Name);
179     }
180 
```

Lines 166–179 **Round_AdvancesAfterEveryPlayerHasHadATurn**: The round number only increases when play returns to the first player.

```
181     [Fact]
182     public void Score_FilledCategory_ThrowsAndKeepsTurn()
183     {
184         var game = NewGame(Fives(), "Alice");
185         game.Roll();
186         game.Score(Category.Chance);
187         game.Roll();
188 
189         Assert.Throws<InvalidOperationException>(() => game.Score(Category.Chance));
190         Assert.Equal(1, game.RollsUsed);
191     }
192 
```

Lines 181–191 **Score_FilledCategory_ThrowsAndKeepsTurn**: Scoring a filled box throws and the turn is not lost.

```
193     [Fact]
194     public void StateChanged_RaisedForEveryAction()
195     {
196         var game = NewGame(Fives(), "Alice");
197         int changes = 0;
198         game.StateChanged += (_, _) => changes++;
199 
200         game.Roll();
201         game.SetHeld(0, true);
202         game.SetHeld(0, true); // no change, no event
203         game.Score(Category.Fives);
204 
205         Assert.Equal(3, changes);
206     }
207 
```

Lines 193–206 **StateChanged_RaisedForEveryAction**: Roll, hold and score each raise `StateChanged`; a no-op hold does not (3 events).

```
208     // ---- Game over ----
209
```

Line 208 Section comment.

```
210     [Fact]
211     public void SinglePlayerGame_EndsAfterThirteenTurns()
212     {
213         var clock = new ManualTimeProvider(Start);
214         var game = new GameEngine([new Player("Alice")], Fives(), clock);
215         GameOverEventArgs? over = null;
216         game.GameOver += (_, e) => over = e;
217         clock.Now = Start.AddMinutes(10);
218
219         foreach (var category in CategoryExtensions.All)
220         {
221             Assert.Null(over);
222             game.Roll();
223             game.Score(category);
224         }
225
226         Assert.True(game.IsGameOver);
227         Assert.False(game.CanRoll);
228         Assert.False(game.CanScore);
229         Assert.NotNull(over);
230         Assert.Equal(Start.AddMinutes(10), over.CompletedAt);
231         // All fives: Fives 25 + Chance 25 + 3oak 25 + 4oak 25 + Yahtzee 50.
232         Assert.Equal(new[] { new PlayerStanding(1, "Alice", 150, false) }, over.Standings);
233         Assert.Throws<InvalidOperationException>(game.Roll);
234     }
235
```

Lines 210–234 **SinglePlayerGame_EndsAfterThirteenTurns**: 13 turns end the game with one `GameOver` carrying the standings (150 points) and the completion time; rolling afterwards throws.

```
236     [Fact]
237     public void MultiPlayerGame_EndsOnlyWhenLastPlayerFinishes()
238     {
239         var game = NewGame(Fives(), "Alice", "Bob");
240         int gameOverCount = 0;
241         game.GameOver += (_, _) => gameOverCount++;
242
243         foreach (var category in CategoryExtensions.All)
244         {
245             game.Roll();
246             game.Score(category); // Alice
247             Assert.False(game.IsGameOver);
248             game.Roll();
249             game.Score(category); // Bob
250         }
251
252         Assert.True(game.IsGameOver);
253         Assert.Equal(1, gameOverCount);
254     }
255
```

Lines 236–254 **MultiPlayerGame_EndsOnlyWhenLastPlayerFinishes**: In a two-player game the game is not over after Alice's 13th turn, only after Bob's; `GameOver` fires once.

```
256     [Fact]
257     public void GetStandings_OrdersByScoreAndSharesRankOnTies()
258     {
259         var game = NewGame(Fives(), "Alice", "Bob", "Carol");
260         game.Roll(); game.Score(Category.Aces);    // Alice 0
261         game.Roll(); game.Score(Category.Yahtzee); // Bob 50
262         game.Roll(); game.Score(Category.Twos);    // Carol 0
263 
264         var standings = game.GetStandings();
265 
266         Assert.Equal(
267             new[]
268             {
269                 new PlayerStanding(1, "Bob", 50, false),
270                 new PlayerStanding(2, "Alice", 0, false),
271                 new PlayerStanding(2, "Carol", 0, false),
272             },
273             standings);
274     }
275 
```

Lines 256–274 **GetStandings_OrdersByScoreAndSharesRankOnTies**: Bob 50 ranks 1st; Alice and Carol tied on 0 share 2nd.

```
276     // ---- AI steps ----
277 
```

Line 276 Section comment.

```
278     [Fact]
279     public void PlayAiStep_RollsThenRerollsWithChosenHoldsThenScores()
280     {
281         var strategy = new ScriptedStrategy { HoldsToReturn = [true, true, false, false, false] };
282         var game = new GameEngine(
283             [new Player("Bot", strategy)],
284             new FixedDiceRoller(6, 6, 1, 2, 3, 6, 6, 6, 6, 6, 6),
285             new ManualTimeProvider(Start));
286 
287         Assert.Equal(AiStepResult.Rolled, game.PlayAiStep()); // first roll: 6 6 1 2 3
288         Assert.Equal(AiStepResult.Rolled, game.PlayAiStep()); // hold 6 6: 6 6 6 6 6
289         Assert.Equal(new[] { 6, 6, 6, 6, 6 }, game.Dice);
290         Assert.Equal(AiStepResult.Rolled, game.PlayAiStep()); // third roll
291         Assert.Equal(AiStepResult.Scored, game.PlayAiStep()); // no rolls left: must score
292 
293         Assert.Equal(2, strategy.ChooseHoldsCalls);
294         Assert.Single(game.Players[0].ScoreCard.Scores);
295     }
296 
```

Lines 278–295 **PlayAiStep_RollsThenRerollsWithChosenHoldsThenScores**: An AI turn is roll, re-roll with holds, re-roll, score: four steps, with `ChooseHolds` asked twice.

```
297     [Fact]
298     public void PlayAiStep_StrategyStopsEarly_ScoresImmediately()
299     {
300         var strategy = new ScriptedStrategy { HoldsToReturn = null, CategoryToReturn = Category.Yahtzee };
301         var game = new GameEngine([new Player("Bot", strategy)], Fives(), new ManualTimeProvider(Start));
302 
303         game.PlayAiStep();
304         var result = game.PlayAiStep();
305 
306         Assert.Equal(AiStepResult.Scored, result);
307         Assert.Equal(50, game.Players[0].ScoreCard.GetScore(Category.Yahtzee));
308     }
309 
```

Lines 297–308 **PlayAiStep_StrategyStopsEarly_ScoresImmediately**: A strategy returning null holds scores after the first roll.

```
310     [Fact]
311     public void PlayAiStep_HumanPlayer_Throws()
312     {
313         var game = NewGame(Fives(), "Alice");
314 
315         Assert.Throws<InvalidOperationException>(() => game.PlayAiStep());
316     }
317 
```

Lines 310–316 **PlayAiStep_HumanPlayer_Throws**: `PlayAiStep` refuses to play for a human.

```
318     [Fact]
319     public void PlayAiStep_StrategyChoosesFilledCategory_Throws()
320     {
321         var strategy = new ScriptedStrategy { CategoryToReturn = Category.Chance };
322         var game = new GameEngine([new Player("Bot", strategy)], Fives(), new ManualTimeProvider(Start));
323         game.PlayAiStep();
324         game.PlayAiStep(); // scores Chance
325         game.PlayAiStep(); // rolls
326 
327         Assert.Throws<InvalidOperationException>(() => game.PlayAiStep());
328     }
329 
```

Lines 318–328 **PlayAiStep_StrategyChoosesFilledCategory_Throws**: An AI choosing a filled category is rejected.

```
330     [Fact]
331     public void MixedHumanAndAiGame_PlaysToCompletion()
332     {
333         var game = new GameEngine(
334             [new Player("Alice"), new Player("Bot", new ScriptedStrategy())],
335             Fives(),
336             new ManualTimeProvider(Start));
337 
338         while (!game.IsGameOver)
339         {
340             if (game.CurrentPlayer.IsAi)
341             {
342                 while (game.PlayAiStep() != AiStepResult.Scored) { }
343             }
344             else
345             {
346                 game.Roll();
347                 game.Score(game.CurrentPlayer.ScoreCard.AvailableCategories.Last());
348             }
349         }
350 
351         Assert.All(game.Players, player => Assert.True(player.ScoreCard.IsComplete));
352     }
353 }
```

Lines 330–353 **MixedHumanAndAiGame_PlaysToCompletion**: A human and an AI play a full game alternately; both cards end complete.

12.8 Save-file tests

`tests/Yahtzee.Core.Tests/SaveGameTests.cs · 233 lines`

Snapshots, JSON round trips, real file I/O, and every kind of invalid file that must be rejected.

```
1  using Yahtzee.Core.AI;
2  using Yahtzee.Core.Game;
3  using Yahtzee.Core.Model;
4  using Yahtzee.Core.Persistence;
5
```

Lines 1–4 Imports.

```
6  namespace Yahtzee.Core.Tests;
7
```

Line 6 Test namespace.

```
8  public class SaveGameTests
9  {
```

Lines 8–9 Test class.

```
10     private static readonly DateTimeOffset Start = new(2026, 10, 8, 9, 30, 0, TimeSpan.Zero);
11
12     private static FixedDiceRoller Fives() => new([.. Enumerable.Repeat(5, 1000)]);
13
14     private static IPlayerStrategy CreateScripted(string name) =>
15         name == "Scripted" ? new ScriptedStrategy() : throw new ArgumentException($"Unknown strategy {name}");
16
17     private static GameEngine Restore(GameSnapshot snapshot) =>
18         GameEngine.Restore(snapshot, Fives(), CreateScripted, new ManualTimeProvider(Start.AddHours(1)));
19
```

Lines 10–18 Fixed start time and helpers: a fives roller, a strategy factory that only knows "Scripted", and a `Restore` shortcut with a later clock (to prove the start time comes from the file).

```
20     /// <summary>Alice and Bob have each played one turn; Alice has rolled once and held two dice.</summary>
21     private static GameEngine MidTurnGame()
22     {
23         var roller = new FixedDiceRoller([.. Enumerable.Repeat(5, 10), 3, 3, 6, 2, 1]);
24         var game = new GameEngine([new Player("Alice"), new Player("Bob", new ScriptedStrategy())], roller, new
ManualTimeProvider(Start));
25         game.Roll();
26         game.Score(Category.Fives);    // Alice 25
27         game.Roll();
28         game.Score(Category.Yahtzee);  // Bob 50
29         game.Roll();                   // Alice rolls 3 3 6 2 1
30         game.SetHeld(0, true);
31         game.SetHeld(1, true);
32         return game;
33     }
34
```

Lines 20–33 `MidTurnGame`: a realistic state to save, with Alice mid-turn after one roll with two dice held.

```
35     // ---- Snapshots and restoring ----
36
```

Line 35 Section comment.

```
37      [Fact]
38      public void Snapshot_CapturesMidTurnState()
39      {
40          var snapshot = MidTurnGame().CreateSnapshot();
41
42          Assert.Equal(SaveGameSerializer.CurrentFormatVersion, snapshot.FormatVersion);
43          Assert.Equal(Start, snapshot.StartedAt);
44          Assert.Equal(0, snapshot.CurrentPlayerIndex);
45          Assert.Equal(1, snapshot.RollsUsed);
46          Assert.Equal(new[] { 3, 3, 6, 2, 1 }, snapshot.Dice);
47          Assert.Equal(new[] { true, true, false, false, false }, snapshot.Held);
48          Assert.Null(snapshot.Players[0].Strategy);
49          Assert.Equal("Scripted", snapshot.Players[1].Strategy);
50          Assert.Equal(25, snapshot.Players[0].Scores[Category.Fives]);
51          Assert.Null(snapshot.Players[0].Scores[Category.Aces]);
52          Assert.Equal(13, snapshot.Players[1].Scores.Count);
53      }
54
```

Lines 37–53 **Snapshot_CapturesMidTurnState**: The snapshot contains the version, start time, current player, roll count, dice, holds, strategies and scores.

```
55      [Fact]
56      public void Snapshot_BeforeFirstRoll_HasBlankDice()
57      {
58          var game = new GameEngine([new Player("Alice")], Fives(), new ManualTimeProvider(Start));
59
60          var snapshot = game.CreateSnapshot();
61
62          Assert.Equal(0, snapshot.RollsUsed);
63          Assert.Equal(new[] { 0, 0, 0, 0, 0 }, snapshot.Dice);
64      }
65
```

Lines 55–64 **Snapshot_BeforeFirstRoll_HasBlankDice**: Before rolling, the dice are saved as zeros.

```
66      [Fact]
67      public void Restore_ResumesExactlyWhereSaved()
68      {
69          var original = MidTurnGame();
70
71          var restored = Restore(original.CreateSnapshot());
72
73          Assert.Equal("Alice", restored.CurrentPlayer.Name);
74          Assert.Equal(2, restored.Round);
75          Assert.Equal(1, restored.RollsUsed);
76          Assert.Equal(original.Dice, restored.Dice);
77          Assert.Equal(original.Held, restored.Held);
78          Assert.Equal(Start, restored.StartedAt);
79          Assert.True(restored.Players[1].IsAi);
80          Assert.Equal(50, restored.Players[1].ScoreCard.GetScore(Category.Yahtzee));
81          Assert.True(restored.CanHold);
82      }
83
```

Lines 66–82 **Restore_ResumesExactlyWhereSaved**: Restoring gives the same player, round, rolls, dice, holds, start time, AI player and scores.

```
84      [Fact]
85      public void Restore_GameCanBePlayedToTheEnd()
86      {
87          var restored = Restore(MidTurnGame().CreateSnapshot());
88          int gameOverCount = 0;
89          restored.GameOver += (_, _) => gameOverCount++;
90
91          while (!restored.IsGameOver)
92          {
93              if (restored.CurrentPlayer.IsAi)
94              {
95                  while (restored.PlayAiStep() != AiStepResult.Scored) { }
96              }
97              else
98              {
99                  if (restored.RollsUsed == 0)
100                     restored.Roll();
101                 restored.Score(restored.CurrentPlayer.ScoreCard.AvailableCategories.First());
102             }
103         }
104
105         Assert.Equal(1, gameOverCount);
106         Assert.All(restored.Players, player => Assert.True(player.ScoreCard.IsComplete));
107     }
108
```

Lines 84–107 **Restore_GameCanBePlayedToTheEnd**: A restored game (including its AI player) can be played to a normal finish.

```
109     [Fact]
110     public void CreateSnapshot_AfterGameOver_Throws()
111     {
112         var game = new GameEngine([new Player("Alice")], Fives(), new ManualTimeProvider(Start));
113         foreach (var category in CategoryExtensions.All)
114         {
115             game.Roll();
116             game.Score(category);
117         }
118
119         Assert.False(game.CanSave);
120         Assert.Throws<InvalidOperationException>(game.CreateSnapshot);
121     }
122
```

Lines 109–121 **CreateSnapshot_AfterGameOver_Throws**: Finished games cannot be saved.

```
123     // ---- JSON format ----
124
```

Line 123 Section comment.

```
125     [Fact]
126     public void Json_RoundTripsAndIsReadable()
127     {
128         var snapshot = MidTurnGame().CreateSnapshot();
129
130         string json = SaveGameSerializer.Serialize(snapshot);
131         var reloaded = SaveGameSerializer.Deserialize(json);
132
133         Assert.Contains("\"formatVersion\": 1", json);
134         Assert.Contains("\"Fives\": 25", json);
135         Assert.Contains("\"strategy\": null", json);
136         Assert.Equal(snapshot.Dice, reloaded.Dice);
137         Assert.Equal(snapshot.Held, reloaded.Held);
138         Assert.Equal(snapshot.Players[1].Scores[Category.Yahtzee], reloaded.Players[1].Scores[Category.Yahtzee]);
139         Assert.Equal(snapshot.StartedAt, reloaded.StartedAt);
140     }
141
```

Lines 125–140 **Json_RoundTripsAndIsReadable**: JSON → snapshot → identical data, and the text contains readable names such as `"Fives": 25`.

```
142     [Fact]
143     public async Task SaveAsync_And_LoadAsync_UseTheFileSystem()
144     {
145         string path = Path.Combine(Path.GetTempPath(), $"yahtzee-test-{Guid.NewGuid():N}{SaveGameSerializer.FileExtension}");
146         try
147         {
148             var snapshot = MidTurnGame().CreateSnapshot();
149 
150             await SaveGameSerializer.SaveAsync(snapshot, path);
151             var loaded = await SaveGameSerializer.LoadAsync(path);
152 
153             Assert.Equal(snapshot.RollsUsed, loaded.RollsUsed);
154             Assert.False(File.Exists(path + ".tmp"));
155         }
156         finally
157         {
158             File.Delete(path);
159         }
160     }
161 
```

Lines 142–160 **SaveAsync_And_LoadAsync_UseTheFileSystem**: An `async` test writing and reading a real temporary file, and checking no `.tmp` file is left behind.

```
162     [Theory]
163     [InlineData("")]
164     [InlineData("not json")]
165     [InlineData("null")]
166     [InlineData("{}")]
167     [InlineData("""{ "formatVersion": 1 }""")]
168     public void Deserialize_RejectsMalformedFiles(string json) =>
169         Assert.Throws<InvalidSaveFileException>(() => SaveGameSerializer.Deserialize(json));
170 
```

Lines 162–169 **Deserialize_RejectsMalformedFiles**: Empty text, non-JSON, `null`, `{}` and a file with only a version are all rejected. The last case uses a raw string literal.

```
171     [Fact]
172     public void Deserialize_RejectsOtherFormatVersions()
173     {
174         string json = SaveGameSerializer.Serialize(MidTurnGame().CreateSnapshot()).Replace("\"formatVersion\": 1",
"\"formatVersion\": 99");
175 
176         var ex = Assert.Throws<InvalidSaveFileException>(() => SaveGameSerializer.Deserialize(json));
177         Assert.Contains("version 99", ex.Message);
178     }
179 
```

Lines 171–178 **Deserialize_RejectsOtherFormatVersions**: Version 99 is refused and named in the message.

```
180     [Fact]
181     public void Deserialize_RejectsNullPlayerName()
182     {
183         string json = SaveGameSerializer.Serialize(MidTurnGame().CreateSnapshot()).Replace("\"name\": \"Alice\"", "\"name\":
null");
184 
185         Assert.Throws<InvalidSaveFileException>(() => SaveGameSerializer.Deserialize(json));
186     }
187 
```

Lines 180–186 **Deserialize_RejectsNullPlayerName**: A null name is rejected by the strict JSON options.

```
188     // ---- Impossible games are rejected ----
189 
```

Line 188 Section comment. These tests edit snapshots with `with` expressions (non-destructive record copies).

```
190     [Fact]
191     public void Restore_RejectsScorecardsThatDisagreeWithTurnOrder()
192     {
193         var snapshot = MidTurnGame().CreateSnapshot();
194         var bobScores = new Dictionary<Category, int?>(snapshot.Players[1].Scores) { [Category.Chance] = 20 }; // Bob has played
twice
195         var tampered = snapshot with { Players = [snapshot.Players[0], snapshot.Players[1] with { Scores = bobScores }] };
196
197         var ex = Assert.Throws<InvalidSaveFileException>(() => Restore(tampered));
198         Assert.Contains("Bob", ex.Message);
199     }
200
```

Lines 190–199 **Restore_RejectsScorecardsThatDisagreeWithTurnOrder**: Giving Bob an extra score breaks the turn order; the message names Bob.

```
201     [Theory]
202     [InlineData(-1)]
203     [InlineData(4)]
204     public void Restore_RejectsInvalidRollCount(int rollsUsed) =>
205         Assert.Throws<InvalidSaveFileException>(() => Restore(MidTurnGame().CreateSnapshot() with { RollsUsed = rollsUsed }));
206
```

Lines 201–205 **Restore_RejectsInvalidRollCount**: −1 and 4 rolls are impossible.

```
207     [Fact]
208     public void Restore_RejectsInvalidDice() =>
209         Assert.Throws<InvalidSaveFileException>(() => Restore(MidTurnGame().CreateSnapshot() with { Dice = [3, 3, 9, 2, 1] }));
210
```

Lines 207–209 **Restore_RejectsInvalidDice**: A die showing 9 is rejected.

```
211     [Fact]
212     public void Restore_RejectsMissingCurrentPlayer() =>
213         Assert.Throws<InvalidSaveFileException>(() => Restore(MidTurnGame().CreateSnapshot() with { CurrentPlayerIndex = 2 }));
214
```

Lines 211–213 **Restore_RejectsMissingCurrentPlayer**: Index 2 in a two-player game is rejected.

```
215     [Fact]
216     public void Restore_RejectsUnknownStrategy()
217     {
218         var snapshot = MidTurnGame().CreateSnapshot();
219         var tampered = snapshot with { Players = [snapshot.Players[0], snapshot.Players[1] with { Strategy = "Cheater" }] };
220
221         var ex = Assert.Throws<InvalidSaveFileException>(() => Restore(tampered));
222         Assert.Contains("Cheater", ex.Message);
223     }
224
```

Lines 215–223 **Restore_RejectsUnknownStrategy**: An unknown strategy name is rejected and named.

```
225     [Fact]
226     public void Restore_RejectsDuplicateNames()
227     {
228         var snapshot = MidTurnGame().CreateSnapshot();
229         var tampered = snapshot with { Players = [snapshot.Players[0], snapshot.Players[1] with { Name = "ALICE" }] };
230
231         Assert.Throws<InvalidSaveFileException>(() => Restore(tampered));
232     }
233 }
```

Lines 225–233 **Restore_RejectsDuplicateNames**: "Alice" and "ALICE" cannot both play.

12.9 Database tests

`tests/Yahtzee.Core.Tests/GameHistoryRepositoryTests.cs · 102 lines`

Runs against a real SQLite file in the temp folder, a fresh one for every test.

```
1  using Microsoft.Data.Sqlite;
2  using Yahtzee.Core.Data;
3  using Yahtzee.Core.Game;
4
```

Lines 1–3 Imports.

```
5  namespace Yahtzee.Core.Tests;
6
```

Line 5 Test namespace.

```
7  /// <summary>Runs each test against a fresh database file in the temp folder, deleted afterwards.</summary>
8  public sealed class GameHistoryRepositoryTests : IDisposable
9  {
```

Lines 7–9 xUnit creates a new instance per test and calls `Dispose` afterwards (`IDisposable`).

```
10     private static readonly DateTimeOffset Day1 = new(2026, 10, 1, 18, 0, 0, TimeSpan.Zero);
11
12     private readonly string _path = Path.Combine(Path.GetTempPath(), $"yahtzee-test-{Guid.NewGuid():N}.db");
13     private readonly SqliteGameHistoryRepository _repository;
14
```

Lines 10–13 A fixed date, a unique temp file name, and the repository under test.

```
15     public GameHistoryRepositoryTests() => _repository = new SqliteGameHistoryRepository(_path);
16
```

Line 15 Constructor = per-test setup.

```
17     public void Dispose()
18     {
19         SqliteConnection.ClearAllPools(); // pooled connections keep the file open
20         File.Delete(_path);
21     }
22
```

Lines 17–21 Teardown: release pooled connections, then delete the file.

```
23     private static CompletedGame Game(DateTimeOffset playedAt, params (string Name, int Score, bool IsAi)[] results) =>
24         new(playedAt, [.. results.Select(r => new PlayerStanding(0, r.Name, r.Score, r.IsAi))]);
25
```

Lines 23–24 Helper that builds a game from (name, score, isAi) tuples.

```
26     [Fact]
27     public async Task EmptyDatabase_HasNoScores()
28     {
29         Assert.Empty(await _repository.GetTopScoresAsync(5));
30         Assert.Empty(await _repository.GetPlayerNamesAsync());
31         Assert.True(File.Exists(_path));
32     }
33
```

Lines 26–32 **EmptyDatabase_HasNoScores**: A new database is empty, and the file has been created.

```
34      [Fact]
35      public async Task TopScores_AreBestFiveAcrossAllGames()
36      {
37          await _repository.RecordGameAsync(Game(Day1, ("Alice", 180, false), ("Bob", 210, false)));
38          await _repository.RecordGameAsync(Game(Day1.AddDays(1), ("Alice", 250, false), ("Bot", 190, true)));
39          await _repository.RecordGameAsync(Game(Day1.AddDays(2), ("Carol", 120, false), ("Bob", 230, false)));
40
41          var top = await _repository.GetTopScoresAsync(5);
42
43          Assert.Equal(new[] { 250, 230, 210, 190, 180 }, top.Select(score => score.Score));
44          Assert.Equal("Alice", top[0].PlayerName);
45          Assert.Equal(Day1.AddDays(1), top[0].PlayedAt);
46          Assert.True(top[3].IsAi);
47      }
48
```

Lines 34–47 **TopScores_AreBestFiveAcrossAllGames**: Six results over three games → the best five in order, with the right date and AI flag.

```
49      [Fact]
50      public async Task TopScoresForPlayer_FiltersByNameIgnoringCase()
51      {
52          await _repository.RecordGameAsync(Game(Day1, ("Alice", 180, false), ("Bob", 210, false)));
53          await _repository.RecordGameAsync(Game(Day1.AddDays(1), ("alice", 250, false)));
54
55          var alice = await _repository.GetTopScoresForPlayerAsync("ALICE", 5);
56
57          Assert.Equal(new[] { 250, 180 }, alice.Select(score => score.Score));
58      }
59
```

Lines 49–58 **TopScoresForPlayer_FiltersByNameIgnoringCase**: "ALICE" finds the scores of "Alice" and "alice".

```
60      [Fact]
61      public async Task TopScores_RespectsCountAndBreaksTiesByDate()
62      {
63          for (int day = 0; day < 7; day++)
64              await _repository.RecordGameAsync(Game(Day1.AddDays(day), ("Alice", 200, false)));
65
66          var top = await _repository.GetTopScoresAsync(5);
67
68          Assert.Equal(5, top.Count);
69          Assert.Equal(Day1, top[0].PlayedAt); // the earliest of the tied scores comes first
70      }
71
```

Lines 60–70 **TopScores_RespectsCountAndBreaksTiesByDate**: Seven equal scores → five returned, earliest first.

```
72      [Fact]
73      public async Task PlayerNames_AreDistinctAndSorted()
74      {
75          await _repository.RecordGameAsync(Game(Day1, ("Carol", 1, false), ("Alice", 2, false)));
76          await _repository.RecordGameAsync(Game(Day1, ("Carol", 3, false), ("Bob", 4, false)));
77
78          Assert.Equal(new[] { "Alice", "Bob", "Carol" }, await _repository.GetPlayerNamesAsync());
79      }
80
```

Lines 72–79 **PlayerNames_AreDistinctAndSorted**: Each name once, alphabetically.

```
81      [Fact]
82      public async Task PlayerNamesWithQuotes_AreStoredSafely()
83      {
84          const string trickyName = "O'Brien\"; DROP TABLE Games; --";
85          await _repository.RecordGameAsync(Game(Day1, (trickyName, 100, false)));
86
87          var scores = await _repository.GetTopScoresForPlayerAsync(trickyName, 5);
88
89          Assert.Equal(trickyName, Assert.Single(scores).PlayerName);
90          Assert.Single(await _repository.GetTopScoresAsync(5)); // the Games table is still there
91      }
92
```

Lines 81–91 **PlayerNamesWithQuotes_AreStoredSafely**: A name containing quotes and an SQL-injection attempt is stored and found literally, and the `Games` table still exists: parameterised queries work.

```
93      [Fact]
94      public async Task DataSurvivesANewRepositoryInstance()
95      {
96          await _repository.RecordGameAsync(Game(Day1, ("Alice", 123, false)));
97
98          var reopened = new SqliteGameHistoryRepository(_path);
99
100         Assert.Equal(123, Assert.Single(await reopened.GetTopScoresAsync(5)).Score);
101     }
102 }
```

Lines 93–102 **DataSurvivesANewRepositoryInstance**: Data is really on disk: a second repository on the same file sees it.

12.10 AI tests

`tests/Yahtzee.Core.Tests/AiStrategyTests.cs · 107 lines`

Checks that every strategy plays legal, complete games, that stronger strategies really score more, and specific decisions.

```
1  using Xunit.Abstractions;
2  using Yahtzee.Core.AI;
3  using Yahtzee.Core.Game;
4  using Yahtzee.Core.Model;
5
```

Lines 1–4 Imports; `ITestOutputHelper` lets a test print text into the test log.

```
6  namespace Yahtzee.Core.Tests;
7
```

Line 6 Test namespace.

```
8  public class AiStrategyTests(ITestOutputHelper output)
9  {
```

Lines 8–9 xUnit supplies the output helper through the primary constructor.

```
10     /// <summary>Plays complete one-player games with the given strategy and returns the average score.</summary>
11     private static double AverageScore(Func<int, IPlayerStrategy> createStrategy, int games)
12     {
13         int total = 0;
14         for (int seed = 0; seed < games; seed++)
15         {
16             // Seeded dice make every run of the test identical.
17             var game = new GameEngine([new Player("Bot", createStrategy(seed))], new RandomDiceRoller(new Random(seed)));
18             while (!game.IsGameOver)
19                 game.PlayAiStep(); // throws if the strategy ever makes an illegal move
20             total += game.Players[0].Total;
21         }
22         return (double)total / games;
23     }
24
```

Lines 10–23 `AverageScore`: plays whole games using only `PlayAiStep`; the engine would throw on any illegal move. Seeded dice make the results identical on every run.

```
25     // ---- Every strategy plays legal, complete games ----
26
```

Line 25 Section comment.

```
27     [Theory]
28     [InlineData(RandomStrategy.StrategyName)]
29     [InlineData(GreedyStrategy.StrategyName)]
30     [InlineData(ExpectedValueStrategy.StrategyName)]
31     public void Strategy_PlaysCompleteLegalGames(string name)
32     {
33         double average = AverageScore(_ => StrategyCatalog.Create(name), games: 10);
34
35         Assert.InRange(average, 5, 375); // 375 is far above any realistic total under these rules
36     }
37
```

Lines 27–36 **Strategy_PlaysCompleteLegalGames**: Each strategy (by catalog name) completes 10 games with a plausible average.

```
38      [Fact]
39      public void StrongerStrategies_ScoreMoreOnAverage()
40      {
41          double random = AverageScore(seed => new RandomStrategy(new Random(seed + 1000)), games: 30);
42          double greedy = AverageScore(_ => new GreedyStrategy(), games: 30);
43          double expected = AverageScore(_ => new ExpectedValueStrategy(), games: 30);
44
45          output.WriteLine($"Average over 30 games: random {random:F1}, greedy {greedy:F1}, expected value {expected:F1}");
46          Assert.True(greedy > random + 30, $"greedy {greedy:F1} vs random {random:F1}");
47          Assert.True(expected > greedy, $"expected value {expected:F1} vs greedy {greedy:F1}");
48      }
49
```

Lines 38–48 **StrongerStrategies_ScoreMoreOnAverage**: 30 games each. Measured: random 39, greedy 176, expected value 228. Greedy must beat random by 30+ points, and expected value must beat greedy. The averages are written to the test output.

```
50      // ---- Greedy decisions ----
51
```

Line 50 Section comment.

```
52      [Fact]
53      public void Greedy_KeepsMostCommonFace() =>
54          Assert.Equal(new[] { false, true, true, false, true }, new GreedyStrategy().ChooseHolds([2, 5, 5, 1, 5], 2, new
ScoreCard()));
55
```

Lines 52–54 **Greedy_KeepsMostCommonFace**: 2 5 5 1 5 → hold the three 5s.

```
56      [Fact]
57      public void Greedy_KeepsRunOfFourWhenStraightIsOpen() =>
58          Assert.Equal(new[] { true, true, true, true, false }, new GreedyStrategy().ChooseHolds([1, 2, 3, 4, 6], 2, new
ScoreCard()));
59
```

Lines 56–58 **Greedy_KeepsRunOfFourWhenStraightIsOpen**: 1 2 3 4 6 → hold 1-2-3-4, re-roll the 6.

```
60      [Fact]
61      public void Greedy_StopsOnYahtzee() =>
62          Assert.Null(new GreedyStrategy().ChooseHolds([4, 4, 4, 4, 4], 2, new ScoreCard()));
63
```

Lines 60–62 **Greedy_StopsOnYahtzee**: Five 4s → stop.

```
64      [Fact]
65      public void Greedy_ScoresHighestPoints() =>
66          Assert.Equal(Category.LargeStraight, new GreedyStrategy().ChooseCategory([2, 3, 4, 5, 6], new ScoreCard()));
67
```

Lines 64–66 **Greedy_ScoresHighestPoints**: 2-6 straight → Large Straight (40).

```
68      [Fact]
69      public void Greedy_SacrificesCheapestBoxWhenNothingScores()
70      {
71          // With Twos, Fours, Sixes, Chance and both "of a kind" boxes filled, 2 2 4 4 6 scores 0 in every open box,
72          // so Greedy gives up the box that is usually worth least: Aces.
73          var card = new ScoreCard(new Dictionary<Category, int> { [Category.Twos] = 4, [Category.Fours] = 8, [Category.Sixes] =
18,
74              [Category.Chance] = 20, [Category.ThreeOfAKind] = 20, [Category.FourOfAKind] = 0 });
75
76          Assert.Equal(Category.Aces, new GreedyStrategy().ChooseCategory([2, 2, 4, 4, 6], card));
77      }
78
```

Lines 68–77 **Greedy_SacrificesCheapestBoxWhenNothingScores**: Everything open scores 0 → give up Aces.

```
79      // ---- Expected-value decisions ----
80
```

Line 79 Section comment.

```
81      [Fact]
82      public void ExpectedValue_StopsOnYahtzee() =>
83          Assert.Null(new ExpectedValueStrategy().ChooseHolds([6, 6, 6, 6, 6], 2, new ScoreCard()));
84
```

Lines 81–83 **ExpectedValue_StopsOnYahtzee**: Five 6s → stop.

```
85      [Fact]
86      public void ExpectedValue_KeepsFourOfAKindAndRerollsTheOddDie()
87      {
88          var holds = new ExpectedValueStrategy().ChooseHolds([5, 5, 2, 5, 5], 2, new ScoreCard());
89
90          Assert.Equal(new[] { true, true, false, true, true }, holds);
91      }
92
```

Lines 85–91 **ExpectedValue_KeepsFourOfAKindAndRerollsTheOddDie**: 5 5 2 5 5 → re-roll only the 2.

```
93      [Fact]
94      public void ExpectedValue_SavesChanceForLaterRatherThanWastingIt() =>
95          // 1 1 2 3 5 scores 12 in Chance (10 below par) but only 2 in Aces (par 2): Aces is the better use.
96          Assert.Equal(Category.Aces, new ExpectedValueStrategy().ChooseCategory([1, 1, 2, 3, 5], new ScoreCard()));
97
```

Lines 93–96 **ExpectedValue_SavesChanceForLaterRatherThanWastingIt**: 1 1 2 3 5 → Aces (at par) rather than 12 in Chance (10 below par).

```
98      // ---- Catalog ----
99
```

Line 98 Section comment.

```
100     [Fact]
101     public void Catalog_CreatesEveryListedStrategy() =>
102         Assert.All(StrategyCatalog.Names, name => Assert.Equal(name, StrategyCatalog.Create(name).Name));
103
```

Lines 100–102 **Catalog_CreatesEveryListedStrategy**: Every listed name creates a strategy reporting that name.

```
104     [Fact]
105     public void Catalog_RejectsUnknownName() =>
106         Assert.Throws<ArgumentException>(() => StrategyCatalog.Create("Cheater"));
107 }
```

Lines 104–107 **Catalog_RejectsUnknownName**: Unknown names throw.

13. What remains: report, video, lab test and submission

All program features are finished. What is left is yours to do, and it is worth 20 marks directly (report 15, video 5). The video also decides whether the 75 feature and quality marks count.

1. Test the executable on a lab machine

- Copy `publish\Yahtzee.exe` to a Windows 11 lab PC (no installation needed) and play a short game, save, load, check the high scores.
- Rebuild it at any time with `dotnet publish src/Yahtzee.WinForms -p:PublishProfile=Standalone`.

2. Your report (15 marks), max 10 pages, required structure

1. **Introduction** (purpose, remit, assumptions, e.g. at most 6 players, simplified rules, Windows only).
2. **Requirements checklist** with **exactly** the Appendix A rows (the table in section 1 has them all as fully implemented).
3. **Design considerations**: section 5.6 lists the decisions and reasons; explain them in your own words.
4. **Developer guide**: classes and responsibilities (5.2), interfaces, **save-file format** (5.7), **database organisation** (5.8).
5. **GenAI usage statement**, with your prompts and the model's responses in an appendix (source: `docs/genai-log.md`; fill in the "my evaluation" column yourself, since the policy requires you to evaluate the output).
6. **Conclusion**: what you are proud of, what you would do differently.
7. References (optional).

Declare the external dependency: `Microsoft.Data.Sqlite` 10.0.12 (bundling SQLitePCLRaw / native SQLite).

3. Your video (5 marks, mandatory)

Tip: start the program with `Yahtzee.exe --database C:\temp\video.db` for a clean high-score table, then narrate:

1. New game: name the players, mix a human and a computer (and show the three computer levels).
2. A turn: whose turn, rolls left, rolling, holding dice (HELD / WILL ROLL), the score preview, scoring.
3. The computer taking its turn.
4. Save after a roll (Ctrl+S), close the program: show the save prompt (try Cancel, then exit), reopen with Ctrl+O and continue from the same roll.
5. A single-player game too (it can be quick: a few rounds, then show a finished one).
6. Finish a game: the results summary.
7. High scores (Ctrl+H): top 5 overall, top 5 of a player, with dates.

4. Submission checklist

Item	Done?
Zip: source code without bin/ obj/ .vs/ (docs/guide-src/build/ can be left out too)	☐
Zip: standalone executable (publish\Yahtzee.exe)	☐
Zip: report PDF	☐
Video uploaded to Canvas or OneDrive link in the report	☐
Executable tested on a lab Windows 11 machine	☐
GenAI appendix with prompts, responses and your evaluation	☐
Able to explain every file (this guide)	☐
Submitted on Canvas before 29 October 2026	☐

14. Glossary

Class library	A project compiled to a DLL that other projects use; it cannot run on its own.
DPI / display scaling	Windows enlarges the UI on high-resolution screens (125%, 150%...); programs must scale pixel sizes accordingly.

async / await	Lets slow work (disk, database) run without freezing the window; `await` resumes the method when the work is done.
Event	A notification a class raises ("the dice were rolled") that other code can subscribe to with a handler method.
Expected value	The average result over all possible outcomes, weighted by their probability.
Hotseat	Several people taking turns on the same computer.
Immutable	Cannot be changed after creation (e.g. records, get-only properties).
Interface	A contract listing members a class must implement; code can depend on the contract instead of a concrete class.
Modal dialog	A window that blocks the window behind it until closed (`ShowDialog`).
Parameterised query	SQL whose values are sent separately from the SQL text, which prevents SQL injection.
Partial class	One class whose code is split across several files.
Pure function	Output depends only on input; no side effects.
Repository	An object that hides how data is stored behind an interface of meaningful operations.
Self-contained publish	An executable that carries its own .NET runtime, so the PC needs nothing installed.
Snapshot	A copy of the complete game state at one moment, used for saving.
State machine	A design where an object is in one of a few states and only certain actions move it between them.
Strategy pattern	Interchangeable algorithms behind one interface (here: AI players).
Transaction	A group of database changes that are stored all together or not at all.
Test double	A stand-in object used in tests (fixed dice, fake clock, scripted AI).
UI Automation	The Windows accessibility API; used here by test scripts to click the real application and take screenshots.
xUnit [Fact] / [Theory]	A single test / a test run once per row of `[InlineData]`.